



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería Informática

Herramientas para la monitorización y análisis de procesos de modelado 3D en Blender

Documentación técnica



Presentado por María Molina Goyena
Universidad de Burgos — 6 de junio de 2026
Tutores: Dr. Bruno Rodríguez García y Dr. Raúl Marticorena Sánchez

Índice general

Índice general	i
Índice de figuras	iv
Índice de tablas	vii
A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Viabilidad del proyecto	20
A.4. Análisis de riesgos	25
A.5. Conclusión del plan de proyecto	26
B Especificación de Requisitos	29
B.1. Introducción	29
B.2. Objetivos generales	29
B.3. Catálogo de requisitos	30
B.4. Trazabilidad de requisitos	37
B.5. Casos de uso del sistema	38
B.6. Trazabilidad entre casos de uso y requisitos	46
C Especificación de Diseño	47
C.1. Introducción	47
C.2. Diseño de datos	47
C.3. Diseño arquitectónico	52
C.4. Diseño del primer <i>add-on</i> : Data Logger 3D	57
C.5. Diseño del segundo <i>add-on</i> : análisis y visualización	60

C.6. Diseño procedimental	63
C.7. Diseño de métricas A1–A16	64
C.8. Diseño de métricas geométricas y UV	64
C.9. Decisiones de diseño relevantes	65
D Documentación técnica de programación	67
D.1. Introducción	67
D.2. Estructura de directorios	67
D.3. Preparación del entorno de desarrollo	68
D.4. Primer add-on: <code>Data_Logger_3D.py</code>	71
D.5. Segundo add-on: análisis y visualización	76
D.6. Tabla completa de funciones	79
D.7. Instalación, ejecución y recarga en desarrollo	84
D.8. Dependencias externas	84
D.9. Licencia del software desarrollado	85
D.10. Pruebas del sistema	86
D.11. Mantenimiento y ampliación	90
E Manual de usuario	93
E.1. Introducción	93
E.2. Requisitos del sistema	93
E.3. Instalación y configuración	94
E.4. Uso del Data Logger 3D	98
E.5. Uso del add-on de análisis	107
E.6. Interpretación de métricas	122
E.7. Diagnóstico y resolución de incidencias	122
E.8. Consideraciones éticas y privacidad	122
E.9. Recomendaciones para análisis de datos	123
E.10. Resumen operativo	123
F Anexo de sostenibilización curricular	125
F.1. Introducción	125
F.2. Relación con los Objetivos de Desarrollo Sostenible	125
F.3. Sostenibilidad tecnológica	126
F.4. Sostenibilidad educativa	127
F.5. Sostenibilidad social y ética	127
F.6. Sostenibilidad económica	128
F.7. Limitaciones desde la perspectiva de sostenibilidad	129
F.8. Reflexión personal	129
F.9. Conclusión	130

Índice general

III

Bibliografía

131

Índice de figuras

A.1. Burndown global del proyecto.	3
A.2. Burndown del Sprint 1.	5
A.3. Burndown del Sprint 2.	6
A.4. Burndown del Sprint 3.	7
A.5. Burndown del Sprint 4.	8
A.6. Burndown del Sprint 5.	10
A.7. Burndown del Sprint 6.	11
A.8. Burndown del Sprint 7.	13
A.9. Burndown del Sprint 8.	14
A.10. Diagrama de flujo del sistema de captura y análisis planificado.	19
B.1. Diagrama de casos de uso del sistema de monitorización, análisis y visualización.	39
C.1. Flujo general de la <i>suite</i> : captura, exportación CSV, análisis de métricas y visualización en Blender.	53
C.2. Organización general de componentes y comunicación entre el logger, el CSV y el <i>add-on</i> de análisis.	54
C.3. Estructura modular del código y separación entre lógica de cálculo, interfaz y visualización.	56
C.4. Secuencia de interacción completa entre Blender, el <i>add-on</i> Data Logger 3D, el CSV intermedio y el <i>add-on</i> de análisis y visualización.	57
C.5. Flujo interno de detección de cambios del Data Logger 3D durante una sesión de modelado.	59
C.6. Flujo de persistencia y exportación del CSV generado por el Data Logger 3D.	59
C.7. Secuencia de análisis, cálculo de métricas y visualización de resultados en el segundo <i>add-on</i>	62

D.1. Panel principal del add-on Data Logger 3D con controles de captura y exportación.	74
D.2. Panel de depuración del add-on Data Logger 3D durante una sesión de captura.	75
D.3. Ejemplo de gráfico de interacción temporal. La diferencia de escala entre métricas hace que la velocidad de navegación domine visualmente la representación.	78
E.1. Ventana de preferencias de Blender para la instalación de add-ons.	95
E.2. Botón de instalación de add-ons desde las preferencias de Blender.	95
E.3. Add-on Data Logger 3D instalado y activado en Blender.	96
E.4. Add-on de análisis y visualización instalado y activado en Blender.	97
E.5. Configuración de Auto Run Python Scripts en las preferencias de Blender.	98
E.6. Ubicación de la pestaña Data Logger en la barra lateral del visor 3D.	100
E.7. Ventana de consentimiento mostrada antes de iniciar la captura de datos técnicos.	101
E.8. Panel principal del Data Logger 3D al parar la captura.	102
E.9. Panel principal del Data Logger 3D durante una sesión de captura activa.	103
E.10. Panel de depuración del Data Logger 3D con información del último evento registrado.	104
E.11. Botón de exportación del CSV generado por el Data Logger 3D.	105
E.12. Acceso al editor de texto de Blender para consultar los bloques internos del archivo.	106
E.13. Contenido CSV almacenado como bloque de texto interno dentro del archivo de Blender.	106
E.14. Archivo CSV exportado en la misma carpeta que el proyecto de Blender.	107
E.15. Pestaña de análisis CSV del add-on de análisis y visualización. .	108
E.16. Pestaña de métricas de malla del add-on de análisis y visualización.	109
E.17. Pestaña de gráficos y visualizaciones del add-on de análisis y visualización.	111
E.18. Selección de un archivo CSV generado por el Data Logger 3D. .	113
E.19. Selección de una carpeta con varios CSV para análisis comparativo.	114
E.20. Acción de cálculo de métricas estadísticas a partir del CSV seleccionado.	115
E.21. Proceso de cálculo y visualización de métricas estadísticas del CSV en el add-on de análisis.	116

E.22. Selección del objeto activo y el de referencia para el cálculo de métricas geométricas y UV.	118
E.23. Resultados de métricas UV, geométricas y de similitud calculadas sobre el objeto seleccionado.	119
E.24. Visualización temporal generada a partir de los datos de una sesión.	120
E.25. Comparación visual de varias sesiones a partir de múltiples archivos CSV.	121
E.26. Gráfico radar generado para representar métricas normalizadas de una sesión.	121

Índice de tablas

A.1. Resumen cronológico de los sprints del proyecto.	16
A.2. Revisión resumida de ejecución por sprint a partir de la planificación.	17
A.3. Tabla comparativa de hitos temporales y desviaciones.	18
A.4. Estimación de costes del proyecto.	24
A.5. Matriz de riesgos técnicos, funcionales y éticos del proyecto. . .	26
B.1. Trazabilidad resumida entre requisitos, bloques funcionales y responsabilidades de implementación.	38
B.2. CU-01: Inicializar monitorización no intrusiva.	39
B.3. CU-02: Auditar cambios topológicos de la malla.	40
B.4. CU-03: Detectar operadores de teclado relevantes.	40
B.5. CU-04: Capturar transformaciones espaciales del objeto activo. .	41
B.6. CU-05: Registrar cambios de modo y contexto de trabajo. . . .	41
B.7. CU-06: Gestionar consentimiento y seudonimización.	42
B.8. CU-07: Persistir y exportar el registro CSV.	42
B.9. CU-08: Cargar y validar colecciones de CSV.	43
B.10.CU-09: Calcular métricas estadísticas A1–A16.	43
B.11.CU-10: Calcular similitud geométrica y distancia de Hausdorff. .	44
B.12.CU-11: Analizar mapas UV.	44
B.13.CU-12: Diagnosticar normales, ángulos y estructura de malla. .	45
B.14.CU-13: Generar gráficos estadísticos 2D.	45
B.15.CU-14: Crear visualizaciones procedurales 3D en Blender. . . .	46
B.16.Trazabilidad entre casos de uso y requisitos.	46
C.1. Campos completos del CSV utilizado como formato intermedio entre captura y análisis.	49

C.2. Diferencias entre los campos de identificación de usuario en las versiones del esquema CSV.	51
C.3. Variables derivadas principales calculadas por el <i>add-on</i> de análisis.	52
C.4. Responsabilidades principales de la arquitectura.	54
C.5. Capas lógicas utilizadas para organizar el diseño de la suite.	55
C.6. Diseño interno del <i>add-on</i> Data Logger 3D.	58
C.7. Estados principales del diseño de ejecución del Data Logger 3D.	60
C.8. Diseño interno del <i>add-on</i> de análisis y visualización.	61
C.9. Transformación de entradas y salidas en el diseño del <i>add-on</i> de análisis.	63
C.10. Diseño de presentación de las métricas A1–A16.	64
C.11. Diseño de métricas geométricas y UV.	65
C.12. Decisiones de diseño relevantes de la <i>suite</i>	66
D.1. Funciones y clases principales del <i>add-on</i> Data Logger 3D.	80
D.2. Funciones principales de cálculo y validación del <i>add-on</i> de análisis.	82
D.3. Funciones y grupos de interfaz/visualización del <i>add-on</i> de análisis.	83
D.4. Resumen del análisis de licencias de los componentes utilizados	85
D.5. Resumen de pruebas funcionales, de calidad y compatibilidad del sistema	87
E.1. Requisitos básicos para utilizar la <i>suite</i>	94
E.2. Lectura práctica de los bloques A1–A16.	115
E.3. Problemas frecuentes y soluciones recomendadas.	122
F.1. Relación del proyecto con distintos Objetivos de Desarrollo Sostenible.	126

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Este apartado describe la planificación, organización, viabilidad y gestión de riesgos seguidas durante el desarrollo del proyecto. El objetivo de este anexo es documentar no solo la planificación inicial, sino también la evolución real del trabajo a lo largo de las distintas iteraciones, considerando las desviaciones, ajustes técnicos y decisiones de refactorización que se produjeron durante el desarrollo.

El sistema desarrollado se ha estructurado como una *suite* de software formada por dos complementos o *add-ons* independientes y complementarios para el entorno de modelado 3D Blender [2]:

- **Data Logger 3D o módulo de recopilación:** encargado de registrar automáticamente y en segundo plano la actividad técnica del usuario durante sesiones de modelado 3D. Este *add-on* captura eventos, estados, transformaciones espaciales, cambios topológicos, operaciones relevantes, cambios de modo, modificaciones UV y otros indicadores asociados al proceso de modelado. Los datos se almacenan en formato CSV para su posterior análisis.
- **Add-on de análisis y visualización:** encargado de procesar de forma diferida los CSV generados por el logger. Este segundo componente lee y valida los datos, calcula métricas estadísticas, métricas geométricas, métricas UV y genera resultados visuales mediante paneles, gráficos y visualizaciones integradas en Blender.

Esta división modular permite separar la fase de adquisición de datos de la fase de interpretación. La captura debe ser ligera, estable y poco intrusiva, ya que se ejecuta durante la sesión de modelado. En cambio, el análisis puede incorporar operaciones más costosas, como cálculos estadísticos, comparaciones geométricas, generación de gráficos y visualizaciones 3D, porque se ejecuta posteriormente y bajo demanda.

El proyecto se ha llevado a cabo mediante un **enfoque iterativo e incremental** [17, 19], construyendo inicialmente versiones de funcionalidad mínima viable que posteriormente han sido refinadas y ampliadas. Esta estrategia permitió detectar problemas de forma temprana, especialmente en relación con la API de Blender, la gestión de modos de edición, la captura de eventos y la validación de métricas. Asimismo, facilitó que el alcance evolucionara desde un registrador básico de actividad hacia una *suite* completa de captura, análisis y visualización.

Para la gestión del código se ha utilizado GitHub [9], lo que ha permitido mantener un historial de commits con la evolución del proyecto. Este historial se ha empleado como fuente de contraste para revisar la planificación y comprobar qué tareas fueron ejecutadas realmente en cada etapa. La documentación se ha redactado y maquetado en L^AT_EX [10], siguiendo la estructura institucional del Trabajo de Fin de Grado.

A.2. Planificación temporal

El desarrollo se organizó en ciclos de trabajo o *sprints*, con una orientación aproximada quincenal. Cada iteración incorporó nuevas funcionalidades o mejoras sobre la base existente. La planificación inicial partía de una evolución progresiva: primero un logger funcional, después la normalización del CSV, posteriormente el análisis de datos y finalmente la visualización, validación y refinamiento de interfaz.

No obstante, la ejecución real mostró que varias tareas tuvieron que redistribuirse entre sprints. La razón principal fue la complejidad técnica del entorno Blender, especialmente en la detección de eventos internos, el comportamiento diferente entre *Object Mode* y *Edit Mode*, la captura de UV y el cálculo correcto de métricas a partir de CSV reales. Por ello, la planificación se mantuvo como una guía de trabajo, pero la ejecución siguió un patrón incremental y adaptativo.

La planificación temporal se resume visualmente en la Figura A.1, que muestra el burndown global del proyecto.

La evolución del trabajo durante el Sprint 1 se muestra en la Figura A.2.

La evolución del trabajo durante el Sprint 2 se muestra en la Figura A.3.

La evolución del trabajo durante el Sprint 3 se muestra en la Figura A.4.

La evolución del trabajo durante el Sprint 4 se muestra en la Figura A.5.

La evolución del trabajo durante el Sprint 5 se muestra en la Figura A.6.

La evolución del trabajo durante el Sprint 6 se muestra en la Figura A.7.

La evolución del trabajo durante el Sprint 7 se muestra en la Figura A.8.

La evolución del trabajo durante el Sprint 8 se muestra en la Figura A.9.

El flujo general de captura y análisis considerado en la planificación se representa en la Figura A.10.

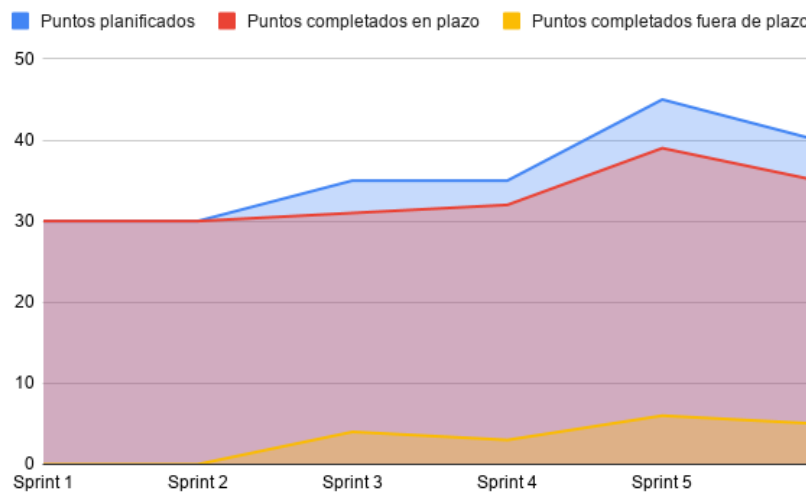


Figura A.1: Burndown global del proyecto.

Fase previa

Esta fase se desarrolló durante el mes de noviembre.

Durante la fase previa se realizó el estudio inicial de viabilidad técnica y el análisis del entorno de ejecución de Blender. Se revisaron las posibilidades ofrecidas por la API `bpy`, los operadores internos de Blender, los manejadores persistentes y los mecanismos de acceso a objetos, mallas, modos de edición y transformaciones.

El objetivo previsto era comprobar si resultaba viable registrar actividad técnica de una sesión de modelado sin interferir en el flujo de trabajo del usuario. Esta fase permitió confirmar que la solución era viable, aunque también se detectó una limitación importante: Blender no proporciona todos los eventos de modelado de forma directa y homogénea. Esta dificultad condicionó decisiones posteriores, como la comparación de estados mediante *snapshots*, el uso de temporizadores, la detección de operadores recientes y el empleo de `bmesh` para acceder correctamente a mallas en modo edición.

Sprint 0: definición inicial del proyecto

Esta fase se desarrolló durante enero.

El Sprint 0 se dedicó a la definición del dominio del problema, el alcance funcional y la arquitectura inicial. En esta etapa se planteó la separación del sistema en dos *add-ons* independientes:

- un primer *add-on* centrado en la recopilación de datos durante el modelado;
- un segundo *add-on* orientado al análisis posterior de los registros generados.

La planificación prevista consistía en delimitar requisitos, establecer el formato de intercambio y definir el flujo general de datos. La ejecución real confirmó la validez de esta separación, aunque el desarrollo posterior mostró que el segundo *add-on* tendría un peso mayor del inicialmente previsto. El módulo de análisis no se limitó a leer CSV, sino que terminó incorporando validación de datos, métricas estadísticas, métricas geométricas, métricas UV, gráficos, visualizaciones y una interfaz organizada por pestañas.

Sprint 1: primera versión funcional del Data Logger 3D

Esta fase se desarrolló durante la primera y segunda semana de febrero.

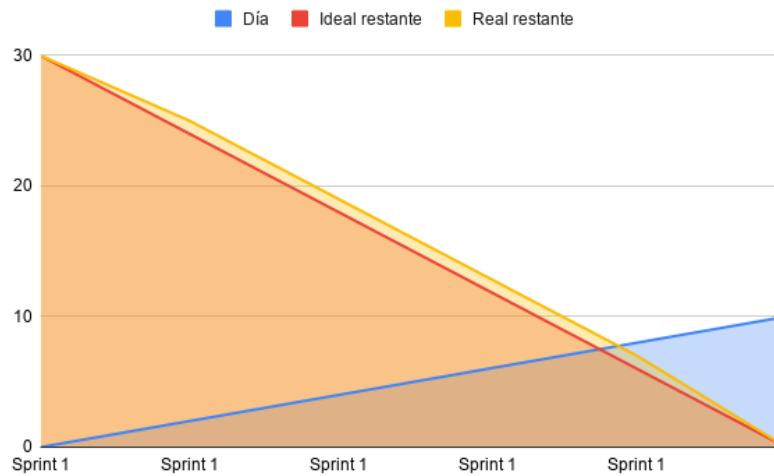


Figura A.2: Burndown del Sprint 1.

El primer sprint de desarrollo se centró en crear una primera versión funcional del *add-on* de captura. El objetivo previsto era disponer de un prototipo mínimo capaz de ejecutarse dentro de Blender, mostrar una interfaz básica y registrar datos en formato CSV.

Las tareas previstas fueron:

- creación de la estructura inicial del *add-on*;
- definición de operadores básicos;
- incorporación de una interfaz gráfica inicial;
- captura de eventos o estados relevantes;
- escritura inicial de registros en CSV.

La ejecución real se ajustó de forma razonable a lo planificado. El historial de commits refleja una primera versión de *Data Logger 3D* y una interfaz gráfica básica durante el mes de febrero. Esta etapa permitió obtener un prototipo inicial funcional, aunque todavía limitado en cuanto a precisión, robustez y normalización de datos.

El resultado principal fue una primera base operativa sobre la que se pudieron construir las versiones posteriores. Sin embargo, la experiencia

adquirida en este sprint también evidenció que la captura de datos en Blender requería una aproximación más cuidadosa que una simple lectura periódica de valores. Era necesario distinguir cambios reales de estados repetidos, evitar registros redundantes y controlar el impacto en el rendimiento.

Sprint 2: refinamiento de captura y normalización del CSV

Esta fase se desarrolló durante la tercera y cuarta semana de febrero.

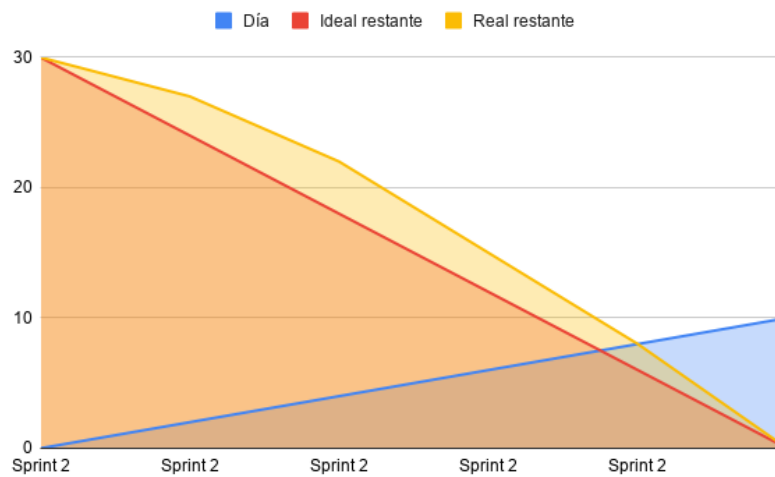


Figura A.3: Burndown del Sprint 2.

El segundo sprint se planificó como una fase de refinamiento del logger. El objetivo principal era mejorar la captura de eventos, reorganizar los datos registrados y avanzar hacia una estructura CSV más estable.

Las tareas previstas fueron:

- corregir errores de versiones iniciales;
- mejorar el registro de eventos significativos;
- reorganizar archivos del proyecto;
- avanzar en la normalización del CSV;
- preparar el formato de datos para el futuro *add-on* de análisis.

La ejecución real se centró especialmente en corregir versiones tempranas y reorganizar el código. En esta fase se realizaron movimientos de archivos, eliminación de componentes obsoletos y ajustes relacionados con la captura de eventos significativos. Esta desviación respecto a una planificación puramente funcional resultó positiva, ya que permitió reducir ruido en los datos y preparar una base más coherente para las métricas posteriores.

El principal aprendizaje de este sprint fue que el CSV debía tratarse como un contrato entre los dos *add-ons*. Cualquier cambio en la estructura del logger afectaría directamente al análisis posterior, por lo que la cabecera, los campos registrados y la semántica de cada columna debían mantenerse documentados y controlados.

Sprint 3: implementación inicial del *add-on* de análisis

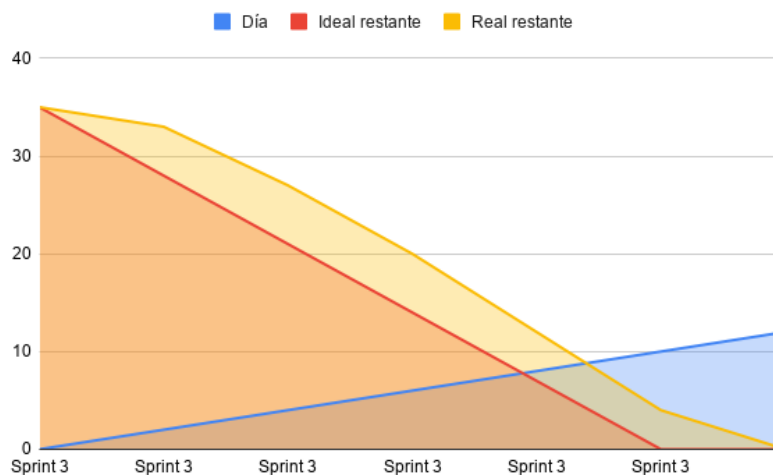


Figura A.4: Burndown del Sprint 3.

Esta fase se desarrolló durante la primera y segunda semana de marzo.

El tercer sprint tuvo como objetivo iniciar el segundo *add-on* de la *suite*. La planificación contemplaba la lectura de CSV, el cálculo de primeras métricas temporales y espaciales, y una interfaz inicial para el análisis.

Las tareas previstas fueron:

- crear la estructura base del *add-on* de análisis;

- implementar la lectura de archivos CSV;
- calcular primeras métricas temporales y espaciales;
- mostrar resultados básicos en Blender;
- preparar primeras visualizaciones.

La ejecución real incorporó varias versiones asociadas a la implementación del análisis temporal y espacial. También se subieron archivos ZIP del *add-on* de análisis, se eliminaron versiones antiguas y se reorganizaron carpetas. Esto indica que el sprint no solo permitió implementar cálculo analítico, sino también consolidar la estructura del segundo complemento.

Durante esta fase comenzaron a aparecer problemas propios del análisis de datos: valores incompletos, necesidad de interpretar columnas temporales, cálculo de velocidades, distancias y evolución de actividad. Por ello, la implementación inicial tuvo que combinar lectura de datos, depuración de CSV y primeras decisiones sobre cómo presentar los resultados.

Sprint 4: métricas, visualización inicial y evolución del logger

Esta fase se desarrolló durante la tercera y cuarta semana de marzo.

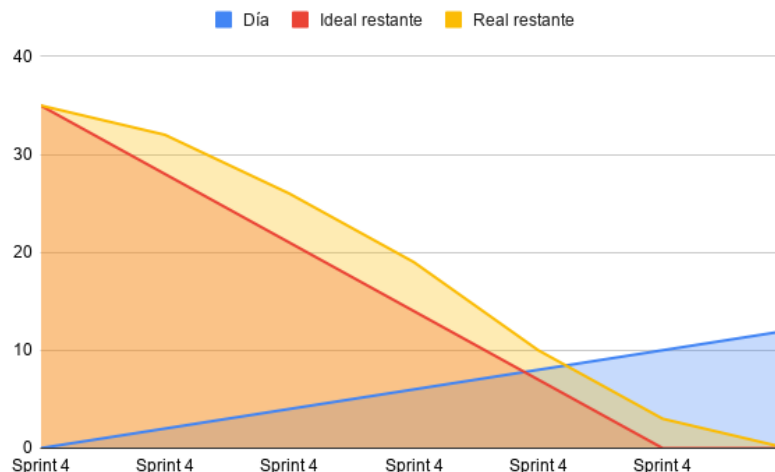


Figura A.5: Burndown del Sprint 4.

El cuarto sprint se planificó como una fase de ampliación de métricas y primeras visualizaciones. El objetivo previsto era avanzar en el cálculo de métricas temporales, espaciales y topológicas, además de desarrollar elementos gráficos iniciales.

Las tareas previstas fueron:

- ampliar métricas temporales y espaciales;
- iniciar métricas topológicas;
- mejorar gráficos y ejes 3D;
- probar el análisis con datos de prueba;
- continuar la documentación técnica.

La ejecución real combinó estas tareas con una evolución importante del logger. Se incorporaron mejoras en ejes 3D, datos de prueba reales y actualizaciones de documentación. Además, el logger avanzó hacia una versión más robusta, incorporando cambios significativos en la detección de eventos, el sistema de logging y la interfaz de control.

Este sprint muestra una primera desviación clara: parte del esfuerzo previsto para análisis y visualización tuvo que compartirse con la mejora del sistema de captura. La causa fue técnica: si el logger no registraba correctamente los eventos, las métricas calculadas posteriormente podían perder precisión o resultar inconsistentes. Por tanto, se priorizó reforzar la calidad de los datos de entrada antes de ampliar excesivamente la capa de análisis.

Sprint 5: refactorización del logger y soporte avanzado de Blender

Esta fase se desarrolló durante la primera y segunda semana de abril.

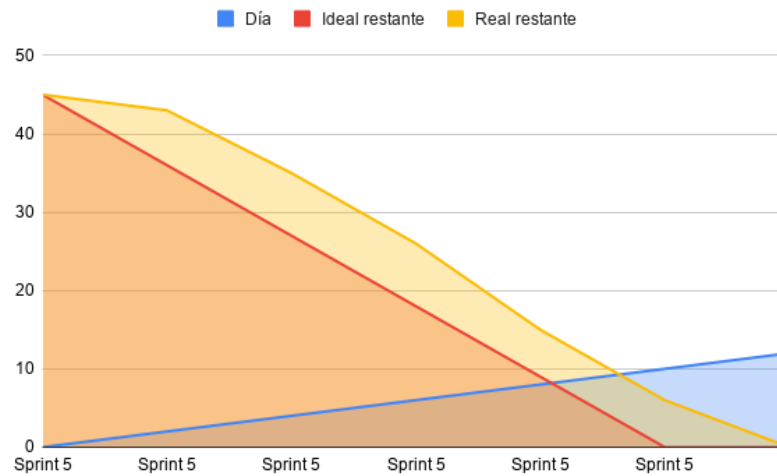


Figura A.6: Burndown del Sprint 5.

El quinto sprint se centró en la refactorización profunda del subsistema de captura. El objetivo era mejorar la robustez del logger, especialmente en relación con *Edit Mode*, operadores de Blender, deltas, UV y persistencia CSV.

Las tareas previstas fueron:

- incorporar soporte para *Edit Mode* mediante `bmesh`;
- mejorar la detección de operadores como `Ctrl+V`, `Shift+D`, `Alt+D` y `Merge`;
- detectar cambios UV;
- reducir registros redundantes;
- añadir controles de inicio y parada;
- incorporar un panel de depuración;
- corregir el registro de deltas en *Object Mode*.

La ejecución real confirma que este sprint fue uno de los más relevantes desde el punto de vista técnico. Se corrigió el registro de deltas en *Object Mode*, evitando perder cambios al borrar objetos. También se restauró el

comportamiento nativo de *Shift+D* y *Alt+D*, se mejoró el sistema de logging, se añadió detección robusta de UV y se incorporó un panel de depuración.

La revisión de este sprint muestra que el objetivo se cumplió, aunque con mayor complejidad de la prevista. La diferencia entre los modos de Blender obligó a utilizar estrategias distintas para leer la malla en modo objeto y en modo edición. Además, algunas operaciones de usuario no podían capturarse únicamente mediante cambios de contadores, por lo que fue necesario combinar detección de operadores, banderas internas, hash UV y comparación de *snapshots*.

El resultado fue una versión mucho más estable del logger, capaz de registrar únicamente cambios significativos y reducir el crecimiento innecesario del CSV.

Sprint 6: análisis avanzado, métricas y gráficos

Esta fase se desarrolló durante la tercera y cuarta semana de abril.

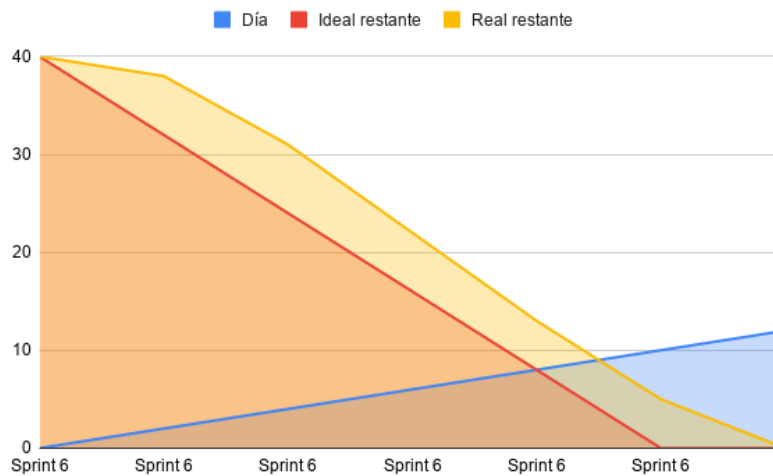


Figura A.7: Burndown del Sprint 6.

El sexto sprint se dedicó al desarrollo avanzado del *add-on* de análisis. La planificación contemplaba la ampliación del cálculo estadístico, la incorporación de métricas sobre CSV y la generación de gráficos enriquecidos.

Las tareas previstas fueron:

- implementar análisis estadístico de CSV;

- calcular media, mediana, desviación estándar, mínimos y máximos cuando procediera;
- ampliar métricas A1–A16;
- incorporar visualizaciones con Matplotlib [11];
- mejorar la presentación de resultados en la interfaz;
- actualizar la documentación técnica.

La ejecución real incluyó la incorporación de análisis estadístico de CSV, actualización de métricas y mejoras de documentación. Se añadieron cálculos descriptivos y se avanzó en la representación gráfica. No obstante, algunas funciones de cálculo de métricas CSV necesitaron correcciones posteriores, especialmente cuando se aplicaron a múltiples CSV o a datos reales.

La revisión de este sprint evidencia que el cálculo de métricas no fue una tarea cerrada en una única iteración. Fue necesario ajustar funciones, validar entradas, controlar valores no numéricos y revisar la interpretación de los resultados. Esta situación es coherente con la naturaleza incremental del proyecto: la validez de las métricas dependía tanto del formato del CSV como de la variedad de sesiones analizadas.

Sprint 7: validación con datos reales y corrección de métricas

Esta fase se desarrolló durante la primera y segunda semana de mayo.

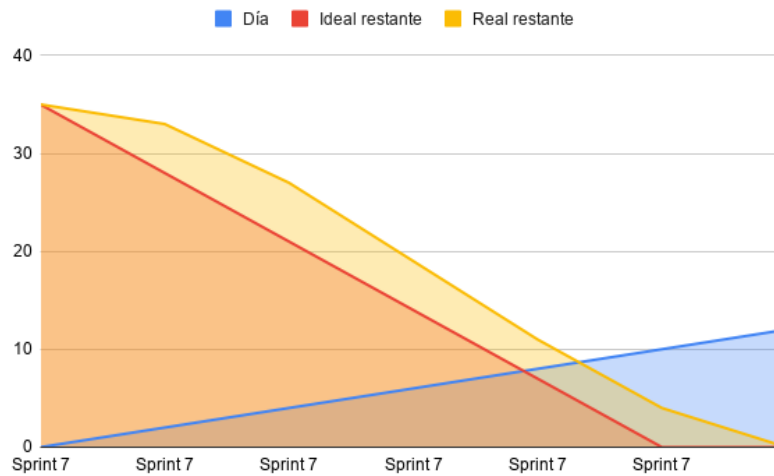


Figura A.8: Burndown del Sprint 7.

El séptimo sprint se planificó como una fase de validación, pruebas funcionales y corrección de defectos. El objetivo era comprobar el comportamiento del sistema con datos reales y mejorar la estabilidad de las métricas.

Las tareas previstas fueron:

- validar el sistema con CSV reales;
- comprobar el cálculo de métricas A1–A16;
- revisar el análisis de múltiples CSV;
- corregir errores de cálculo;
- integrar el segundo *add-on* en la documentación;
- preparar el sistema para su cierre.

La ejecución real muestra la incorporación de un CSV real procedente de la actividad realizada en la EASD, Escuela de Arte y Superior de Diseño de Burgos, así como varias correcciones sobre **Calculate CSV Metric Stats** y métricas CSV múltiples. También se llevó a cabo la reestructuración de anexos e integración completa del segundo *add-on* en la documentación del TFG.

Este sprint cumplió su función de validación, pero también reveló errores que no habían aparecido en pruebas más controladas. La lectura de CSV múltiples y el cálculo estadístico sobre datos reales exigieron revisar funciones, controlar casos límite y reforzar la validación. Por tanto, la ejecución real fue más correctiva de lo previsto, pero permitió aumentar la fiabilidad del sistema antes de la fase final.

Sprint 8: cierre, refactorización final e interfaz por pestañas

Esta fase se desarrolló durante la tercera y cuarta semana de mayo.

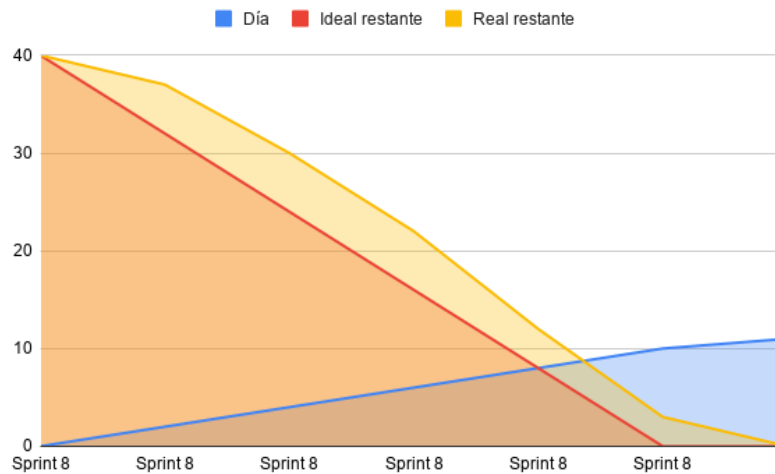


Figura A.9: Burndown del Sprint 8.

El último sprint se dedicó a la consolidación de la *suite* completa. La planificación contemplaba el refinamiento de la interfaz, la revisión de documentación, la corrección de errores pendientes y la preparación de la versión final.

Las tareas previstas fueron:

- revisar la documentación final;
- mejorar la interfaz del *add-on* de análisis;
- eliminar redundancias visuales;

- incorporar textos breves y tooltips;
- organizar resultados de forma más clara;
- cerrar una versión estable de la *suite*.

La ejecución real incluyó una refactorización final importante. Se reorganizó la estructura interna del código, separando la lógica de cálculo, la interfaz y la generación de visualizaciones. También se incorporaron optimizaciones mediante estructuras tipo KDTree para mejorar cálculos geométricos relacionados con búsquedas de proximidad entre mallas. Además, se revisaron funciones de métricas procedentes de CSV y métricas de malla, incluyendo similitud, distancia de Hausdorff, normales, ángulos, duplicados de vértices y métricas UV.

En la interfaz se incorporó una organización por pestañas, separando la navegación entre métricas estadísticas, métricas geométricas, métricas UV y visualizaciones. Esta decisión mejoró la claridad del *add-on* y redujo la sobrecarga visual. El sprint cumplió su objetivo de cierre, aunque también asumió tareas de optimización técnica que inicialmente podrían haberse previsto en fases anteriores.

El resultado final fue una *suite* más organizada, mantenible y coherente, con una separación más clara entre captura, análisis, cálculo geométrico e interfaz.

Resumen de sprints

La Tabla [A.1](#) sintetiza de forma cronológica las fases fundamentales que han articulado el ciclo de vida del desarrollo.

Sprint	Periodo	Funcionalidades principales	Resultado
Fase previa	Noviembre	Estudio de Blender, API <code>bpy</code> , eventos y viabilidad técnica.	Confirmación de viabilidad y detección de limitaciones iniciales.
Sprint 0	Enero	Requisitos, alcance y arquitectura basada en dos <i>add-ons</i> .	Definición del enfoque modular.
Sprint 1	Febrero semanas 1–2	Logger inicial, interfaz básica y CSV.	Primera versión funcional del sistema de captura.
Sprint 2	Febrero semanas 3–4	Refinamiento de captura, reorganización y normalización CSV.	Base más estable para el intercambio de datos.
Sprint 3	Marzo semanas 1–2	<i>Add-on</i> de análisis, lectura CSV y primeras métricas.	Integración inicial del análisis temporal y espacial.
Sprint 4	Marzo semanas 3–4	Métricas, ejes 3D, pruebas y evolución del logger.	Mejora simultánea de análisis y captura.
Sprint 5	Abril semanas 1–2	Refactorización del logger, <code>bmesh</code> , operadores, UV y deltas.	Versión avanzada y robusta del Data Logger 3D.
Sprint 6	Abril semanas 3–4	Análisis estadístico, métricas CSV y gráficos.	<i>Add-on</i> de análisis más completo.
Sprint 7	Mayo semanas 1–2	Validación funcional con datos reales procedentes de la actividad realizada en la EASD, Escuela de Arte y Superior de Diseño de Burgos, y corrección de métricas.	Sistema comprobado con CSV real y funciones corregidas.
Sprint 8	Mayo semanas 3–4	Refactorización final, KD-Tree, pestañas, documentación e interfaz.	Versión final de la <i>suite</i> completa.

Tabla A.1: Resumen cronológico de los sprints del proyecto.

Revisión de ejecución por sprint

La revisión de cada sprint se ha realizado contrastando la planificación prevista con la evolución real reflejada en el historial de commits. Este análisis permite observar que el desarrollo mantuvo una orientación incremental, aunque algunas tareas se desplazaron entre iteraciones debido a la complejidad técnica de Blender, la depuración de métricas y la necesidad de reorganizar progresivamente la arquitectura del sistema.

Sprint	Plan previsto	Ejecución real
Fase previa	Estudiar la viabilidad técnica y la API de Blender.	Se confirmó la viabilidad y se detectó la necesidad de usar temporizadores, snapshots y bmesh .
Sprint 0	Definir requisitos, alcance y arquitectura de dos <i>add-ons</i> .	Se mantuvo la arquitectura modular, aunque el <i>add-on</i> de análisis adquirió mayor alcance.
Sprint 1	Crear un logger inicial con interfaz básica y salida CSV.	Se obtuvo una primera versión funcional de <i>Data Logger 3D</i> .
Sprint 2	Mejorar captura, organización del proyecto y CSV.	Se corrigieron versiones iniciales y se estabilizó el registro de eventos significativos.
Sprint 3	Implementar lectura CSV y primeras métricas de análisis.	Se incorporó análisis temporal y espacial, junto con mejoras de interfaz y reorganización.
Sprint 4	Ampliar métricas y primeras visualizaciones.	Se avanzó en gráficos y pruebas, pero también se reforzó el logger para mejorar la calidad del CSV.
Sprint 5	Refactorizar logger, operadores, UV, deltas y <i>Edit Mode</i> .	Se corrigieron deltas, duplicaciones, detección de operadores, UV y panel de depuración.
Sprint 6	Desarrollar métricas CSV, análisis estadístico y visualizaciones.	Se añadieron estadísticas descriptivas y métricas, aunque algunas funciones requirieron ajustes posteriores.
Sprint 7	Validar con datos reales y corregir errores.	Se incorporó un CSV real procedente de la actividad realizada en la EASD, Escuela de Arte y Superior de Diseño de Burgos, y se corrigieron métricas CSV múltiples.
Sprint 8	Cerrar documentación, interfaz y versión final.	Se refactorizó el análisis, se añadió KD-Tree, se corrigieron métricas de malla y se organizó la interfaz por pestañas.

Tabla A.2: Revisión resumida de ejecución por sprint a partir de la planificación.

En conjunto, la revisión muestra que la planificación inicial se mantuvo como guía general, pero el desarrollo real evolucionó de forma más iterativa de lo previsto. Las principales desviaciones no se debieron a cambios arbitrarios de alcance, sino a problemas técnicos detectados durante la integración con Blender, especialmente en la captura de eventos, la gestión de modos de edición, la validación de CSV y el cálculo de métricas geométricas. Esta evolución permitió mejorar la robustez del sistema y llegar a una versión final más completa que la inicialmente planteada.

Tabla de desviaciones: planificación frente a ejecución

La Tabla A.3 expone las desviaciones temporales más relevantes detectadas durante el ciclo de vida del software, contrastando hitos planificados con resultados reales.

Hito técnico	Plan.	Real	Desv.	Justificación técnica
Prototipo inicial del logger	16 Feb	16 Feb	0 d	Se alcanzó una primera versión funcional con interfaz básica y registro inicial.
Implementación v0.3	05 Mar	11 Mar	+6 d	La lectura de CSV, el análisis temporal y espacial y la reorganización del <i>addon</i> de análisis requirieron más iteraciones de las previstas.
Soporte avanzado de <i>Edit Mode</i>	10 Mar	20 Mar	+10 d	La lectura de mallas editables mediante bmesh y la detección robusta de eventos resultaron más complejas de lo esperado.
Análisis estadístico CSV	20 Abr	27 Abr	+7 d	La integración de estadísticas descriptivas y validación de datos exigió revisar funciones de cálculo.
Validación con datos reales	10 May	15 May	+5 d	El CSV real procedente de la actividad realizada en la EASD, Escuela de Arte y Superior de Diseño de Burgos, permitió detectar errores en métricas múltiples y obligó a corregir funciones de cálculo.
Documentación e interfaz final	15 May	18 May	+3 d	El refinamiento de la interfaz, la mejora de gráficos comparativos y la actualización documental se extendieron hasta la fase final.
Refactorización final y KDTTree	20 May	26 May	+6 d	La reorganización entre cálculo, interfaz y visualización, junto con optimizaciones geométricas, se incorporó como mejora final de estabilidad y mantenibilidad.

Tabla A.3: Tabla comparativa de hitos temporales y desviaciones.

Análisis de gestión de riesgos en la planificación

Los retrasos identificados se fundamentan principalmente en decisiones técnicas orientadas a mejorar la estabilidad del sistema. Durante el desarrollo se detectó que una captura demasiado frecuente o poco filtrada podía generar CSV excesivamente grandes y afectar al rendimiento de Blender. Por ello, se optó por un enfoque basado en cambios significativos, comparación de estados y temporizadores.

Uno de los principales riesgos fue la degradación del rendimiento en el hilo principal de Blender durante el modelado. Para mitigarlo, se evitó realizar análisis pesados durante la captura. El logger se limitó a registrar datos técnicos y el cálculo de métricas quedó desplazado al segundo *add-on*. Esta separación permitió que la sesión de modelado no se viera interrumpida por operaciones estadísticas o geométricas complejas.

También se identificó un riesgo relacionado con la detección incompleta de eventos. Algunas operaciones de Blender no producen cambios evidentes en los contadores topológicos, pero sí son relevantes para el análisis del proceso. Este problema se mitigó mediante la combinación de detección de operadores, banderas internas, hash UV y comparación de snapshots.

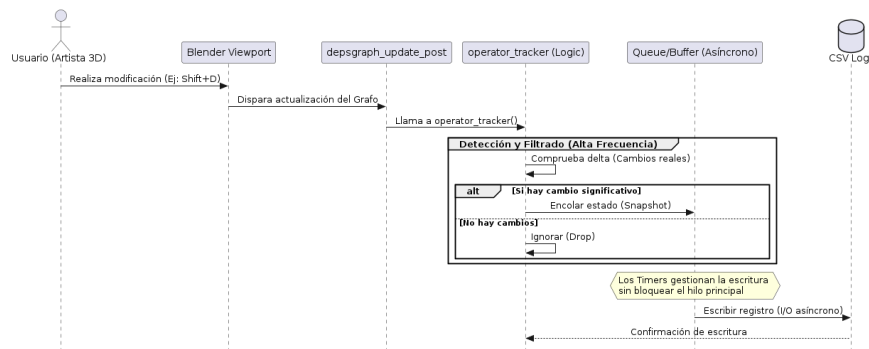


Figura A.10: Diagrama de flujo del sistema de captura y análisis planificado.

Lecciones aprendidas

El proceso de desarrollo ha permitido extraer varias conclusiones técnicas y metodológicas. En primer lugar, la integración con Blender requiere una comprensión detallada del contexto activo, ya que el acceso a la malla, a los operadores y a los datos UV varía según el modo de trabajo. Por este motivo, una parte significativa del esfuerzo se destinó a robustecer la captura antes de ampliar el análisis.

En segundo lugar, el CSV funcionó como un elemento central de la arquitectura. La calidad de las métricas dependía directamente de la estabilidad del formato de datos, por lo que fue necesario normalizar cabeceras, documentar campos y validar valores antes del cálculo.

En tercer lugar, la interfaz necesitó evolucionar junto al sistema. A medida que aumentaba el número de métricas, la presentación inicial resultaba insuficiente. La organización final por pestañas, bloques compactos y tooltips permitió reducir la carga cognitiva del usuario y mejorar la comprensión de los resultados.

Finalmente, la incorporación de optimizaciones como KDTree en métricas geométricas evidenció la importancia de considerar el rendimiento desde fases tempranas. Las comparaciones entre mallas pueden crecer rápidamente en coste computacional, por lo que el uso de estructuras de búsqueda espacial resultó adecuado para mejorar la eficiencia.

A.3. Viabilidad del proyecto

Viabilidad técnica

La viabilidad técnica de la solución queda respaldada por la flexibilidad de Blender como plataforma extensible mediante Python [18]. El proyecto se apoya en la API `bpy`, en operadores personalizados, en manejadores persistentes, en temporizadores y en acceso directo a estructuras de datos de escena.

El primer *add-on* utiliza módulos estándar de Python y componentes propios de Blender para capturar datos durante la sesión. El uso de `bmesh` permite acceder a la topología en *Edit Mode*, mientras que `mathutils` facilita operaciones espaciales sobre matrices, vectores y cajas envolventes.

El segundo *add-on* resulta técnicamente viable porque el análisis se realiza de forma diferida. Esta decisión evita sobrecargar el proceso de modelado y permite incorporar cálculos más complejos, como:

- cálculo de métricas A1–A16 a partir de registros CSV;
- validación de datos y control de valores no numéricos;
- cálculo de métricas geométricas de malla;
- cálculo de métricas UV;

- generación de gráficos mediante Matplotlib [11];
- procesamiento numérico mediante NumPy [15];
- optimización de búsquedas espaciales mediante estructuras tipo KD-Tree cuando procede;
- visualización de resultados dentro del entorno de Blender.

La principal dificultad técnica procede de la complejidad de la API de Blender y de la necesidad de trabajar con diferentes estados de escena. No obstante, las pruebas iterativas y la refactorización progresiva permitieron alcanzar una solución funcional y extensible.

Viabilidad económica

Desde el punto de vista económico, el proyecto resulta viable porque se ha desarrollado principalmente con tecnologías de código abierto o sin coste directo de licencia en el contexto académico. Blender, Python, NumPy, Matplotlib, GitHub en su modalidad básica y L^AT_EX pueden utilizarse sin costes de adquisición para este proyecto.

No obstante, para estimar el coste económico de forma más realista no debe imputarse al proyecto el coste total de los recursos hardware y software utilizados, ya que estos bienes siguen teniendo valor una vez finalizado el desarrollo. El equipo informático puede seguir utilizándose en otros proyectos o venderse en el mercado de segunda mano, recuperando parte de su valor. Por ello, una forma sencilla y coherente de imputación consiste en considerar únicamente la pérdida de valor correspondiente al periodo de uso durante el proyecto. En términos contables, esta imputación puede aproximarse mediante la amortización del bien durante el tiempo en que ha sido utilizado.

Costes de software

El software utilizado en el proyecto no ha supuesto un coste directo de licencia:

- Blender: 0 €.
- Python: 0 €.
- GitHub: 0 € en el uso académico básico.

- NumPy: 0 €.
- Matplotlib: 0 €.
- L^AT_EX: 0 €.

Por tanto, no se imputa un coste de adquisición de software al proyecto. En caso de utilizar licencias comerciales en un entorno profesional, debería aplicarse un criterio equivalente al del hardware: imputar únicamente la parte proporcional correspondiente al tiempo de uso o a la amortización de dichas licencias durante el proyecto.

Costes de hardware

Para la ejecución del proyecto se ha utilizado una estación de trabajo capaz de ejecutar Blender de forma fluida. Se toma como referencia un equipo de gama media con un valor estimado de 1200 €. Dado que este equipo no se consume completamente durante el proyecto y conserva valor al finalizar, no sería correcto imputar su coste total.

Aplicando una amortización lineal de 48 meses y considerando una dedicación aproximada de cuatro meses al proyecto, el coste imputable al proyecto se calcula como:

$$\text{Coste hardware} = \frac{1200 \text{ euros}}{48 \text{ meses}} \times 4 \text{ meses} = 100 \text{ euros}$$

Por tanto, el coste imputable de hardware asciende a 100 €, correspondiente a la pérdida de valor estimada durante el periodo de uso del proyecto.

Coste estimado de desarrollo

Aunque el proyecto se enmarca en un contexto académico, puede estimarse un coste equivalente de desarrollo tomando como referencia una dedicación de cuatro meses y una remuneración mensual aproximada de 1200 € para un perfil junior.

El coste salarial bruto sería:

$$1200 \text{ euros/mes} \times 4 \text{ meses} = 4800 \text{ euros}$$

Sin embargo, desde el punto de vista empresarial, el coste de personal no se limita al salario bruto. También debe incluirse la Seguridad Social a cargo de la empresa. Para el año 2026, en el Régimen General, se considerarán los siguientes tipos empresariales: contingencias comunes, desempleo, Fondo de Garantía Salarial, formación profesional y Mecanismo de Equidad Intergeneracional.

En el caso de una contratación general o indefinida, los porcentajes empresariales aplicables son:

- Contingencias comunes: 23,60
- Desempleo: 5,50
- Fondo de Garantía Salarial: 0,20
- Formación Profesional: 0,60
- Mecanismo de Equidad Intergeneracional: 0,75

Por tanto, el porcentaje total de Seguridad Social a cargo de la empresa considerado en esta estimación es:

$$23,60 + 5,50 + 0,20 + 0,60 + 0,75 = 30,65$$

La cuota empresarial estimada sería:

$$4800 \text{ euros} \times 0,3065 = 1471,20 \text{ euros}$$

Así, el coste total de desarrollo, incluyendo salario bruto y Seguridad Social a cargo de la empresa, asciende a:

$$4800 \text{ euros} + 1471,20 \text{ euros} = 6271,20 \text{ euros}$$

Concepto	Coste imputado
Software utilizado	0 €
Hardware amortizado durante cuatro meses	100 €
Salario bruto estimado	4800 €
Seguridad Social a cargo de la empresa, 30,65 %	1471,20 €
Coste total estimado	6371,20 €

Tabla A.4: Estimación de costes del proyecto.

Viabilidad legal

El proyecto se apoya en herramientas y bibliotecas de uso permitido en contextos académicos. Blender se distribuye bajo licencia GNU GPL, Python bajo licencia propia de la Python Software Foundation y las bibliotecas científicas utilizadas cuentan con licencias compatibles con su uso en investigación y desarrollo.

La solución propuesta no modifica el núcleo de Blender, sino que se integra mediante su sistema de *add-ons*. Desde el punto de vista documental, las referencias externas se incorporan mediante la bibliografía correspondiente y no se incluyen recursos propietarios sin autorización.

Licencia del software desarrollado

Los *add-ons* desarrollados para este proyecto se distribuyen bajo la licencia **GNU General Public License v3.0 o posterior (GPL-3.0-or-later)**. Esta elección resulta coherente con el ecosistema de Blender, ya que los complementos hacen uso directo de su API de Python mediante módulos como `bpy`, `bmesh` y `mathutils`. Al integrarse como extensiones ejecutadas dentro de Blender, la distribución del código debe mantenerse compatible con la licencia GPL del propio entorno.

La licencia GPL permite usar, estudiar, modificar y redistribuir el software, siempre que las versiones derivadas mantengan la misma licencia y se conserve el acceso al código fuente. En este proyecto, dicha condición se satisface al publicar los archivos fuente de los *add-ons* junto con el repositorio y al incluir un archivo `LICENSE` con el texto completo de la licencia.

Por tanto, el producto software resultante se libera como software libre bajo GPL-3.0-or-later, incluyendo tanto el *add-on Data Logger 3D* como el *add-on* de análisis y visualización.

Viabilidad ética

La viabilidad ética del proyecto se fundamenta en la transparencia de la captura y en la minimización de datos personales. El logger registra datos técnicos asociados al proceso de modelado, como transformaciones, cambios topológicos, modos, UV, operadores relevantes y estado de la escena. No registra contenido textual externo, conversaciones, credenciales ni datos personales directos.

Además, el sistema incorpora un mecanismo de consentimiento explícito antes de iniciar la captura. El identificador de usuario se genera de forma pseudonimizada, evitando almacenar nombres reales. Aun así, los CSV generados deben tratarse como datos de investigación, ya que pueden reflejar patrones de comportamiento técnico de una persona durante el modelado. En particular, el CSV incluido como muestra en el repositorio procede de la actividad realizada en la EASD, Escuela de Arte y Superior de Diseño de Burgos, y se incorpora únicamente como evidencia de validación funcional del flujo de captura y análisis, no como resultado de un estudio experimental completo con usuarios.

A.4. Análisis de riesgos

El desarrollo del proyecto ha estado condicionado por varios riesgos técnicos, funcionales y éticos. La Tabla [A.5](#) recoge los principales riesgos identificados, su impacto y las medidas aplicadas para reducirlos.

Riesgo	Impacto	Medida de mitigación
Degradación de rendimiento durante el modelado	Alto	Separación entre captura y análisis, temporizador de muestreo, registro basado en cambios significativos y reducción de operaciones pesadas durante la sesión.
Disparidad entre <i>Object Mode</i> y <i>Edit Mode</i>	Alto	Uso diferenciado de acceso a datos mediante <code>obj.data</code> y <code>bmesh.from_edit_mesh()</code> según el modo activo.
Crecimiento excesivo del CSV	Medio	Registro diferencial, eliminación de filas duplicadas y escritura solo cuando se detectan cambios relevantes o eventos forzados.
Pérdida accidental de datos	Medio	Incrustación del CSV en el archivo <code>.blend</code> y exportación manual a archivo externo.
Detección incompleta de operadores	Medio	Normalización de identificadores de operadores, lectura de operadores recientes y uso de banderas específicas para eventos relevantes.
Cambios UV no detectados por topología	Medio	Uso de operadores UV conocidos y cálculo de hash global de coordenadas UV.
Errores en métricas CSV	Medio	Validación de valores, revisión de funciones de cálculo y pruebas con múltiples CSV y datos reales.
Coste elevado en métricas geométricas	Medio	Uso de estructuras de búsqueda espacial como KD-Tree en cálculos de proximidad y comparación entre mallas.
Sobrecarga visual de la interfaz	Bajo	Organización por pestañas, bloques compactos, reducción de textos visibles y uso de tooltips.
Riesgos de privacidad	Medio	Consentimiento explícito, seudonimización del identificador y limitación de la captura a datos técnicos.
Dependencia de versiones de Blender	Medio	Desarrollo sobre versiones recientes, documentación de dependencias y separación modular para facilitar futuras adaptaciones.

Tabla A.5: Matriz de riesgos técnicos, funcionales y éticos del proyecto.

La aplicación de estas medidas permitió reducir la deuda técnica, mejorar la estabilidad de los *add-ons* y asegurar una evolución controlada del sistema. En particular, la separación entre logger y análisis fue una decisión clave para mitigar riesgos de rendimiento y facilitar la ampliación posterior de métricas y visualizaciones.

A.5. Conclusión del plan de proyecto

El proyecto siguió una planificación iterativa e incremental, aunque la ejecución real presentó ajustes derivados de la complejidad técnica del entorno Blender. La evolución del sistema no fue lineal: algunas tareas de análisis obligaron a revisar la captura, y algunos problemas detectados con datos reales exigieron volver sobre funciones de métricas ya implementadas.

A pesar de estas desviaciones, la planificación permitió mantener una dirección clara. La *suite* evolucionó desde un logger inicial hasta un sistema formado por dos *add-ons* complementarios, con captura automática, exportación CSV, análisis estadístico, métricas geométricas, métricas UV, visualizaciones e interfaz organizada por pestañas.

La revisión del historial de commits muestra una progresión coherente: primero se construyó la base del logger, después se normalizó el intercambio de datos, posteriormente se incorporó el análisis, se validaron métricas con datos reales y finalmente se refactorizó el sistema para mejorar rendimiento, mantenibilidad e interfaz. Por tanto, el proyecto puede considerarse viable, trazable y técnicamente coherente con los objetivos planteados.

Apéndice *B*

Especificación de Requisitos

B.1. Introducción

Este apéndice recoge la especificación de requisitos del sistema desarrollado. La solución se organiza como una *suite* de dos add-ons para Blender: *Data Logger 3D*, orientado a la monitorización y captura de datos durante sesiones de modelado 3D, y *Analysis 3D* de análisis y visualización, encargado de procesar los ficheros CSV generados, calcular métricas estadísticas, geométricas y UV, y presentar los resultados dentro del propio entorno de Blender.

Los requisitos se agrupan en requisitos funcionales, no funcionales, de interfaz y ético-legales. Además, se incorporan tablas de trazabilidad y una relación resumida de casos de uso, con el fin de conectar las necesidades del proyecto con los componentes principales de la *suite*.

B.2. Objetivos generales

La especificación de requisitos responde a los siguientes objetivos generales del sistema:

- Capturar de forma no intrusiva la actividad del usuario durante procesos de modelado 3D en Blender.
- Registrar datos temporales, espaciales, topológicos, operacionales y UV en ficheros CSV estructurados.

- Evitar la generación de registros redundantes mediante un sistema de comparación de estados, snapshots y deltas.
- Analizar los CSV generados para obtener métricas estadísticas A1–A16 sobre la sesión de modelado.
- Calcular métricas geométricas y UV sobre objetos de Blender, incluyendo área UV, islas UV, *stretch*, densidad de texeles, normales, ángulos, duplicados, no cuadrángulos y similitud geométrica.
- Presentar los resultados mediante paneles integrados en Blender, bloques compactos, textos breves, tooltips y visualizaciones 2D o 3D.
- Garantizar consentimiento explícito, transparencia, minimización de datos y seudonimización del identificador de usuario.
- Mantener una arquitectura modular que separe captura, persistencia, cálculo de métricas, visualización e interfaz.

B.3. Catálogo de requisitos

Requisitos funcionales del Data Logger 3D

El primer add-on debe encargarse de registrar la actividad del usuario durante la sesión de modelado sin alterar el flujo normal de trabajo. La captura se basa en un temporizador no intrusivo, en la comparación de estados sucesivos de la escena y en la detección de determinadas operaciones relevantes.

RF-01 Captura temporal. El sistema debe registrar marcas temporales absolutas y relativas mediante campos como `TimeStamp`, `Minute` y `Second`, de forma que pueda reconstruirse la evolución de la sesión.

RF-02 Captura espacial de la vista. El sistema debe registrar la posición del punto de vista del usuario en la escena mediante coordenadas como `UserX`, `UserY` y `UserZ`.

RF-03 Captura del objeto activo. El sistema debe registrar información básica del objeto activo, incluyendo posición, centro, radio aproximado y variaciones espaciales mediante campos como `ObjX`, `ObjY`, `ObjZ`, `ObjRadius` y sus deltas asociados.

- RF-04 Captura del radio de escena.** El sistema debe estimar el radio global de la escena a partir de los objetos de tipo malla presentes en el entorno.
- RF-05 Captura topológica diferencial.** El sistema debe registrar cambios en la estructura de la malla, especialmente variaciones en el número de vértices, triángulos, polígonos no cuadrangulares y normales potencialmente invertidas.
- RF-06 Captura en Object Mode y Edit Mode.** El sistema debe obtener información de la malla tanto en *Object Mode* como en *Edit Mode*, utilizando estructuras adecuadas para cada contexto.
- RF-07 Uso de BMesh.** El sistema debe emplear `bmesh` cuando sea necesario consultar o evaluar geometría editable sin depender únicamente de la malla evaluada en modo objeto.
- RF-08 Captura de cambios de modo.** El sistema debe detectar transiciones entre modos de trabajo, especialmente entre *Object Mode* y *Edit Mode*.
- RF-09 Captura de cambios de objeto.** El sistema debe registrar inserciones, eliminaciones o cambios cuantitativos en el número de objetos de la escena.
- RF-10 Captura de modificadores.** El sistema debe registrar variaciones en el número total de modificadores asociados a los objetos monitorizados.
- RF-11 Detección de Ctrl+V.** El sistema debe detectar operaciones de pegado que puedan introducir geometría, objetos o contenido en la escena.
- RF-12 Detección de Shift+D.** El sistema debe identificar operaciones de duplicación no enlazada.
- RF-13 Detección de Alt+D.** El sistema debe identificar operaciones de duplicación enlazada.
- RF-14 Detección de operaciones Merge.** El sistema debe detectar operaciones de fusión de vértices, aristas o caras.
- RF-15 Detección de operaciones UV.** El sistema debe identificar operaciones UV relevantes, como *unwrap*, *smart project*, *pack islands*, *minimize stretch* u otras transformaciones del mapa UV.

- RF-16 Hash UV.** El sistema debe generar una firma o hash de las coordenadas UV para detectar modificaciones que no siempre se reflejan en contadores geométricos tradicionales.
- RF-17 Sistema de snapshots.** El sistema debe construir representaciones compactas del estado de la escena y compararlas con estados anteriores para registrar únicamente cambios relevantes.
- RF-18 Escritura incremental en CSV.** El sistema debe escribir una nueva fila únicamente cuando se detecte una modificación significativa o una operación pendiente de volcado.
- RF-19 Persistencia interna en el archivo .blend.** El sistema debe poder almacenar una copia del CSV como bloque de texto interno del archivo .blend, reduciendo el riesgo de pérdida de datos durante la sesión.
- RF-20 Restauración desde el archivo .blend.** El sistema debe poder restaurar el CSV interno al reiniciar el logger o reabrir el proyecto, siempre que exista información previa disponible.
- RF-21 Exportación CSV.** El sistema debe permitir exportar el registro a un fichero CSV externo asociado al proyecto.
- RF-22 Consentimiento.** El sistema debe solicitar aceptación explícita antes de iniciar la captura de datos.
- RF-23 Advertencia de Auto Run.** El sistema debe advertir al usuario si Blender no permite la ejecución automática de scripts y esta limitación afecta a la monitorización.
- RF-24 Panel de control.** El sistema debe ofrecer una interfaz básica para iniciar, detener y exportar la captura.
- RF-25 Panel de depuración.** El sistema debe incluir, al menos durante desarrollo y validación, un panel de depuración con variables internas útiles para comprobar el funcionamiento.

Requisitos funcionales del add-on de análisis y visualización

El segundo add-on debe procesar los registros generados por *Data Logger 3D*, validar su contenido, calcular métricas estadísticas y geométricas, y mostrar los resultados mediante una interfaz clara dentro de Blender.

- RF-26 Detección de CSV.** El sistema debe permitir seleccionar archivos CSV individuales o carpetas que contengan varios registros de sesión.
- RF-27 Lectura y validación.** El sistema debe leer los CSV con codificación compatible y validar la presencia de los campos necesarios para el cálculo de métricas.
- RF-28 Rechazo de valores no finitos.** El sistema debe impedir el procesamiento de valores NaN, infinitos o datos corruptos cuando comprometan la validez de los cálculos.
- RF-29 Métricas A1–A16.** El sistema debe calcular el conjunto de métricas estadísticas A1–A16 definidas para caracterizar la sesión de modelado.
- RF-30 Duración en horas.** La métrica A1 debe expresarse en horas para facilitar la interpretación de sesiones de distinta duración.
- RF-31 Agrupación A2–A3.** Las métricas A2 y A3 deben presentarse de forma conjunta cuando su interpretación sea complementaria.
- RF-32 Agrupación A6–A9.** Las métricas A6, A7, A8 y A9 deben poder agruparse en un bloque común cuando compartan valores o dependan del mismo cálculo base.
- RF-33 Cuartiles selectivos.** El sistema debe mostrar cuartiles únicamente en aquellas métricas donde aporten información útil, evitando repeticiones innecesarias.
- RF-34 Separación A12, A13 y A16.** Las métricas A12, A13 y A16 deben mantenerse separadas cuando sus cuartiles o valores describan aspectos independientes de la sesión.
- RF-35 Métricas geométricas.** El sistema debe calcular propiedades geométricas de la malla seleccionada, incluyendo medidas de estructura, calidad y comparación con referencia cuando proceda.
- RF-36 Área UV.** El sistema debe calcular el área ocupada por las coordenadas UV, preferiblemente a partir de una triangulación de las caras en el espacio UV.
- RF-37 Islas UV.** El sistema debe contar las islas UV existentes en la capa activa del objeto.

- RF-38 UV stretch.** El sistema debe estimar la deformación entre longitudes o áreas 3D y sus equivalentes en el espacio UV.
- RF-39 Texel density.** El sistema debe calcular una medida de densidad UV relacionada con el área geométrica y el área del mapa UV.
- RF-40 Normal percentage.** El sistema debe calcular el porcentaje de normales con orientación anómala o potencialmente invertida.
- RF-41 Angle.** El sistema debe calcular ángulos relevantes entre caras adyacentes o componentes geométricos de la malla.
- RF-42 Non-quads.** El sistema debe cuantificar polígonos no cuadrangulares como indicador de estructura topológica.
- RF-43 Vertex duplicate.** El sistema debe detectar o estimar la presencia de vértices duplicados cuando esta información sea necesaria para evaluar la calidad de la malla.
- RF-44 Similitud geométrica.** El sistema debe comparar un objeto con una referencia a partir de coordenadas de vértices, siempre que exista una referencia válida.
- RF-45 Similitud topológica.** El sistema debe comparar elementos como vértices, aristas y caras entre objeto y referencia cuando la comparación sea aplicable.
- RF-46 Distancia de Hausdorff.** El sistema debe poder calcular una distancia de Hausdorff o aproximación equivalente cuando proceda en el análisis de similitud geométrica.
- RF-47 Gráficos 2D.** El sistema debe generar gráficos estadísticos a partir de los CSV procesados, por ejemplo mediante Matplotlib.
- RF-48 Visualizaciones 3D.** El sistema debe generar representaciones procedurales dentro de Blender cuando la métrica o el resultado lo requieran.
- RF-49 Paneles de resultados.** El sistema debe mostrar los resultados en paneles ordenados, legibles y consistentes con el flujo de trabajo de Blender.
- RF-50 Separación entre cálculo e interfaz.** El sistema debe mantener separada la lógica de cálculo de la lógica de presentación para facilitar mantenimiento y pruebas.

Requisitos no funcionales

RNF-01 Rendimiento. La captura no debe degradar de forma perceptible la interacción del usuario con Blender.

RNF-02 No intrusión. El usuario debe poder modelar sin modificar sustancialmente su flujo habitual de trabajo.

RNF-03 Reducción de redundancia. El sistema debe evitar filas duplicadas o estados no modificados en el CSV.

RNF-04 Robustez. Las excepciones derivadas de la API de Blender, de cambios de modo o de ausencia de objetos no deben provocar la interrupción completa del sistema.

RNF-05 Validación de datos. El análisis debe rechazar entradas corruptas, incompletas o no numéricas cuando afecten a los cálculos.

RNF-06 Gestión de memoria. Las estructuras temporales, especialmente las generadas con `bmesh`, deben liberarse tras su uso.

RNF-07 Modularidad. La captura, la persistencia, el análisis estadístico, las métricas geométricas y la interfaz deben organizarse en módulos diferenciados.

RNF-08 Mantenibilidad. El código debe estar organizado por responsabilidades y usar nombres comprensibles para facilitar futuras ampliaciones.

RNF-09 Portabilidad. Los datos deben poder intercambiarse mediante CSV, evitando formatos cerrados o dependientes exclusivamente de una instalación concreta.

RNF-10 Legibilidad de interfaz. La interfaz debe evitar líneas excesivamente largas, repeticiones y descripciones extensas visibles de forma permanente.

RNF-11 Escalabilidad. El sistema debe poder procesar varias sesiones CSV sin requerir cambios estructurales en el add-on.

RNF-12 Tolerancia a fallos. La ausencia de UV, objeto activo, malla válida o referencia geométrica debe gestionarse de forma controlada mediante mensajes o resultados no aplicables.

RNF-13 Pruebas parciales fuera de Blender. Siempre que sea posible, las funciones de cálculo independientes de la API de Blender deben poder probarse fuera del entorno gráfico.

Requisitos de interfaz

RIU-01 Panel Data Logger. Debe existir una pestaña lateral para controlar la captura del Data Logger.

RIU-02 Botón Start/Stop. La interfaz debe permitir iniciar y detener el logger de forma explícita.

RIU-03 Botón Export CSV. La interfaz debe permitir exportar el CSV generado.

RIU-04 Panel Debug. Debe existir un panel plegable o sección secundaria con información técnica de depuración.

RIU-05 Panel de análisis. El segundo add-on debe mostrar resultados estadísticos, geométricos y UV en paneles integrados en Blender.

RIU-06 Botón de cálculo de métricas CSV. Debe existir un operador de interfaz para calcular las métricas estadísticas de los registros CSV.

RIU-07 Bloques compactos. Los resultados deben organizarse en bloques visuales compactos para reducir la carga de lectura.

RIU-08 Tooltips. Las descripciones largas deben trasladarse a ayudas contextuales o tooltips.

RIU-09 Encabezados destacados. Los títulos de bloque deben diferenciarse visualmente para facilitar la navegación por los resultados.

RIU-10 Textos breves. Las etiquetas visibles deben ser concisas y evitar explicaciones extensas dentro del panel.

RIU-11 Interfaz en inglés. La versión final del panel analítico debe estar traducida al inglés cuando sea necesario para homogeneizar la presentación.

RIU-12 Eliminación de resumen ejecutivo. La interfaz de resultados no debe incluir un resumen ejecutivo redundante si este duplica información ya presente en los bloques principales.

Requisitos éticos y legales

RLE-01 Consentimiento explícito. El registro solo debe iniciarse tras la aceptación expresa del usuario.

RLE-02 Seudonimización. El identificador del usuario debe generarse mediante un mecanismo de hash o técnica equivalente que evite almacenar datos personales directos.

RLE-03 Transparencia. La interfaz debe informar de la naturaleza técnica de los datos recogidos y de su finalidad académica o de validación.

RLE-04 Minimización. No se deben capturar datos personales innecesarios para el análisis del proceso de modelado.

RLE-05 Control de exportación. La exportación del CSV debe requerir una acción consciente del usuario.

RLE-06 Uso académico. Los datos se consideran orientados a investigación, validación del sistema y análisis educativo, sin atribuir conclusiones personales no justificadas.

B.4. Trazabilidad de requisitos

La trazabilidad se plantea en dos niveles. En primer lugar, la Tabla [B.1](#) relaciona los bloques funcionales del sistema con los principales módulos o responsabilidades de implementación. En segundo lugar, la Tabla [B.16](#) vincula cada caso de uso con los requisitos que lo justifican.

Bloque funcional	Requisitos	Módulos o responsabilidades principales
Captura temporal y espacial	RF-01 a RF-04	Temporizador de muestreo, posición de vista, objeto activo, radio de objeto y radio de escena.
Captura topológica y contexto	RF-05 a RF-10	Consulta de malla, evaluación en <i>Object Mode</i> y <i>Edit Mode</i> , uso de bmesh , cambios de modo, objetos y modificadores.
Operadores y cambios UV	RF-11 a RF-16	Interceptación de <i>Ctrl+V</i> , <i>Shift+D</i> , <i>Alt+D</i> , operaciones <i>Merge</i> , operadores UV y firma hash de coordenadas UV.
Persistencia y control de sesión	RF-17 a RF-25	Snapshots, deltas, escritura incremental en CSV, persistencia interna en .blend , restauración, exportación, consentimiento, aviso de <i>Auto Run</i> y panel de control.
Análisis estadístico	RF-26 a RF-34	Detección y lectura de CSV, validación de datos, cálculo de métricas A1–A16, agrupación de métricas y cuartiles selectivos.
Métricas geométricas y UV	RF-35 a RF-46	Área UV, islas UV, <i>stretch</i> , densidad de texeles, normales, ángulos, no cuadrángulos, vértices duplicados, similitud y distancia de Hausdorff cuando proceda.
Visualización e interfaz	RF-47 a RF-50, RIU	Gráficos 2D, visualizaciones 3D, paneles de resultados, bloques compactos, tooltips y separación entre cálculo e interfaz.
Calidad, ética y privacidad	RNF, RLE	Rendimiento, robustez, modularidad, tolerancia a fallos, consentimiento, seudonimización, transparencia y minimización de datos.

Tabla B.1: Trazabilidad resumida entre requisitos, bloques funcionales y responsabilidades de implementación.

B.5. Casos de uso del sistema

Como apoyo visual a la especificación funcional, la Figura B.1 puede sintetizar las interacciones principales entre el usuario, el add-on de captura y el add-on de análisis. La figura se mantiene como elemento recomendado, ya que facilita la lectura de la relación entre captura no intrusiva, generación de CSV, procesamiento de métricas y visualización en Blender.

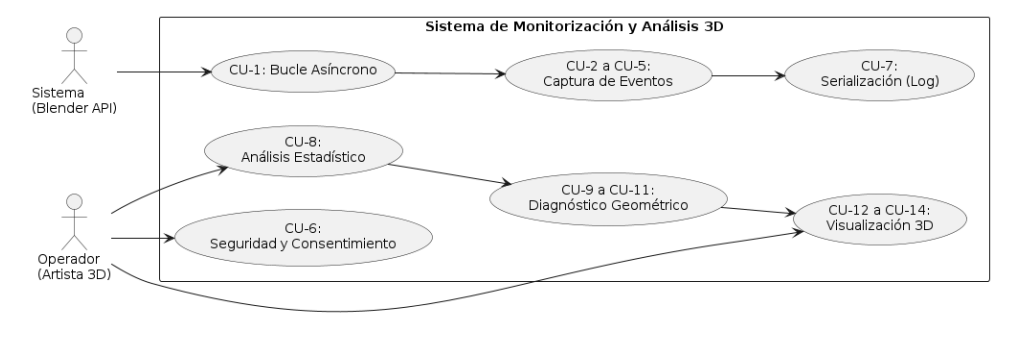


Figura B.1: Diagrama de casos de uso del sistema de monitorización, análisis y visualización.

A continuación se describen los casos de uso principales. Se ha conservado el nivel de detalle necesario para justificar el comportamiento del sistema, pero se evita incluir secuencias excesivamente dependientes de funciones internas concretas. Los nombres de operadores o variables se citan únicamente cuando ayudan a reconocer la correspondencia con la implementación.

CU-01	Inicializar monitorización no intrusiva
Requisitos asociados	RF-01 a RF-04, RF-17, RF-22, RF-24, RNF-01, RNF-02, RLE-01
Descripción	El sistema inicia un temporizador periódico que evalúa el estado de la escena sin bloquear la interfaz de Blender ni alterar el flujo de modelado.
Precondición	El usuario ha aceptado el consentimiento y activa el control de inicio en el panel del Data Logger.
Secuencia principal	El sistema cambia el estado de captura a activo, registra el instante inicial, prepara las estructuras de comparación y comienza a muestrear la escena de forma periódica.
Postcondición	La sesión queda en estado de escucha activa y preparada para registrar cambios relevantes.
Excepciones	El consentimiento no ha sido aceptado, la ejecución automática está deshabilitada o Blender impide registrar el temporizador.
Prioridad	Alta

Tabla B.2: CU-01: Inicializar monitorización no intrusiva.

CU-02	Auditar cambios topológicos de la malla
Requisitos asociados	RF-05, RF-06, RF-07, RF-17, RF-18, RNF-03, RNF-04, RNF-06
Descripción	El sistema compara el estado topológico de la malla activa para detectar variaciones en vértices, triángulos, no cuadrángulos y normales potencialmente invertidas.
Precondición	Existe un objeto de tipo malla activo y la captura se encuentra iniciada.
Secuencia principal	El sistema obtiene la información geométrica disponible, emplea estructuras adecuadas según el modo de trabajo, contrasta los valores con el snapshot anterior y registra solo las diferencias relevantes.
Postcondición	Las variaciones topológicas quedan disponibles para su escritura incremental en el CSV.
Excepciones	No hay malla activa, se produce un cambio de modo durante la lectura o la geometría no contiene datos válidos.
Prioridad	Alta

Tabla B.3: CU-02: Auditar cambios topológicos de la malla.

CU-03	Detectar operadores de teclado relevantes
Requisitos asociados	RF-11, RF-12, RF-13, RF-14, RF-18, RNF-02, RNF-04
Descripción	El sistema identifica operaciones frecuentes de modelado asociadas a combinaciones de teclado, como pegado, duplicación enlazada, duplicación no enlazada y fusión de elementos.
Precondición	El Data Logger está activo y los operadores o atajos relevantes se encuentran registrados.
Secuencia principal	El usuario ejecuta una acción como <i>Ctrl+V</i> , <i>Shift+D</i> , <i>Alt+D</i> o <i>Merge</i> ; el sistema marca el evento, actualiza las banderas correspondientes y fuerza el registro de la operación en la siguiente fila válida.
Postcondición	La operación queda reflejada en el registro cronológico y puede utilizarse posteriormente en las métricas de actividad.
Excepciones	El atajo entra en conflicto con otra herramienta, se modifica el mapa de teclas o la operación no queda expuesta de forma fiable por la API.
Prioridad	Media

Tabla B.4: CU-03: Detectar operadores de teclado relevantes.

CU-04	Capturar transformaciones espaciales del objeto activo
Requisitos asociados	RF-02, RF-03, RF-04, RF-17, RF-18, RNF-01, RNF-03
Descripción	El sistema monitoriza los cambios de posición, escala, radio y contexto espacial asociados al objeto activo y a la escena.
Precondición	La captura está activa y existe un objeto seleccionado o una escena con objetos de tipo malla.
Secuencia principal	El sistema consulta las propiedades espaciales relevantes, las compara con el estado previo mediante tolerancias y registra los deltas cuando existe una modificación significativa.
Postcondición	Los cambios espaciales se incorporan al registro de sesión sin almacenar filas redundantes.
Excepciones	El objeto activo desaparece, se elimina la malla durante la captura o las transformaciones no pueden leerse temporalmente.
Prioridad	Alta

Tabla B.5: CU-04: Capturar transformaciones espaciales del objeto activo.

CU-05	Registrar cambios de modo y contexto de trabajo
Requisitos asociados	RF-06, RF-08, RF-17, RF-18, RNF-04
Descripción	El sistema detecta transiciones entre modos de trabajo, especialmente entre <i>Object Mode</i> y <i>Edit Mode</i> , para contextualizar las operaciones de modelado.
Precondición	La monitorización está activa y Blender informa correctamente del modo de contexto.
Secuencia principal	El usuario cambia de modo; el sistema compara el modo actual con el anterior y registra la transición junto con la marca temporal correspondiente.
Postcondición	El cambio contextual queda registrado y puede emplearse en métricas relacionadas con estados de trabajo.
Excepciones	Pérdida temporal de foco, cambio de área de Blender o consulta de contexto no válida.
Prioridad	Media

Tabla B.6: CU-05: Registrar cambios de modo y contexto de trabajo.

CU-06	Gestionar consentimiento y seudonimización
Requisitos asociados	RF-22, RF-23, RLE-01, RLE-02, RLE-03, RLE-04
Descripción	El sistema solicita aceptación explícita antes de iniciar la captura y evita almacenar identificadores personales directos.
Precondición	El add-on se encuentra instalado o cargado en Blender.
Secuencia principal	El sistema comprueba si existe aceptación previa; si no existe, muestra el aviso correspondiente, bloquea la captura y solo permite continuar tras la aceptación informada.
Postcondición	La captura queda habilitada o bloqueada según el estado del consentimiento.
Excepciones	No se puede guardar la preferencia local o el usuario rechaza el consentimiento.
Prioridad	Alta

Tabla B.7: CU-06: Gestionar consentimiento y seudonimización.

CU-07	Persistir y exportar el registro CSV
Requisitos asociados	RF-18, RF-19, RF-20, RF-21, RLE-05, RIU-03, RNF-09
Descripción	El sistema conserva los datos registrados durante la sesión y permite exportarlos a un fichero CSV externo.
Precondición	Existen filas registradas o un bloque interno recuperable dentro del archivo <code>.blend</code> .
Secuencia principal	El sistema mantiene el registro incremental, puede incrustar una copia interna en el proyecto y exporta el CSV cuando el usuario lo solicita mediante la interfaz.
Postcondición	El registro queda disponible como archivo externo para su análisis posterior.
Excepciones	Fallo de escritura, permisos insuficientes, ruta no válida o ausencia de registros exportables.
Prioridad	Alta

Tabla B.8: CU-07: Persistir y exportar el registro CSV.

CU-08	Cargar y validar colecciones de CSV
Requisitos asociados	RF-26, RF-27, RF-28, RNF-05, RNF-11
Descripción	El add-on de análisis localiza uno o varios CSV, comprueba su estructura y prepara los datos para el cálculo de métricas.
Precondición	El usuario selecciona un archivo o carpeta con registros generados por el Data Logger.
Secuencia principal	El sistema detecta los CSV, lee sus cabeceras y valores, descarta o informa de datos no válidos y carga en memoria las series numéricas utilizables.
Postcondición	Los datos quedan preparados para el análisis estadístico y la visualización.
Excepciones	CSV vacío, columnas ausentes, codificación incompatible o valores no finitos.
Prioridad	Alta

Tabla B.9: CU-08: Cargar y validar colecciones de CSV.

CU-09	Calcular métricas estadísticas A1–A16
Requisitos asociados	RF-29 a RF-34, RIU-06, RIU-07, RIU-08, RIU-10, RIU-12
Descripción	El sistema calcula las métricas estadísticas de la sesión o conjunto de sesiones, aplicando agrupaciones y cuartiles selectivos cuando corresponda.
Precondición	Existen datos CSV validados.
Secuencia principal	El usuario solicita el cálculo; el sistema obtiene la duración en horas, actividad general, pausas, picos, velocidades, tamaños locales, transiciones y métricas de eventos, agrupando A2–A3 y A6–A9 cuando procede y manteniendo separadas A12, A13 y A16.
Postcondición	Las métricas A1–A16 quedan disponibles para su presentación en paneles.
Excepciones	Series temporales incompletas, duración nula o ausencia de columnas necesarias.
Prioridad	Alta

Tabla B.10: CU-09: Calcular métricas estadísticas A1–A16.

CU-10	Calcular similitud geométrica y distancia de Hausdorff
Requisitos asociados	RF-43, RF-44, RF-45, RF-46, RNF-06, RNF-12
Descripción	El sistema compara un objeto con una referencia para estimar similitud geométrica, similitud topológica y distancia de Hausdorff o una aproximación equivalente cuando resulte aplicable.
Precondición	Existe un objeto seleccionado y, si la métrica lo requiere, un objeto de referencia válido.
Secuencia principal	El sistema obtiene coordenadas transformadas al espacio global, compara vértices y estructura topológica, penaliza diferencias relevantes y calcula el indicador de similitud o distancia.
Postcondición	Los resultados de comparación quedan disponibles en el panel de análisis.
Excepciones	Ausencia de referencia, geometría sin vértices válidos, diferencias extremas de escala o mallas no comparables.
Prioridad	Alta

Tabla B.11: CU-10: Calcular similitud geométrica y distancia de Hausdorff.

CU-11	Analizar mapas UV
Requisitos asociados	RF-36, RF-37, RF-38, RF-39, RNF-06, RNF-12
Descripción	El sistema evalúa la capa UV activa para calcular área UV, número de islas, deformación y densidad de texeles.
Precondición	El objeto seleccionado posee una malla con coordenadas UV válidas.
Secuencia principal	El sistema consulta la capa UV activa, recorre la conectividad de las coordenadas, estima áreas o relaciones geométricas y presenta las métricas resultantes.
Postcondición	Las métricas UV se incorporan al bloque correspondiente de resultados.
Excepciones	Objeto sin UV, capa UV vacía, caras degeneradas o coordenadas inconsistentes.
Prioridad	Media

Tabla B.12: CU-11: Analizar mapas UV.

CU-12	Diagnosticar normales, ángulos y estructura de malla
Requisitos asociados	RF-35, RF-40, RF-41, RF-42, RF-43, RNF-06, RNF-12
Descripción	El sistema calcula métricas de calidad geométrica relacionadas con normales, ángulos entre caras, polígonos no cuadrangulares y vértices duplicados.
Precondición	Existe una malla seleccionada y evaluable.
Secuencia principal	El sistema inspecciona la geometría, calcula porcentajes o recuentos relevantes y genera indicadores de diagnóstico para apoyar la interpretación de la calidad del modelo.
Postcondición	Los resultados geométricos quedan visibles en el panel de análisis.
Excepciones	Malla vacía, caras degeneradas, normales nulas o datos no aplicables al tipo de objeto.
Prioridad	Media

Tabla B.13: CU-12: Diagnosticar normales, ángulos y estructura de malla.

CU-13	Generar gráficos estadísticos 2D
Requisitos asociados	RF-47, RIU-05, RIU-07, RIU-08, RIU-10, RNF-07
Descripción	El sistema genera representaciones gráficas bidimensionales a partir de las series de datos procesadas, integrándolas en Blender cuando sea necesario.
Precondición	Existen datos numéricos válidos y se ha seleccionado un tipo de gráfico compatible.
Secuencia principal	El usuario solicita una visualización; el sistema prepara los vectores de datos, genera el gráfico mediante una librería de representación y lo muestra como resultado asociado a la sesión.
Postcondición	El gráfico queda disponible para la interpretación visual de las métricas.
Excepciones	Datos insuficientes, tipo de gráfico incompatible o error de renderizado.
Prioridad	Media

Tabla B.14: CU-13: Generar gráficos estadísticos 2D.

CU-14	Crear visualizaciones procedurales 3D en Blender
Requisitos asociados	RF-48, RF-49, RF-50, RNF-07, RNF-08, RIU-05
Descripción	El sistema traduce resultados analíticos en elementos visuales tridimensionales dentro de la escena de Blender, como nubes de puntos, superficies o mapas de radar.
Precondición	Existen métricas calculadas y una visualización tridimensional aplicable.
Secuencia principal	El sistema limpia visualizaciones anteriores si procede, calcula posiciones o relaciones espaciales, genera la geometría procedural y asigna materiales o etiquetas necesarias para su lectura.
Postcondición	La visualización queda integrada en la escena y puede inspeccionarse desde Blender.
Excepciones	Datos nulos, escala no representable, fallo al crear geometría o ausencia de contexto de escena válido.
Prioridad	Media

Tabla B.15: CU-14: Crear visualizaciones procedurales 3D en Blender.

B.6. Trazabilidad entre casos de uso y requisitos

Caso de uso	Requisitos asociados
CU-01	RF-01 a RF-04, RF-17, RF-22, RF-24, RNF-01, RNF-02, RLE-01
CU-02	RF-05, RF-06, RF-07, RF-17, RF-18, RNF-03, RNF-04, RNF-06
CU-03	RF-11 a RF-14, RF-18, RNF-02, RNF-04
CU-04	RF-02, RF-03, RF-04, RF-17, RF-18, RNF-01, RNF-03
CU-05	RF-06, RF-08, RF-17, RF-18, RNF-04
CU-06	RF-22, RF-23, RLE-01 a RLE-04
CU-07	RF-18 a RF-21, RLE-05, RIU-03, RNF-09
CU-08	RF-26, RF-27, RF-28, RNF-05, RNF-11
CU-09	RF-29 a RF-34, RIU-06 a RIU-08, RIU-10, RIU-12
CU-10	RF-43 a RF-46, RNF-06, RNF-12
CU-11	RF-36 a RF-39, RNF-06, RNF-12
CU-12	RF-35, RF-40 a RF-43, RNF-06, RNF-12
CU-13	RF-47, RIU-05, RIU-07, RIU-08, RIU-10, RNF-07
CU-14	RF-48, RF-49, RF-50, RNF-07, RNF-08, RIU-05

Tabla B.16: Trazabilidad entre casos de uso y requisitos.

Apéndice C

Especificación de Diseño

C.1. Introducción

Este apéndice describe el diseño de la *suite* de monitorización, análisis y visualización desarrollada para Blender, dentro del contexto de los sistemas de interacción 3D y de la analítica de procesos basada en registros de eventos [4, 20]. La solución se organiza en dos *add-ons* complementarios. El primero, *Data Logger 3D*, se ejecuta durante la sesión de modelado y registra eventos, estados y variaciones relevantes en formato CSV. El segundo *add-on* trabaja de forma posterior sobre esos CSV, valida los datos, calcula métricas estadísticas A1–A16, métricas geométricas y UV, y presenta los resultados mediante paneles y visualizaciones integradas en Blender.

La información de este anexo se ordena desde los elementos más estables del diseño hasta los aspectos más próximos a la interfaz. En primer lugar se define el modelo de datos; después se describe la arquitectura general; a continuación se detallan los dos *add-ons*; finalmente se documentan las métricas, la interfaz y las decisiones de diseño. Esta organización evita mezclar decisiones de arquitectura con instrucciones de uso o detalles de programación, que quedan desarrollados en los anexos D y E.

C.2. Diseño de datos

El sistema no utiliza una base de datos relacional. La persistencia de datos se realiza mediante archivos CSV generados por el *add-on* Data Logger 3D y posteriormente consumidos por el *add-on* de análisis. Por este motivo, no procede incluir un diagrama entidad-relación ni un modelo relacional

clásico. En su lugar, el diseño de datos se documenta mediante la descripción de los campos del CSV y las variables derivadas calculadas durante el análisis.

Campos completos del CSV

El CSV constituye el trueque entre el *add-on* de captura y el *add-on* de análisis. El uso de un formato tabular intermedio resulta adecuado para una fase posterior de limpieza, validación y análisis exploratorio de datos [12, 6]. Cada fila representa un cambio significativo de la sesión, no una muestra periódica indiferenciada. La generación de filas se basa en la comparación entre un *snapshot* actual y una línea base anterior, de forma que solo se registran deltas cuando se detecta una variación relevante.

La Tabla C.1 recoge los campos completos del CSV utilizado como formato intermedio entre el módulo de captura y el módulo de análisis.

Tabla C.1: Campos completos del CSV utilizado como formato intermedio entre captura y análisis.

Grupo	Campo	Descripción de diseño
Identificación	SchemaVersion	Versión del esquema CSV utilizado. En la versión actual toma el valor 2.
Identificación	LoggerVersion	Versión del <i>add-on Data Logger 3D</i> que generó el archivo CSV. Permite relacionar los datos con la versión concreta del software.
Identificación	SessionID	Identificador único de la sesión de registro. Permite diferenciar sesiones distintas, incluso cuando pertenecen al mismo usuario.
Identificación	UserID	Identificador seudónimo persistente asociado al usuario en el esquema v2. Sustituye al campo heredado <code>USER_ID</code> del esquema v1 y no almacena el nombre real del usuario.
Tiempo	TimeStamp	Marca temporal absoluta utilizada para ordenar registros.
Tiempo	Minute, Second	Tiempo relativo desde el inicio de la captura. Facilita el análisis temporal de la sesión.
Vista del usuario	UserX, UserY, UserZ	Posición estimada de la cámara o vista 3D del usuario.
Escena	SceneRadius	Radio aproximado de la escena calculado a partir de objetos de tipo malla.
Objeto activo	ObjX, ObjY, ObjZ	Centro del objeto activo en coordenadas de mundo.
Objeto activo	ObjRadius	Radio aproximado del objeto activo a partir de su caja envolvente.
Deltas espaciales	ObjDeltaX, ObjDeltaY, ObjDeltaZ	Variación espacial del objeto activo respecto al estado anterior.
Deltas espaciales	ObjDeltaRadius	Cambio del radio del objeto activo. Se utiliza como indicio de transformación o escala.
Topología	VertexDelta	Variación en el número de vértices.
Topología	NgonDelta	Variación en el número de caras con más de cuatro lados.
Topología	TriDelta	Variación en el número de triángulos.
Topología	NormalDelta	Variación de normales potencialmente invertidas.
Modo	ObjModeState	Indica si el contexto se encuentra en <i>Object Mode</i> .
Modo	EditModeState	Indica si el contexto se encuentra en <i>Edit Mode</i> .
Modo	ModeChanged	Marca transiciones entre modos de trabajo.
UV	UV	Marca eventos o cambios detectados en coordenadas UV.
Escena	ObjectDelta	Cambio en el número de objetos de la escena.
Escena	ModifierDelta	Cambio en el número total de modificadores.
Operadores	CtrlV, ShiftD, AltD, Merge	Banderas de operadores relevantes para el proceso de modelado.

Continúa en la página siguiente

Tabla C.1: Campos completos del CSV utilizado como formato intermedio entre captura y análisis (continuación).

Grupo	Campo	Descripción de diseño
Visualización	Occlusion	Estado asociado a modos de visualización como <i>wireframe</i> o <i>x-ray</i> .

Compatibilidad entre versiones del esquema CSV

Durante el desarrollo del proyecto el formato CSV ha evolucionado desde una primera versión funcional hasta un esquema más estructurado y versionado. En la versión inicial, denominada esquema v1, el identificador seudónimo del usuario se almacenaba en el campo `USER_ID`. Este formato permitía asociar registros a un mismo usuario, pero no incluía información explícita sobre la versión del esquema, la versión del logger ni la sesión concreta de registro.

En la versión actual, denominada esquema v2, se introduce una cabecera más completa formada, entre otros, por los campos `SchemaVersion`, `LoggerVersion`, `SessionID` y `UserID`. En este nuevo esquema, el campo `UserID` sustituye a `USER_ID` como identificador seudónimo del usuario. La diferencia de nombre no implica dos identificadores distintos, sino una evolución del trueque de datos entre *Data Logger 3D* y *Analysis 3D*.

Para mantener compatibilidad con registros generados por versiones anteriores del logger, *Analysis 3D* detecta automáticamente si el CSV cargado sigue el esquema v1 o v2. En caso de cargar un archivo v1, el sistema normaliza internamente sus campos al formato actual, de forma que los datos antiguos puedan analizarse sin intervención manual.

Versión	Campo de usuario	Descripción
v1	<code>USER_ID</code>	Campo heredado utilizado en las primeras versiones del logger para identificar de forma seudónima al usuario. No incluía campos explícitos de versionado del esquema.
v2	<code>UserID</code>	Campo actual del esquema CSV. Se utiliza junto con <code>SchemaVersion</code> , <code>LoggerVersion</code> y <code>SessionID</code> para mejorar la trazabilidad de los registros.

Tabla C.2: Diferencias entre los campos de identificación de usuario en las versiones del esquema CSV.

Variables derivadas del análisis

El *add-on* de análisis conserva los datos originales y calcula variables derivadas únicamente en memoria. Estas variables permiten obtener métri-

cas agregadas sin alterar el CSV fuente. Las derivaciones principales son: duración total de sesión, diferencias temporales entre filas, velocidad de movimiento de la vista, distancia a objeto, actividad de edición por fila, cambios de modo, eventos UV, evolución de vértices, evolución de errores topológicos, métricas de *Object Mode*, métricas geométricas del objeto activo y métricas UV.

Variable derivada	Origen y finalidad
Tiempo transcurrido	Se obtiene a partir de <code>TimeStamp</code> o de <code>Minute/Second</code> ; permite calcular duración y pausas.
Velocidad de vista	Se calcula con posiciones <code>UserX</code> , <code>UserY</code> , <code>UserZ</code> y diferencias temporales; sirve para métricas espaciales.
Distancia a objeto	Relaciona la posición de la vista con <code>ObjX</code> , <code>ObjY</code> , <code>ObjZ</code> ; permite estudiar proximidad.
Actividad de edición	Agrega deltas topológicos, cambios de objeto, modificadores, UV y operadores; permite distinguir actividad e inactividad.
Tamaño local del objeto	Se aproxima con dimensiones, escala, tamaño disponible o <code>ObjRadius</code> ; se usa en métricas de relación espacial.
Distribuciones por métrica	Generan mínimo, máximo, media y cuartiles solo cuando la métrica tiene distribución significativa.

Tabla C.3: Variables derivadas principales calculadas por el *add-on* de análisis.

C.3. Diseño arquitectónico

La arquitectura se estructura en dos *add-ons* conectados mediante un CSV intermedio. Esta división responde a criterios clásicos de modularidad y separación de responsabilidades, donde cada componente encapsula una función concreta del sistema [16, 8]. Esta separación permite que la captura sea ligera y que el análisis se ejecute bajo demanda. El primer *add-on* no necesita conocer las métricas finales; el segundo *add-on* no necesita intervenir durante la sesión de modelado. La Figura C.1 resume esta relación entre captura, exportación, análisis y visualización.

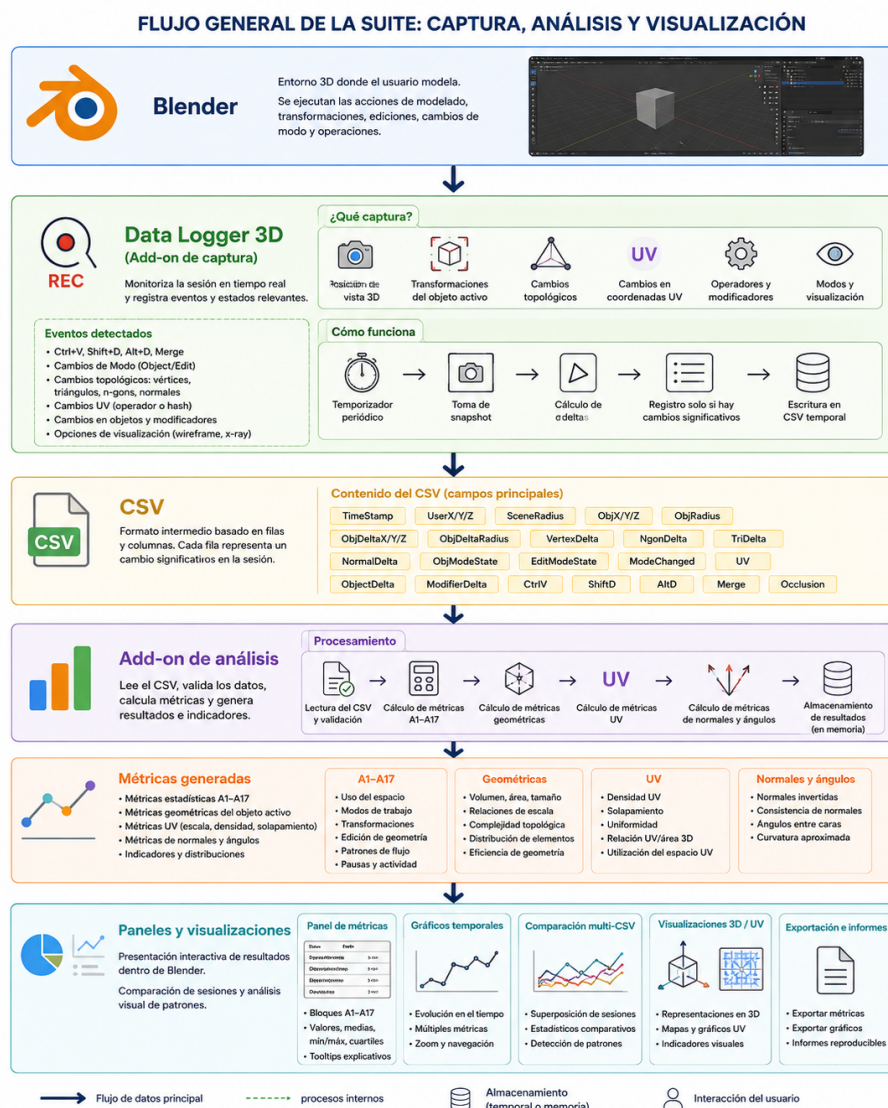


Figura C.1: Flujo general de la *suite*: captura, exportación CSV, análisis de métricas y visualización en Blender.

Nota. Figura elaborada por la autora con asistencia de ChatGPT.

Como se observa en la Figura C.1, el CSV actúa como punto de unión entre los dos *add-ons*. Esta decisión evita acoplar directamente el proceso de captura con el cálculo de métricas y permite revisar los datos de forma independiente antes del análisis.

Componente	Responsabilidad de diseño
<i>Data Logger 3D</i>	Captura estados, operadores y deltas durante el modelado. Debe ser no intrusivo y robusto ante cambios de contexto.
CSV intermedio	Actúa como contrato estable, auditable y portable entre captura y análisis.
<i>add-on</i> de análisis	Lee, valida, calcula métricas y genera visualizaciones sin modificar los datos originales.
Paneles de Blender	Proporcionan acceso a captura, exportación, selección de CSV, cálculo e interpretación de resultados.

Tabla C.4: Responsabilidades principales de la arquitectura.

Modelo de componentes y dependencias internas

El diseño arquitectónico no se limita a la existencia de dos *add-ons*, sino que define una separación de responsabilidades entre captura, persistencia, análisis, cálculo e interfaz. Esta distribución se representa en la Figura C.2, donde se observa cómo cada componente tiene una función delimitada dentro de la *suite*.

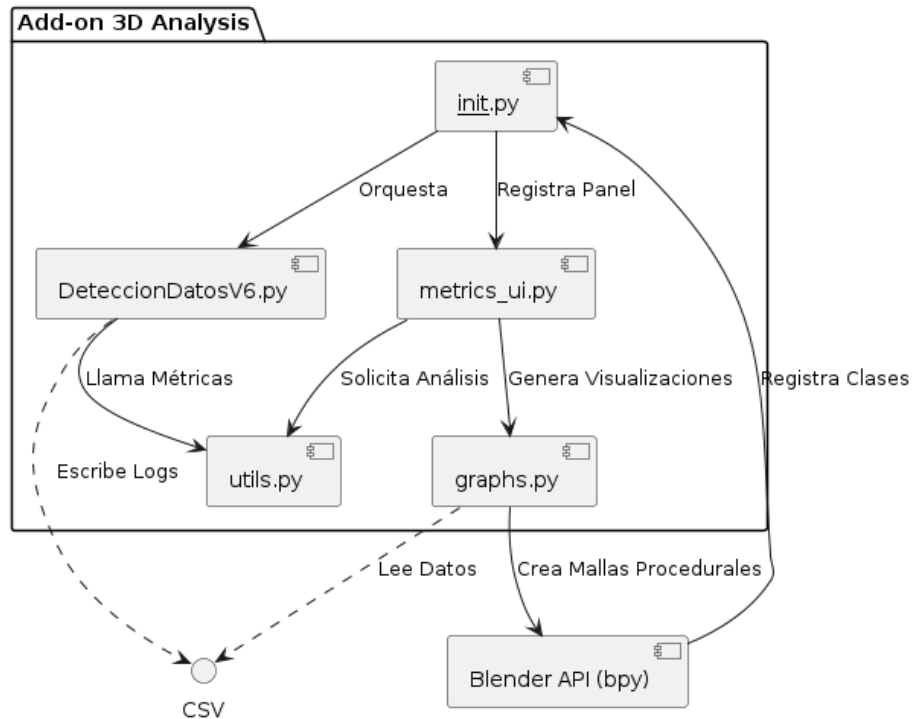


Figura C.2: Organización general de componentes y comunicación entre el logger, el CSV y el *add-on* de análisis.

La Figura C.2 permite identificar las dependencias principales del sistema. El logger depende del contexto de Blender para capturar cambios, pero no depende de los módulos de análisis. A su vez, el *add-on* de análisis depende del CSV exportado y de los objetos seleccionados en Blender cuando se calculan métricas de malla, pero no necesita intervenir durante la captura.

Capa de diseño	Componentes asociados	Criterio de separación
Captura	<i>Data Logger 3D</i> , temporizador, handlers, operadores	Se ejecuta durante el modelado y debe ser ligera, por lo que solo registra cambios significativos.
Persistencia	CSV temporal, bloque interno <code>data_log_internal.csv</code> , exportación	Conserva los datos en un formato estable y revisable, independiente de la interfaz de análisis.
Procesamiento	Lectura, validación, métricas A1–A16, métricas geométricas y UV	Trabaja bajo demanda y puede realizar cálculos más costosos sin afectar a la captura.
Presentación	Paneles, pestañas, tooltips, gráficos y visualizaciones 3D	Muestra resultados al usuario sin modificar los datos originales ni mezclar cálculo con interfaz.

Tabla C.5: Capas lógicas utilizadas para organizar el diseño de la suite.

Diseño modular del código

La organización del código responde a un criterio de bajo acoplamiento, siguiendo el principio de descomposición modular para reducir dependencias internas y facilitar el mantenimiento [16, 17]. El primer *add-on* se mantiene como un módulo único porque su responsabilidad principal es la captura en tiempo real. En cambio, el *add-on* de análisis se divide en módulos especializados para separar lectura de datos, cálculo de métricas, generación de gráficos, propiedades de interfaz y operadores de usuario. Esta separación se resume en la Figura C.3.

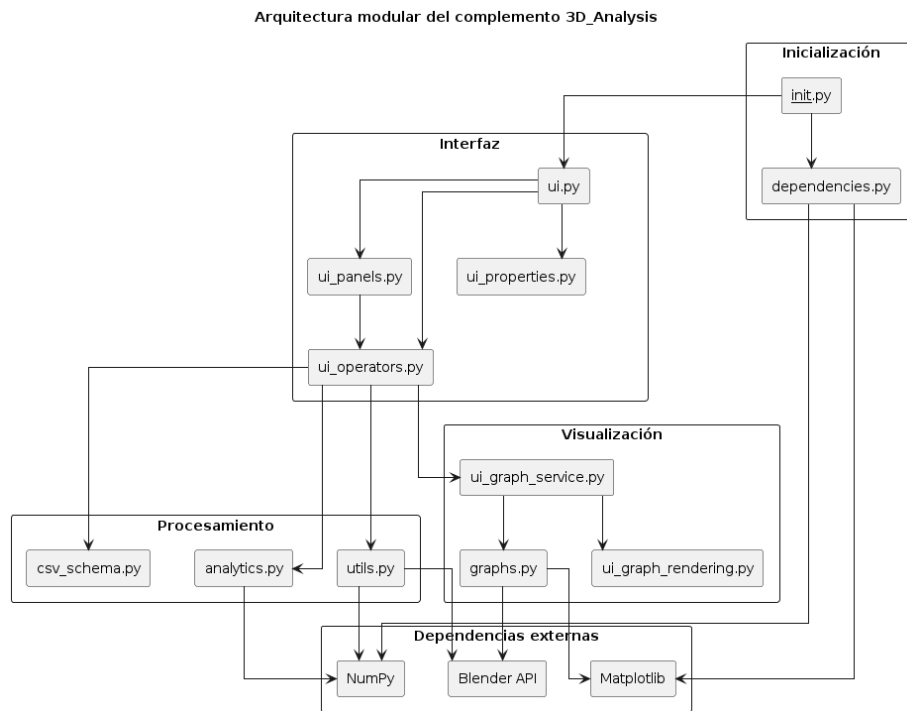


Figura C.3: Estructura modular del código y separación entre lógica de cálculo, interfaz y visualización.

Como muestra la Figura C.3, los módulos de cálculo se mantienen separados de los paneles de interfaz. Esta decisión facilita la depuración de métricas, la sustitución de funciones de cálculo y la incorporación de nuevas visualizaciones sin modificar la lectura del CSV ni la estructura principal del *add-on*.

Interacción entre captura, análisis e interfaz

El flujo de interacción se diseña como una secuencia cerrada: primero se captura, después se persiste, posteriormente se analiza y finalmente se visualiza. La comunicación entre fases se realiza mediante el CSV y las estructuras de resultados generadas en memoria por el *add-on* de análisis. La Figura C.4 representa esta interacción completa entre Blender, el logger, el CSV y el *add-on* de análisis.

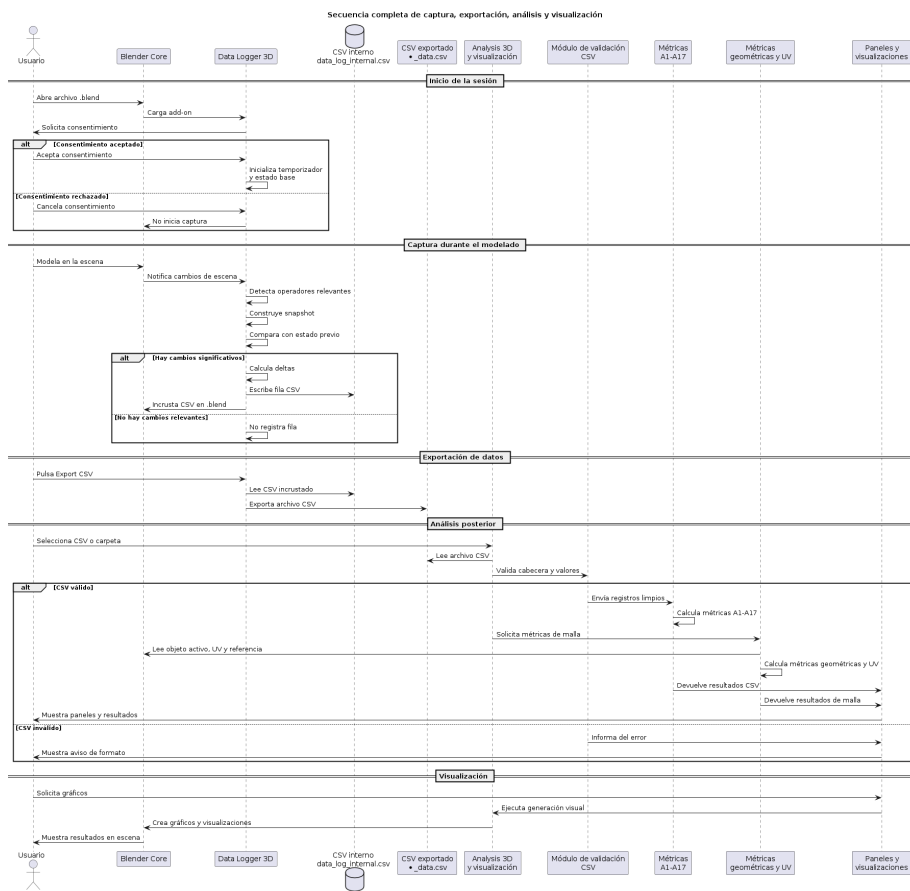


Figura C.4: Secuencia de interacción completa entre Blender, el *add-on* Data Logger 3D, el CSV intermedio y el *add-on* de análisis y visualización.

En la Figura C.4 se observa que el usuario no interactúa directamente con las funciones internas de cálculo. La interfaz lanza operadores de Blender, estos operadores invocan la lógica correspondiente y los resultados se devuelven al panel o a la escena como elementos visuales. Este diseño evita que el usuario tenga que manipular archivos internos o estructuras de datos intermedias.

C.4. Diseño del primer *add-on*: Data Logger 3D

El primer *add-on* se encarga de registrar los cambios relevantes que se producen durante la sesión de modelado. Su diseño se relaciona con los

enfoques de análisis de procesos basados en eventos, donde las acciones se registran para reconstruir posteriormente el comportamiento observado [20]. Su prioridad es registrar información útil sin ralentizar la interacción del usuario. Para ello, se organizan varios subsistemas que comparten un estado global mínimo.

Subsistema	Criterio de diseño
Estado global	Mantiene temporizador, línea base anterior, modo previo, objeto activo previo, hash UV y banderas de operadores.
Captura espacial	Obtiene posición de la vista, centro del objeto activo, radio del objeto y radio de escena mediante matrices y cajas envolventes.
Captura topológica	Cuenta vértices, triángulos, n-gons y normales potencialmente invertidas. Usa bmesh cuando el objeto está en edición.
Operadores	Detecta Ctrl+V , Shift+D , Alt+D y Merge mediante operadores recientes, normalización de identificadores y banderas.
UV	Combina operadores UV conocidos con un hash global de coordenadas UV para detectar cambios que no aparecen en contadores topológicos.
Snapshots y deltas	Compara firma actual y firma previa; solo registra si existe cambio relevante o una bandera forzada.
Temporizador	Ejecuta la comprobación de forma periódica y devuelve un intervalo corto mientras el logger está activo.
Consentimiento	Exige aceptación antes de iniciar la captura y recuerda el estado mediante un bloque de texto interno.
Paneles	Ofrecen control básico de inicio/parada, exportación y depuración.
Exportación	Incrusta el CSV en el archivo .blend y permite exportarlo junto al proyecto.

Tabla C.6: Diseño interno del *add-on* Data Logger 3D.

El diseño del logger puede entenderse como una máquina de estados sencilla. Mientras el logger está detenido no se escriben filas. Cuando el usuario acepta el consentimiento, el sistema inicializa la línea base, activa el temporizador y pasa al estado de captura. En cada ciclo se compara la escena actual con el *snapshot* anterior; si no existen cambios, no se escribe nada, y si existe una variación relevante, se genera una fila CSV y se actualiza la línea base.

El funcionamiento del logger se divide en dos fases principales. La primera corresponde a la detección de cambios durante la sesión de modelado, representada en la Figura C.5. La segunda corresponde a la persistencia y exportación del CSV generado, representada en la Figura C.6. Esta separación permite distinguir entre el registro automático de la actividad y la obtención posterior de un archivo CSV externo para el análisis.

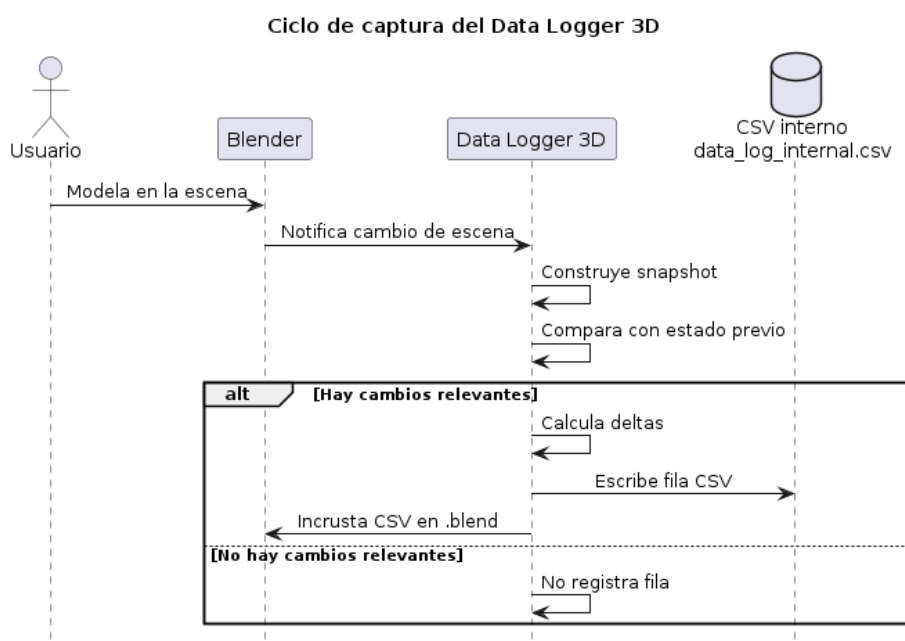


Figura C.5: Flujo interno de detección de cambios del Data Logger 3D durante una sesión de modelado.

La Figura C.5 muestra la fase de observación y detección. En esta parte se obtiene el estado actual de Blender, se identifican datos espaciales, topológicos, de modo, operadores y UV, y se construye una representación resumida de la sesión. Esta información se compara con la línea base anterior para decidir si existe un cambio suficientemente relevante como para registrar una nueva fila.

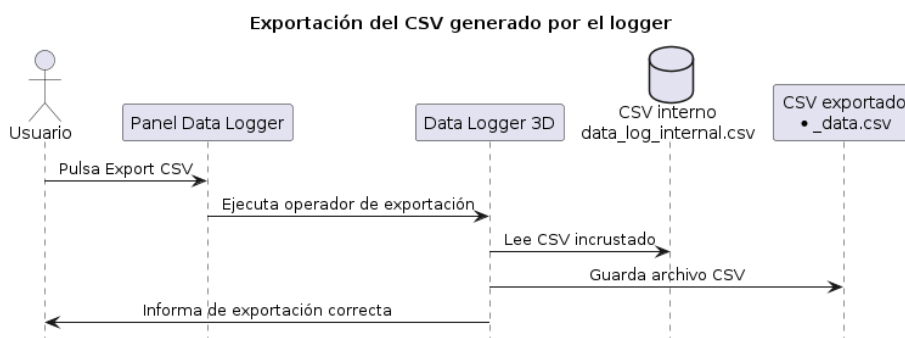


Figura C.6: Flujo de persistencia y exportación del CSV generado por el Data Logger 3D.

La Figura C.6 representa la fase de persistencia y exportación. Durante la captura, los registros se conservan como CSV interno asociado al archivo `.blend`. Cuando el usuario solicita la exportación, el *add-on* recupera ese contenido interno y genera un archivo CSV externo en la carpeta del proyecto. Este archivo actúa como entrada para el *add-on* de análisis y permite revisar los datos fuera de Blender si fuera necesario.

A partir de las fases mostradas en las Figuras C.5 y C.6, el comportamiento del logger puede resumirse como una máquina de estados sencilla. La Tabla C.7 recoge los estados principales y las acciones asociadas a cada transición.

Estado	Entrada o evento	Acción de diseño
Detenido	<i>add-on</i> cargado sin captura activa	No se escriben registros y el panel solo muestra controles de inicio o consentimiento.
Inicialización	Consentimiento aceptado o inicio manual	Se crea o restaura el CSV, se calcula el primer snapshot y se fija la línea base.
Captura	Ciclo del temporizador	Se comparan estado actual y estado previo, se detectan operadores y se evalúa si procede registrar.
Persistencia	Cambio relevante detectado	Se escribe una fila, se eliminan duplicados y se incrusta el CSV en el archivo <code>.blend</code> .
Exportación	Acción del usuario	Se vuelca el CSV incrustado a un archivo externo junto al proyecto.

Tabla C.7: Estados principales del diseño de ejecución del Data Logger 3D.

C.5. Diseño del segundo *add-on*: análisis y visualización

El segundo *add-on* se diseña como una capa de procesamiento posterior. Esta decisión aproxima la herramienta a un flujo de análisis exploratorio: primero se validan los datos, después se calculan métricas y finalmente se generan representaciones visuales interpretables [6, 13]. La lectura CSV, la validación, el cálculo estadístico, el cálculo geométrico y la interfaz se mantienen separados para reducir dependencias cruzadas y facilitar el mantenimiento.

Bloque	Responsabilidad de diseño
Lectura CSV	Localiza archivos individuales o carpetas y convierte las filas a estructuras procesables.
Validación	Comprueba cabeceras, valores numéricos, ausencia de infinitos y tratamiento de datos no disponibles.
Cálculo estadístico	Calcula A1–A16 a partir de posiciones, tiempos, deltas, modos y operadores.
Cálculo geométrico	Evalúa propiedades de malla, similitud, distancia de Hausdorff, normales, ángulos y topología.
Cálculo UV	Calcula área UV, islas, estiramiento y densidad de texeles, entendida como la relación entre resolución de textura y superficie del modelo.
Visualización	Genera gráficos, objetos procedurales y representaciones comparativas dentro de Blender.
Interfaz	Presenta resultados en paneles compactos, con etiquetas breves y tooltips explicativos.

Tabla C.8: Diseño interno del *add-on* de análisis y visualización.

El diseño del *add-on* de análisis sigue una arquitectura por capas. Las rutas seleccionadas por el usuario se gestionan desde la interfaz, la lectura transforma los CSV en estructuras internas, la validación filtra valores no válidos y los módulos de cálculo generan resultados independientes de la presentación. Finalmente, la capa de interfaz decide si esos resultados se muestran como texto, bloques de métricas, gráficos o visualizaciones en la escena.

La secuencia de funcionamiento del segundo *add-on* se muestra en la Figura C.7. A diferencia del logger, este componente se ejecuta bajo demanda: el usuario selecciona un CSV o una carpeta, el sistema valida los datos y posteriormente distribuye la información entre los módulos de cálculo y visualización.

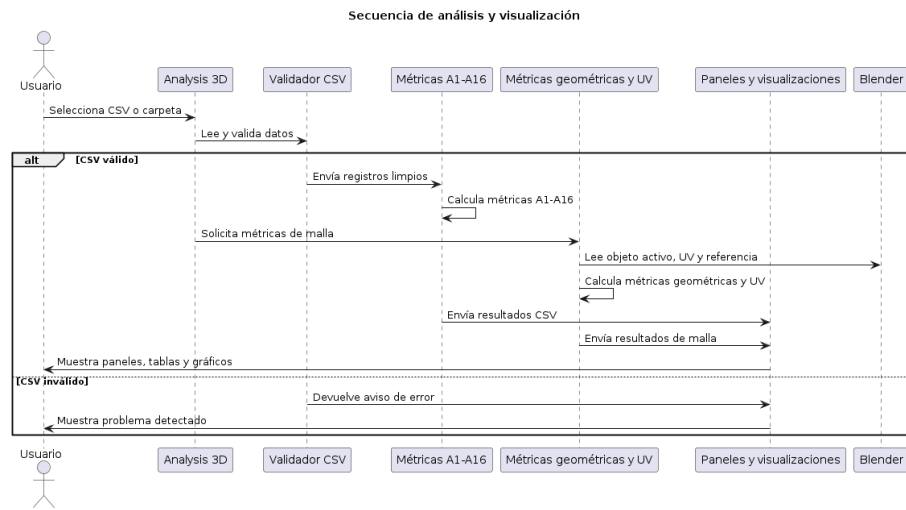


Figura C.7: Secuencia de análisis, cálculo de métricas y visualización de resultados en el segundo *add-on*.

Como se observa en la Figura C.7, el cálculo estadístico y el cálculo de métricas de malla se mantienen separados. Esta separación permite analizar sesiones completas a partir del CSV y, al mismo tiempo, calcular métricas geométricas o UV sobre el objeto activo de Blender cuando el usuario lo solicita.

La Tabla C.9 resume esta transformación desde las entradas de usuario hasta las salidas generadas por el *add-on*. De este modo se documenta qué recibe cada módulo lógico y qué resultado produce dentro del diseño general.

Módulo lógico	Entrada	Salida de diseño
Lectura CSV	Ruta de archivo o carpeta	Lista de registros normalizados para cálculo.
Validación	Registros leídos	Datos numéricos seguros, sin valores infinitos ni campos incompatibles.
Métricas A1–A16	Datos temporales, espaciales y de actividad	Diccionario de métricas estadísticas y distribuciones.
Métricas de malla	Objeto activo y, si procede, referencia	Valores geométricos, UV, similitud y distancias.
Visualización	Resultados calculados	Paneles, gráficos 2D, forest plot, radar y objetos gráficos en Blender.

Tabla C.9: Transformación de entradas y salidas en el diseño del *add-on* de análisis.

C.6. Diseño procedimental

El procedimiento general se divide en captura, persistencia, análisis y visualización. Esta secuencia ya se ha representado a nivel arquitectónico en la Figura C.1 y a nivel de componentes en la Figura C.2. En este apartado se recoge el mismo flujo de forma operativa, indicando el orden lógico de ejecución.

1. Se abre Blender y se activa el *add-on Data Logger 3D*.
2. El sistema comprueba Auto Run y solicita consentimiento si procede.
3. Se inicializa o restaura el CSV asociado a la sesión.
4. El temporizador construye snapshots y detecta cambios significativos.
5. Las filas se escriben, se depuran duplicados y se incrustan en el `.blend`.
6. El CSV se exporta cuando el usuario lo solicita.
7. El *add-on* de análisis lee uno o varios CSV y valida sus datos.
8. Se calculan métricas A1–A16, métricas geométricas y métricas UV.
9. Los resultados se presentan en paneles y visualizaciones dentro de Blender.

C.7. Diseño de métricas A1–A16

Las métricas A1–A16 se organizan por tipología para mejorar su lectura. La interfaz no debe repetir valores equivalentes ni mostrar cuartiles cuando no aportan información interpretativa. Por ello, se agrupan A2–A3 y A6–A9, mientras que A12, A13 y A16 conservan distribución propia.

ID	Etiqueta	Tipo	Diseño de presentación
A1	Total time	Temporal	Duración total, sin cuartiles.
A2	Breaks	Temporal	Agrupada con A3 por comparar interpretación temporal.
A3	Breaks by phase	Temporal	Agrupada con A2; resume distribución de pausas por fase.
A4	Movement speed	Spatial	Muestra resumen estadístico de velocidad.
A5	Proximity to object	Spatial	Muestra relación espacial con el objeto activo.
A6	View manipulation	Spatial	Agrupada con A7–A9 para evitar bloques redundantes.
A7	Speed during inactivity	Spatial	Interpretación conjunta con actividad/inactividad.
A8	Speed during activity	Spatial	Comparación frente a velocidad en inactividad.
A9	Distance travelled	Spatial	Completa el bloque de desplazamiento.
A10	Acceleration strategy 1	Strategy	Bloque independiente de estrategia.
A11	Acceleration strategy 2	Strategy	Bloque independiente de estrategia.
A12	Vertex evolution	Strategy	Mantiene distribución propia y cuartiles.
A13	Error evolution	Strategy	Mantiene distribución propia y cuartiles.
A14	Object Mode	Strategy	Se etiqueta explícitamente como métrica de <i>Object Mode</i> .
A15	UV work	Strategy	Se vincula a eventos o cambios UV.
A16	Mesh evolution	Strategy	Resume variaciones de objeto en <i>Object Mode</i> .

Tabla C.10: Diseño de presentación de las métricas A1–A16.

C.8. Diseño de métricas geométricas y UV

Las métricas geométricas y UV se calculan directamente sobre objetos de Blender. Este bloque se apoya en conceptos habituales de procesamiento de mallas poligonales y comparación geométrica de superficies [3, 5]. Su diseño

se separa de las métricas CSV porque dependen del estado de la malla, de capas UV y, en algunos casos, de un objeto de referencia.

Métrica	Diseño de cálculo
UV area	Área ocupada en el espacio UV mediante triangulación o fórmula poligonal sobre coordenadas UV.
UV islands	Conteo de componentes conexas en el mapa UV.
UV stretch	Relación entre proporciones 3D y proporciones UV para estimar deformación.
UV texel density	Relación entre área geométrica y área UV para estimar la distribución de resolución de textura sobre el modelo.
Normal percentage	Porcentaje de caras con orientación anómala o potencialmente invertida.
Angle	Mediana o resumen de ángulos entre caras adyacentes conectadas por aristas.
Non-quads	Recuento o porcentaje de caras que no son cuadrangulares.
Vertex duplicate	Detección de vértices coincidentes o muy próximos.
Similarity	Comparación geométrica y topológica frente a un objeto de referencia.
Hausdorff distance	Máxima distancia mínima entre conjuntos de vértices cuando existe referencia válida.

Tabla C.11: Diseño de métricas geométricas y UV.

C.9. Decisiones de diseño relevantes

Durante el diseño de la *suite* se tomaron varias decisiones orientadas a mejorar la modularidad, la mantenibilidad y el rendimiento del sistema. Estas decisiones afectan tanto a la separación entre captura y análisis como al formato de intercambio de datos, la forma de registrar cambios, el acceso a la información interna de Blender y la organización de la interfaz.

La Tabla C.12 resume las decisiones de diseño más relevantes y justifica su adopción dentro del proyecto. En conjunto, estas decisiones permiten que el sistema mantenga una captura ligera durante la sesión de modelado y desplace las operaciones más costosas a una fase posterior de análisis.

Decisión	Justificación
Separación en dos <i>add-ons</i>	Evita que el análisis pesado interfiera con el modelado y permite evolucionar cada parte por separado.
CSV como formato intermedio	Facilita trazabilidad, revisión manual, portabilidad y compatibilidad con herramientas externas.
Captura basada en snapshots	Reduce registros redundantes y permite calcular deltas de forma controlada.
Uso de bmesh	Permite leer topología y UV con mayor precisión en <i>Edit Mode</i> .
Hash UV	Detecta cambios de coordenadas UV aunque no cambien los contadores topológicos.
Uso de KDTree	Se incorpora en las métricas geométricas que requieren búsquedas de proximidad entre puntos, especialmente en comparaciones entre mallas, siguiendo la idea de búsqueda espacial eficiente propuesta para árboles multidimensionales [1]. Su uso permite reducir el coste de cálculo frente a comparaciones exhaustivas y mejora el rendimiento en modelos con mayor número de vértices.
Separación lógica/interfaz	Mejora mantenibilidad y permite probar cálculo y validación sin depender del panel gráfico.
Tooltips y bloques compactos	Conservan información contextual sin saturar la interfaz lateral de Blender.

Tabla C.12: Decisiones de diseño relevantes de la *suite*.

Apéndice *D*

Documentación técnica de programación

D.1. Introducción

Este anexo describe la organización técnica de los dos add-ons que forman la *suite*. La estructura se documenta desde una perspectiva de mantenimiento del software, apoyada en principios de modularidad, claridad de responsabilidades y calidad interna del código [16, 17]. A diferencia del anexo C, centrado en el diseño, aquí se explican las responsabilidades de los módulos, funciones, clases, dependencias, instalación, pruebas y pautas de mantenimiento.

La información se ordena de lo general a lo específico. Primero se presenta la estructura técnica del código, después el primer add-on de captura, a continuación el add-on de análisis y, finalmente, las tablas de funciones, dependencias, pruebas y criterios de mantenimiento.

D.2. Estructura de directorios

La estructura técnica del proyecto se organiza en torno a los dos add-ons desarrollados. El primer add-on se distribuye como un único archivo Python, mientras que el segundo se organiza como un paquete modular con ficheros separados para el cálculo, la interfaz, la gestión de dependencias, las visualizaciones y las utilidades auxiliares.

`codigo/`

```

|-- Data_Logger_3D.py
|-- Analysis_3D.zip
'-- Analysis_3D/
    |-- __init__.py
    |-- analytics.py
    |-- csv_schema.py
    |-- constants.py
    |-- dependencies.py
    |-- graphs.py
    |-- operator.py
    |-- ui.py
    |-- ui_graph_rendering.py
    |-- ui_graph_service.py
    |-- ui_helpers.py
    |-- ui_operators.py
    |-- ui_panels.py
    |-- ui_properties.py
    |-- utils.py
    '-- tests
        '-- test_logic.py
|-- core/
|   '-- data_logger_core.py
'-- tests/
    |-- test_core.py
    |-- test_csv_schema.py
    '-- analysis_3d/
        |-- _logger_test_utils.py
        |-- test_analysis_csv_schema.py
        |-- test_analysis_metrics.py
        |-- test_logger_csv_schema.py
        |-- test_logger_deduplication.py
        |-- test_logger_operator_detection.py
        |-- test_logger_privacy.py
        '-- test_logic.py

```

D.3. Preparación del entorno de desarrollo

Este apartado concreta qué debe instalar y configurar un programador que desee modificar el código de los add-ons. La preparación del entorno se basa en el uso del Python integrado en Blender y en sus APIs oficiales, por lo

que las pruebas deben realizarse dentro del mismo entorno de ejecución que utilizará el usuario final [2, 18]. No se trata de una compilación tradicional, ya que los complementos de Blender se ejecutan como scripts de Python interpretados por el propio Blender. Por tanto, el entorno de trabajo debe reproducir el intérprete de Python incluido en Blender y permitir recargar los add-ons durante las pruebas.

Herramientas necesarias

Para comenzar a desarrollar se recomienda preparar el siguiente entorno:

- **Blender 4.0 o superior:** entorno de ejecución principal y proveedor de las APIs `bpy`, `bmesh` y `mathutils`. La versión utilizada debe incluir Python 3.11 o una versión compatible con las dependencias del proyecto.
- **Git:** herramienta necesaria para clonar el repositorio, crear ramas de trabajo, revisar cambios y mantener trazabilidad mediante commits.
- **Editor de código:** se recomienda Visual Studio Code, PyCharm u otro editor con soporte para Python. La autocompletación de `bpy` puede requerir extensiones o stubs externos, pero no es imprescindible para ejecutar el proyecto.
- **Python integrado en Blender:** las pruebas deben realizarse con el Python incluido en Blender, no con una instalación externa independiente, para evitar incompatibilidades con `bpy`.
- **Dependencias científicas:** NumPy y Matplotlib para las funciones de análisis y visualización. Deben estar disponibles dentro del entorno Python de Blender.
- **Sistema $\text{\LaTeX}$:** solo es necesario si se desea modificar y recompilar la documentación del TFG.

Obtención del código fuente

El programador debe clonar o descargar el repositorio del proyecto y localizar los dos componentes principales: el archivo `Data_Logger_3D.py`, correspondiente al logger, y el paquete `Analysis_3D`, correspondiente al add-on de análisis. Se recomienda no trabajar directamente sobre el ZIP final, sino sobre la carpeta descomprimida del paquete de análisis, para poder modificar sus módulos de forma individual.

```
git clone [URL_DEL_REPOSITORIO]
cd [NOMBRE_DEL_REPOSITORIO]
```

Si el repositorio se entrega comprimido, basta con descomprimirlo y conservar la misma estructura de carpetas. La carpeta del add-on de análisis debe mantener su archivo `__init__.py`, ya que Blender lo utiliza como punto de entrada del paquete.

Dependencias externas

Los addons incluyen las librerías externas necesarias dentro de su propia estructura de carpetas, por lo que no es necesario instalar dependencias manualmente en el Python integrado de Blender en un ordenador nuevo.

Esta decisión evita que el usuario tenga que ejecutar comandos con `pip` o modificar la instalación de Blender. Al activar el addon, este carga las librerías incluidas localmente junto al código del addon.

La instalación manual de dependencias en el Python de Blender solo sería necesaria en un entorno de desarrollo, o en caso de utilizar una versión del addon que no incluya dichas librerías externas.

Carga del add-on durante el desarrollo

Durante el desarrollo existen dos formas de probar los cambios. La primera consiste en instalar el archivo o ZIP desde *Edit* → *Preferences* → *Add-ons*. Esta opción reproduce el comportamiento del usuario final. La segunda consiste en copiar o enlazar la carpeta del add-on dentro del directorio de scripts de Blender, lo que facilita modificar archivos y recargar el complemento.

Después de editar el código, se recomienda desactivar y volver a activar el add-on desde las preferencias de Blender. Si se modifican clases, propiedades o paneles, también puede ser necesario reiniciar Blender para limpiar registros anteriores. Las funciones `register` y `unregister` deben mantenerse actualizadas para evitar duplicados de clases, operadores o manejadores.

Flujo recomendado para modificar el código

El flujo de trabajo recomendado para un programador es el siguiente. Se recomienda mantener commits pequeños y descriptivos, y seguir criterios de estilo coherentes con Python para facilitar la revisión del código [9, 21]:

1. Crear una rama de trabajo en Git antes de modificar el código.
2. Abrir Blender y cargar una escena de prueba sencilla.
3. Modificar el módulo correspondiente: logger, métricas, visualización o interfaz.
4. Recargar el add-on y comprobar que no aparecen errores en la consola de Blender.
5. Probar la funcionalidad con un CSV controlado o una malla sencilla.
6. Revisar que las modificaciones no rompen la exportación CSV ni el cálculo de métricas existentes.
7. Registrar el cambio mediante un commit descriptivo.

Este flujo reduce el riesgo de introducir cambios incompatibles entre el logger y el add-on de análisis. En particular, cualquier modificación de la cabecera CSV debe acompañarse de cambios equivalentes en la validación y en la documentación de campos.

D.4. Primer add-on: *Data_Logger_3D.py*

Variables globales

El módulo emplea variables globales para mantener el estado mínimo necesario entre ciclos del temporizador. Entre ellas se encuentran el estado de ejecución del logger, el tiempo inicial de sesión, el último *timestamp* registrado, los contadores topológicos previos, el modo anterior, el objeto activo anterior, la firma de snapshot previa, el hash UV anterior y las banderas de operadores. Este planteamiento evita almacenar un historial completo de la escena y permite calcular deltas de forma eficiente.

Las variables de depuración, como el último operador detectado, la última razón de registro o el estado del hash UV, se mantienen separadas del registro CSV. Su finalidad es facilitar la verificación durante el desarrollo y no sustituir los datos exportados.

Gestión CSV

La gestión CSV se concentra en funciones de inicialización, escritura incremental, restauración e incrustación. El archivo temporal se crea con

una cabecera fija, se actualiza cuando se detectan cambios y se incrusta en el archivo `.blend` mediante un bloque de texto interno. Esta decisión reduce el riesgo de pérdida de datos cuando el usuario guarda el proyecto antes de exportar el CSV definitivo.

Las funciones principales de este bloque son `init_csv`, `append_csv`, `remove_duplicate_rows`, `import_csv_to_blend` y `restore_csv_from_blend`. La función `ensure_file_ends_with_newline` evita errores de concatenación al añadir filas sucesivas.

Captura espacial

La captura espacial obtiene la posición de la vista 3D, el radio de escena y la posición/radio del objeto activo. Para ello se consultan áreas `VIEW_3D`, matrices de vista y cajas envolventes transformadas a coordenadas de mundo. Este bloque proporciona los campos `UserX`, `UserY`, `UserZ`, `SceneRadius`, `ObjX`, `ObjY`, `ObjZ` y `ObjRadius`.

Captura topológica

La captura topológica calcula el número de vértices, triángulos, n-gons y normales potencialmente invertidas. En *Edit Mode* se utiliza `bmesh.from_edit_mesh` para acceder al estado editable de la malla; en *Object Mode* se consulta directamente `obj.data`. La función `get_mesh_stats` centraliza esta lógica y devuelve una tupla con los contadores relevantes.

Detección de operadores

La detección de operadores se basa en la normalización de identificadores de Blender y en banderas temporales. Se detectan operaciones de pegado, duplicación normal, duplicación enlazada y fusión o disolución. El operador `VIEW3D_OT_paste_logger` encapsula `Ctrl+V` para conservar el comportamiento de pegado y registrar el evento de forma explícita.

Detección UV

La detección UV combina dos mecanismos. En primer lugar, se identifican operadores UV conocidos, como despliegue, empaquetado o proyección. En segundo lugar, se calcula un hash global de coordenadas UV mediante `get_global_uv_hash`. Si el hash cambia respecto al estado anterior, el sistema marca un evento UV aunque no se haya producido una variación topológica.

Snapshots y deltas

El snapshot resume la escena en una estructura compacta: posición de usuario, radio de escena, objeto activo, contadores topológicos, modo, hash UV, número de objetos y número de modificadores. La función `build_snapshot` construye esta representación y genera una firma comparable. La función `apply_snapshot_as_baseline` actualiza la línea base cuando el estado se registra correctamente.

Temporizador

El temporizador se registra mediante `bpy.app.timers.register`. La función `logger_timer` ejecuta la captura cada intervalo configurado mientras el logger está activo. Si existe una bandera de registro forzado, se escribe una fila aunque el cambio no haya sido detectado por la comparación ordinaria.

Consentimiento

El consentimiento se implementa mediante un bloque de texto interno llamado `consent_flag`. Si el usuario acepta, el texto almacena el valor `ACCEPTED`. El add-on también comprueba la configuración Auto Run de Blender y muestra un aviso cuando la ejecución automática de scripts está desactivada.

Paneles

El add-on define un panel principal y un panel de depuración. El panel principal permite iniciar o detener el logger y exportar el CSV. El panel de depuración muestra estado del temporizador, último operador, motivo del último registro, estado UV y objeto activo. Este panel se mantiene cerrado por defecto para no saturar la interfaz.



Figura D.1: Panel principal del add-on Data Logger 3D con controles de captura y exportación.

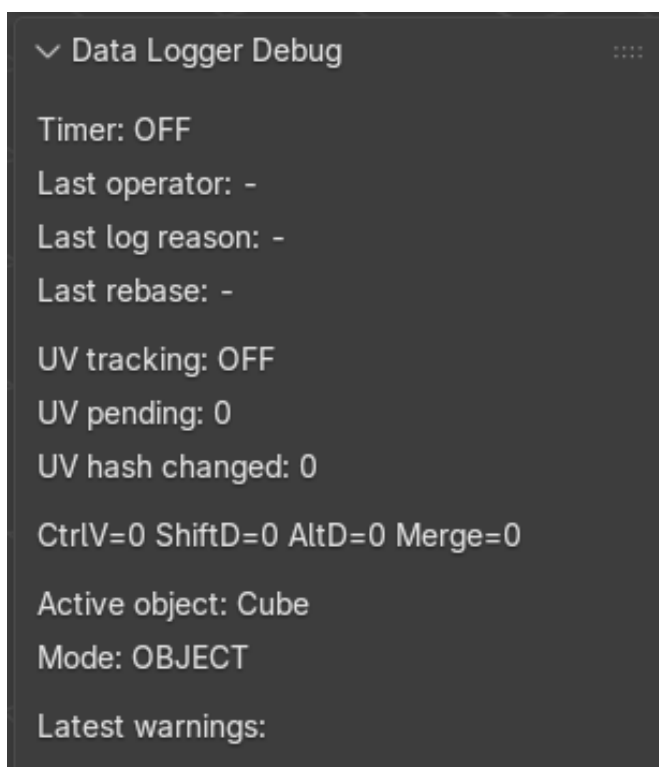


Figura D.2: Panel de depuración del add-on Data Logger 3D durante una sesión de captura.

Exportación

La exportación se realiza mediante `DATA_LOGGER_OT_ExportCSV`. El operador comprueba que el archivo `.blend` esté guardado, que exista el bloque `data_log_internal.csv` y que su contenido no esté vacío. El CSV exportado se guarda en la misma carpeta que el `.blend`, con un nombre derivado del proyecto.

Registro del add-on

Las clases del add-on se registran mediante `bpy.utils.register_class`. Además, se añaden keymaps, manejadores `load_post`, `save_post` y `depsgraph_update_post`. En `unregister` se eliminan keymaps y manejadores para evitar duplicados al recargar el script.

D.5. Segundo add-on: análisis y visualización

Lectura CSV

La lectura de CSV se concentra en la capa de operadores e interfaz. El usuario puede seleccionar archivos o carpetas, y el sistema transforma los registros en estructuras utilizadas por `analytics.py`. El módulo acepta nombres de columnas esperados y variantes razonables en algunos campos temporales o espaciales.

Validación

La validación descarta valores no numéricos cuando no pueden convertirse, ignora valores no finitos y aplica funciones seguras como `safe_float`, `series_to_array` y `clamp_outliers`. Este proceso permite que una fila incompleta no detenga necesariamente todo el análisis.

Métricas A1–A16

El módulo `analytics.py` define el diccionario `ANALYSIS` con las etiquetas y tipologías de A1–A16. La función `compute_metrics_for_csv` calcula las métricas a partir de tiempos, posiciones de vista, posiciones de objeto, deltas, estados de modo y eventos. Las métricas se devuelven en un diccionario indexado por identificador, lo que facilita su presentación posterior.

Métricas UV

Las métricas UV se implementan principalmente en `utils.py`. Incluyen conteo de islas UV, área UV, estiramiento y densidad de texeles. Estas funciones trabajan con capas UV activas y deben contemplar el caso de mallas sin coordenadas UV.

Métricas geométricas

Las métricas geométricas se calculan sobre objetos de tipo malla. Incluyen similitud topológica, distancia de Hausdorff, duplicados de vértices, proporción de caras no cuadrangulares y comparación frente a referencia. Cuando se requiere comparar conjuntos de coordenadas, el cálculo debe controlar el tamaño de la malla para evitar tiempos excesivos.

Métricas de normales

Las funciones `normal_flipped` y `calculate_normal_percentage` evalúan la orientación de caras y calculan el porcentaje de normales potencialmente invertidas. Estas métricas ayudan a detectar problemas de calidad geométrica relevantes para modelos 3D.

Métricas de ángulos

La función `calculate_angle_median` resume los ángulos entre caras adyacentes. La mediana se utiliza para reducir el impacto de valores extremos y ofrecer una medida robusta del comportamiento angular de la malla.

Visualización

El módulo `graphs.py` genera gráficos y objetos de visualización dentro de Blender. Incluye funciones para limpiar visualizaciones previas, crear ejes, etiquetas, puntos, superficies, gráficos temporales, radar y comparativas multi-CSV. También se contemplan gráficos renderizados en memoria para evitar archivos intermedios cuando resulta posible.

La Figura [D.3](#) muestra un ejemplo de gráfico de interacción generado por el *add-on* de análisis. En este caso se representan varias métricas frente al tiempo de sesión: velocidad de navegación, reutilización de atajos y variación de vértices. La visualización permite observar la aparición de picos de actividad durante la sesión, aunque debe interpretarse teniendo en cuenta que las métricas poseen escalas diferentes. Por este motivo, la gráfica se utiliza como demostración funcional de la integración visual, no como comparación estadística directa entre métricas.

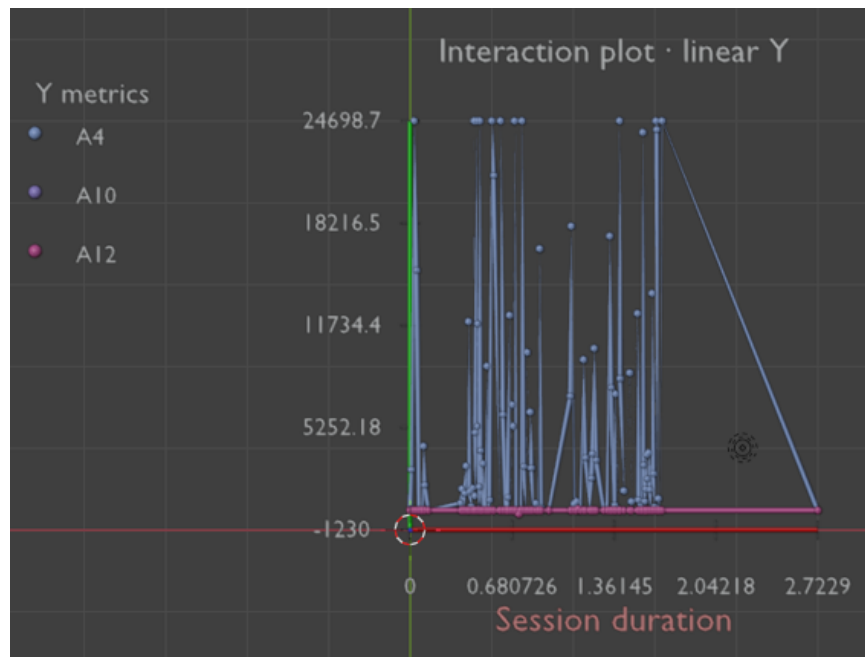


Figura D.3: Ejemplo de gráfico de interacción temporal. La diferencia de escala entre métricas hace que la velocidad de navegación domine visualmente la representación.

Interfaz

La interfaz se distribuye en módulos `ui_*`. Las propiedades se definen en `ui_properties.py`, los paneles en `ui_panels.py`, los operadores de interfaz en `ui_operators.py` y las ayudas de presentación en `ui_helpers.py`. Esta separación evita que el cálculo de métricas dependa directamente de los paneles.

Registro del add-on

El registro se coordina desde `__init__.py` y `ui.py`. Cada módulo registra sus clases de Blender y expone las funciones necesarias para recargar el add-on. La estructura modular facilita desactivar o modificar paneles sin alterar la lógica de cálculo.

D.6. Tabla completa de funciones

Las siguientes tablas agrupan las funciones y clases relevantes por módulo. Para mejorar la legibilidad, la información se divide en tres tablas independientes y se presenta en páginas apaisadas. De este modo se evita que las columnas queden comprimidas o que los nombres largos de funciones se salgan del margen.

Tabla D.1: Funciones y clases principales del add-on Data Logger 3D.

Función / Clase	Módulo	Responsabilidad	Entrada principal	Salida principal
<code>init_csv</code>	<code>Data_Logger_3D.py</code>	Crea el CSV temporal con la cabecera definida para la sesión de captura.	Ruta temporal global.	Archivo CSV inicializado.
<code>append_csv</code>	<code>Data_Logger_3D.py</code>	Añade una fila serializada al CSV, conservando el formato de salto de línea.	Fila CSV serializada.	CSV actualizado.
<code>ensure_file_ends_with_newline</code>	<code>Data_Logger_3D.py</code>	Comprueba que el archivo termina con salto de línea para evitar concatenaciones erróneas entre filas.	Ruta del archivo CSV.	Archivo normalizado.
<code>remove_duplicate_rows</code>	<code>Data_Logger_3D.py</code>	Elimina filas duplicadas manteniendo la cabecera original del CSV.	CSV temporal.	CSV sin duplicados.
<code>import_csv_to_blend</code>	<code>Data_Logger_3D.py</code>	Incrusta el contenido del CSV en un bloque de texto interno del archivo <code>.blend</code> .	CSV temporal.	<code>data_log_internal.csv</code> .
<code>restore_csv_from_blend</code>	<code>Data_Logger_3D.py</code>	Reconstruye el CSV temporal a partir del bloque de texto incrustado en Blender.	Bloque de texto interno.	CSV temporal reconstruido.
<code>get_camera_pos</code>	<code>Data_Logger_3D.py</code>	Obtiene la posición de la vista 3D activa mediante la matriz de vista.	Contexto de Blender.	Coordenadas <code>UserX</code> , <code>UserY</code> , <code>UserZ</code> .
<code>get_active_object_data</code>	<code>Data_Logger_3D.py</code>	Calcula centro y radio del objeto activo a partir de su caja envolvente en coordenadas de mundo.	Objeto activo.	Tupla espacial del objeto.
<code>get_scene_radius</code>	<code>Data_Logger_3D.py</code>	Estima el radio global de la escena considerando las cajas envolventes de las mallas.	Objetos de la escena.	Radio de escena.
<code>get_mesh_stats</code>	<code>Data_Logger_3D.py</code>	Calcula contadores topológicos de vértices, triángulos, n-gons y normales potencialmente invertidas.	Objeto malla.	Contadores topológicos.
<code>get_global_uv_hash</code>	<code>Data_Logger_3D.py</code>	Resume las coordenadas UV de la escena mediante un hash para detectar cambios.	Escena y capas UV.	Cadena hash.
<code>detect_flags_from_operator</code>	<code>Data_Logger_3D.py</code>	Activa banderas de operadores como <code>Ctrl+V</code> , <code>Shift+D</code> , <code>Alt+D</code> , <code>Merge</code> y operaciones UV.	Identificador de operador.	Booleano de cambio.
<code>process_new_operators</code>	<code>Data_Logger_3D.py</code>	Recorre los operadores recientes registrados por Blender y actualiza las banderas pendientes.	Historial de operadores.	Indicador de cambio.
<code>build_snapshot</code>	<code>Data_Logger_3D.py</code>	Construye un resumen del estado de la escena y una firma comparable entre ciclos.	Contexto de Blender.	Snapshot y firma.
<code>apply_snapshot_as_baseline</code>	<code>Data_Logger_3D.py</code>	Actualiza el estado de referencia usado para calcular deltas posteriores.	Snapshot y firma.	Línea base actualizada.

Continúa en la página siguiente

Tabla D.1: Funciones y clases principales del add-on Data Logger 3D (continuación).

Función / Clase	Módulo	Responsabilidad	Entrada principal	Salida principal
<code>collect_data</code>	<code>Data_Logger_3D.py</code>	Calcula deltas, estados de modo, banderas de operador y genera una fila CSV cuando hay cambios.	Estado actual de la escena.	Fila CSV o None .
<code>logger_timer</code>	<code>Data_Logger_3D.py</code>	Ejecuta ciclos periódicos de captura mediante el temporizador de Blender.	Temporizador de Blender.	Intervalo de repetición o None .
<code>DATA_LOGGER_OT_ExportCSV</code>	<code>Data_Logger_3D.py</code>	Exporta el CSV incrustado a un archivo externo junto al archivo <code>.blend</code> .	Contexto de Blender.	Archivo CSV exportado.
<code>DATA_LOGGER_PT_Panel</code>	<code>Data_Logger_3D.py</code>	Muestra el panel principal de control del logger en la barra lateral de Blender.	Contexto de interfaz.	Panel de usuario.
<code>DATA_LOGGER_PT_DebugPanel</code>	<code>Data_Logger_3D.py</code>	Muestra información de depuración sobre operadores, hash UV y estado del temporizador.	Contexto de interfaz.	Panel de depuración.
<code>register/unregister</code>	<code>Data_Logger_3D.py</code>	Registra o elimina clases, keymaps y handlers del add-on.	Runtime de Blender.	Add-on activo o inactivo.

Tabla D.2: Funciones principales de cálculo y validación del add-on de análisis.

Función / Clase	Módulo	Responsabilidad	Entrada principal	Salida principal
<code>safe_float</code>	<code>analytics.py</code>	Convierte campos CSV a número de forma segura y evita errores por valores no numéricos.	Fila y clave.	Valor flotante.
<code>series_to_array</code>	<code>analytics.py</code>	Limpia series numéricas y descarta valores no finitos antes del cálculo estadístico.	Secuencia de valores.	<code>ndarray</code> .
<code>clamp_outliers</code>	<code>analytics.py</code>	Limita valores extremos mediante el rango intercuartílico.	Serie numérica.	Serie acotada.
<code>compute_metrics_for_csv</code>	<code>analytics.py</code>	Calcula las métricas A1–A16 a partir de las filas validadas del CSV.	Filas CSV.	Diccionario de métricas.
<code>ensure_modules</code>	<code>dependencies.py</code>	Comprueba la disponibilidad de dependencias externas necesarias para el análisis.	Lista de módulos.	Estado de disponibilidad.
<code>count_uv_islands</code>	<code>utils.py</code>	Cuenta islas UV a partir de la conectividad de la malla y sus coordenadas UV.	<code>bmesh</code> .	Número de islas UV.
<code>calculate_uv_area</code>	<code>utils.py</code>	Calcula el área ocupada por las coordenadas UV de la malla.	Malla y capa UV.	Área UV.
<code>calculate_uv_stretch</code>	<code>utils.py</code>	Estima la relación de estiramiento entre geometría 3D y despliegue UV.	Objeto malla.	Valor de stretch.
<code>calculate_uv_texel_density</code>	<code>utils.py</code>	Calcula una medida de densidad de texeles asociada al despliegue UV.	Objeto malla.	Densidad.
<code>calculate_normal_percentage</code>	<code>utils.py</code>	Calcula el porcentaje de normales anómalas o invertidas respecto al total evaluado.	Objeto o <code>BMesh</code> .	Porcentaje.
<code>calculate_angle_median</code>	<code>utils.py</code>	Resume mediante la mediana los ángulos entre caras adyacentes.	<code>bmesh</code> .	Mediana angular.
<code>calculate_similarity</code>	<code>utils.py</code>	Compara un objeto analizado con una referencia para estimar su similitud geométrica.	Dos objetos.	Porcentaje o puntuación.
<code>calculate_hausdorff_distance_from_coords</code>	<code>utils.py</code>	Calcula la distancia de Hausdorff entre dos conjuntos de coordenadas.	Dos conjuntos de coordenadas.	Distancia.

Tabla D.3: Funciones y grupos de interfaz/visualización del add-on de análisis.

Función / Clase	Módulo	Responsabilidad	Entrada principal	Salida principal
<code>clear_previous_graphs</code>	<code>graphs.py</code>	Elimina visualizaciones anteriores para evitar solapamientos en la escena.	Escena.	Escena limpia.
<code>create_axis</code>	<code>graphs.py</code>	Crea ejes de referencia para representar métricas en el espacio 3D.	Coordenadas y escala.	Objeto eje.
<code>create_text_label</code>	<code>graphs.py</code>	Genera etiquetas de texto asociadas a gráficos y valores.	Texto y posición.	Objeto de texto.
<code>draw_metric_line</code>	<code>graphs.py</code>	Dibuja una línea asociada a la evolución de una métrica.	Serie métrica.	Curva u objeto gráfico.
<code>create_time_graph</code>	<code>graphs.py</code>	Genera un gráfico temporal a partir de una serie o un CSV.	CSV o serie.	Visualización temporal.
<code>draw_surface_graph</code>	<code>graphs.py</code>	Construye una superficie gráfica a partir de valores X, Y y Z.	Valores X/Y/Z.	Objeto superficie.
<code>draw_multi_csv</code>	<code>graphs.py</code>	Compara varios CSV mediante una representación tridimensional tipo forest plot.	Conjunto de CSV.	Forest plot 3D.
<code>_forest_plot_3d</code>				
<code>draw_radar_graph</code>	<code>graphs.py</code>	Genera un gráfico radar 3D con métricas normalizadas.	Métricas normalizadas.	Objeto radar.
<code>_3d_filled</code>	<code>ui_panels.py</code>	Define los paneles de resultados y la disposición visual de los bloques de métricas.	Contexto de Blender.	Panel de interfaz.
<code>ui_panels</code>				
<code>ui_operators</code>	<code>ui_operators.py</code>	Ejecuta acciones desde botones de la interfaz, como carga de CSV o cálculo de métricas.	Contexto y propiedades.	Resultado de operador.
<code>ui_properties</code>	<code>ui_properties.py</code>	Define propiedades configurables y rutas utilizadas por la interfaz.	Escena o <code>WindowManager</code> .	Propiedades registradas.
<code>register/unregister</code>	<code>__init__.py</code>	Registra o elimina las clases y módulos del add-on de análisis.	Runtime de Blender.	Add-on activo o inactivo.

D.7. Instalación, ejecución y recarga en desarrollo

El proyecto no requiere compilación en sentido estricto, ya que los add-ons se ejecutan como código Python interpretado dentro de Blender. La preparación para el usuario final consiste en instalar `Data_Logger_3D.py` y el ZIP del paquete de análisis desde *Edit* → *Preferences* → *Add-ons*. Sin embargo, para desarrollo se recomienda trabajar con la carpeta fuente del add-on de análisis y recargar el complemento tras cada modificación.

Para comprobar cambios en el logger se debe instalar o ejecutar `Data_Logger_3D.py`, abrir una escena de prueba, aceptar el consentimiento y revisar la consola de Blender. Para comprobar cambios en el add-on de análisis se debe cargar un CSV conocido y ejecutar primero las métricas estadísticas, después las métricas de malla y finalmente las visualizaciones. Si se modifican propiedades registradas, paneles o operadores, la recarga puede requerir desactivar el add-on, volver a activarlo o reiniciar Blender.

La ejecución completa comienza con el add-on de captura, continúa con la exportación del CSV y finaliza con la lectura del CSV desde el add-on de análisis. Las pruebas que dependan de paneles, operadores, mallas reales o contexto gráfico deben realizarse dentro de Blender, ya que no pueden reproducirse correctamente con un intérprete Python externo.

D.8. Dependencias externas

Las dependencias principales son Blender, Python integrado en Blender, `bpy`, `bmesh`, `mathutils`, `csv`, NumPy y Matplotlib [2, 18, 15, 11]. El logger depende principalmente de la API de Blender y de módulos estándar de Python. El add-on de análisis incorpora dependencias científicas para el cálculo de métricas y la representación gráfica de resultados.

La versión desarrollada se ha probado sobre un entorno basado en Python 3.11, integrado en versiones recientes de Blender. Por este motivo, no se garantiza la compatibilidad con Blender 3.0 ni con versiones anteriores que utilicen intérpretes de Python más antiguos o APIs parcialmente diferentes. La retrocompatibilidad con dichas versiones requeriría revisar las llamadas a la API de Blender, las dependencias científicas y los módulos utilizados por el add-on de análisis.

En consecuencia, se recomienda utilizar la *suite* en versiones de Blender compatibles con Python 3.11. Las versiones consideradas compatibles en el

entorno de desarrollo son Blender 4.0 y posteriores, hasta la versión 5.1.2. Para versiones futuras, será necesario verificar que no se hayan producido cambios incompatibles en la API de Blender o en la gestión de add-ons.

D.9. Licencia del software desarrollado

Los add-ons desarrollados se distribuyen bajo la licencia **GNU General Public License v3.0 o posterior (GPL-3.0-or-later)**. Esta elección es coherente con el ecosistema de Blender, ya que el código se integra como complemento ejecutado dentro de Blender y utiliza directamente sus APIs de Python, como `bpy`, `bmesh` y `mathutils` [2, 7].

La licencia GPL permite usar, estudiar, modificar y redistribuir el software, siempre que las obras derivadas mantengan una licencia compatible y se conserve el acceso al código fuente. Por tanto, el producto software resultante, incluyendo *Data Logger 3D* y el add-on de análisis y visualización, se libera como software libre bajo GPL-3.0-or-later.

Tabla D.4: Resumen del análisis de licencias de los componentes utilizados

Componente	Uso en el proyecto	Licencia	Implicación
Blender	Entorno de ejecución de los add-ons	GPL	Condiciona la compatibilidad de los add-ons
Python	Lenguaje de programación	PSF License	Compatible con GPL
<code>bpy</code>	API principal de Blender	Asociada a Blender / GPL	Compatible con GPL
<code>bmesh</code>	Acceso a mallas editables	Asociada a Blender / GPL	Compatible con GPL
<code>mathutils</code>	Operaciones matemáticas de Blender	Asociada a Blender / GPL	Compatible con GPL
<code>csv</code>	Lectura y escritura de CSV	Biblioteca estándar de Python	Compatible
NumPy	Cálculo numérico	BSD	Compatible con GPL
Matplotlib	Generación de gráficos	Licencia Matplotlib	Compatible con GPL
Código propio	Add-ons desarrollados	GPL-3.0-or-later	Licencia final del software

D.10. Pruebas del sistema

Las pruebas del sistema se organizaron en dos niveles complementarios. Por un lado, se realizaron pruebas funcionales dentro de Blender, orientadas a comprobar el comportamiento real de los add-ons en su entorno final de ejecución. Por otro lado, se incorporaron pruebas automatizadas con `unittest` para validar aquellas partes de la lógica interna que pueden ejecutarse fuera de Blender.

Revisión de calidad del código

Aunque durante el desarrollo no se ha utilizado una plataforma específica de revisión estática como SonarQube o Qlty, se ha realizado una revisión manual del código fuente antes de la entrega final. Esta revisión se ha centrado en comprobar la organización modular del proyecto, la separación entre lógica de captura, lógica de análisis e interfaz, la eliminación de versiones antiguas o código no utilizado, la legibilidad de nombres de funciones y variables, y la ausencia de dependencias innecesarias.

La revisión también ha tenido en cuenta aspectos de mantenibilidad, como la agrupación de responsabilidades por módulos, la reducción de duplicidades, la inclusión de comentarios en las partes más dependientes de la API de Blender y la validación básica de errores durante la carga de archivos CSV. Esta revisión manual no sustituye a un análisis estático automatizado, pero permite justificar que se ha realizado una comprobación razonada de la calidad del código entregado.

Como trabajo futuro, se propone incorporar herramientas como SonarQube, Qlty, Ruff, Pylint o Flake8 para obtener métricas automáticas de calidad, complejidad, duplicación y posibles problemas de mantenibilidad.

Tabla D.5: Resumen de pruebas funcionales, de calidad y compatibilidad del sistema

ID	Prueba	Tipo	Resultado esperado	Evidencia
P-01	Instalación del add-on Data Logger 3D	Manual	El add-on se instala y aparece activo en Blender	Vídeo
P-02	Instalación del add-on de análisis	Manual	El add-on se instala y muestra su panel en Blender	Captura / vídeo
P-03	Inicio de monitorización	Manual	El sistema solicita consentimiento e inicia la captura	Vídeo
P-04	Captura de operaciones de modelado	Manual	Se registran cambios de modo, transformaciones y eventos relevantes	CSV generado
P-05	Exportación del CSV	Manual	Se genera un archivo CSV legible y reutilizable	CSV de ejemplo
P-06	Carga del CSV en el add-on de análisis	Manual	El archivo se carga correctamente	Vídeo
P-07	Cálculo de métricas A1–A16	Manual	El sistema calcula y muestra las métricas estadísticas	Captura
P-08	Cálculo de métricas geométricas y UV	Manual	Se obtienen resultados sobre el objeto seleccionado	Captura
P-09	Generación de visualizaciones	Manual	Se generan gráficos o visualizaciones dentro de Blender	Vídeo
P-10	Revisión manual de calidad	Manual	El código queda organizado, sin versiones obsoletas ni dependencias innecesarias	Revisión del repositorio

Pruebas automatizadas con unittest

Además de las pruebas funcionales realizadas directamente en Blender, el proyecto incluye un conjunto de pruebas automatizadas orientadas a comprobar partes concretas de la lógica del sistema. Estas pruebas se han implementado con el módulo estándar `unittest` de Python, por lo que no requieren instalar dependencias adicionales para su ejecución básica.

El objetivo de estas pruebas no es sustituir a la validación funcional dentro de Blender, sino aportar una comprobación adicional sobre aquellas funciones que pueden evaluarse de forma aislada o parcialmente desacoplada del entorno gráfico. En particular, las pruebas se centran en aspectos como la estructura de los CSV, la migración entre versiones de esquema, la anonimización de datos, la deduplicación de registros, la detección de ciertos operadores y el cálculo de métricas analíticas a partir de datos de ejemplo.

La estructura de pruebas incluida en el proyecto queda centralizada fuera del paquete instalable de los add-ons, para separar el código de producción del código de validación automática:

```
tests/
|-- test_core.py
|-- test_csv_schema.py
'-- analysis_3d/
    |-- __init__.py
    |-- _logger_test_utils.py
    |-- test_analysis_csv_schema.py
    |-- test_analysis_metrics.py
    |-- test_logger_csv_schema.py
    |-- test_logger_deduplication.py
    |-- test_logger_operator_detection.py
    |-- test_logger_privacy.py
    '-- test_logic.py
```

Todas las pruebas ejecutables fuera de Blender se lanzan desde la raíz del proyecto con el siguiente comando:

```
PYTHONPATH=. python -m unittest discover -s tests -v
```

Este comando ejecuta tanto las pruebas generales como las pruebas específicas del módulo `Analysis_3D`. En un entorno Python externo a

Blender, algunas pruebas geométricas pueden omitirse automáticamente si no está disponible `mathutils`; el resto de pruebas debe finalizar correctamente.

En Windows PowerShell, el comando equivalente es:

```
$env:PYTHONPATH="."
python -m unittest discover -s tests -v
```

Debe evitarse interpretar como prueba principal del proyecto un descubrimiento global sin fijar la carpeta de pruebas, por ejemplo `python -m unittest discover -v`. El comando recomendado es el que indica explícitamente `-s tests`, ya que mantiene el descubrimiento dentro de la carpeta de pruebas y evita importar paquetes dependientes de Blender de forma accidental.

Los ficheros de prueba cubren distintos aspectos del sistema. Las pruebas de esquema CSV verifican que las cabeceras, versiones y transformaciones entre formatos sean correctas. Las pruebas de privacidad comprueban la generación de identificadores y la eliminación de campos identificativos en exportaciones anonimizadas. Las pruebas de deduplicación validan que no se mantengan registros repetidos innecesarios. Las pruebas de detección de operadores comprueban el reconocimiento de acciones relevantes, como duplicaciones o combinaciones de teclas. Finalmente, las pruebas de análisis verifican el cálculo de métricas a partir de datos CSV de ejemplo.

Un resultado correcto en estas pruebas aumenta la confianza en la lógica comprobada automáticamente, pero no garantiza por sí solo la corrección completa del sistema en una sesión real de Blender. La integración con el contexto gráfico, los paneles, los manejadores internos, los objetos de escena y la interacción del usuario requiere validación funcional dentro del entorno final de ejecución.

Limitaciones de las pruebas

Las pruebas automatizadas presentan una limitación importante: no validan por sí solas la integración completa con Blender. El sistema depende de elementos específicos del entorno anfitrión, como la API `bpy`, los tipos de datos de Blender, el contexto gráfico, los paneles de la interfaz, los operadores, los temporizadores y los manejadores internos. Estos elementos no pueden reproducirse de forma completa mediante una ejecución convencional de pruebas en un intérprete Python externo.

Por esta razón, las pruebas incluidas deben entenderse como una validación parcial de la lógica comprobable de forma automática. Permiten detectar errores en funciones concretas y facilitan el mantenimiento del sistema, pero no sustituyen a las pruebas funcionales realizadas dentro de Blender ni a la validación práctica descrita en la memoria.

Esta limitación se considera especialmente relevante desde el punto de vista de la reproducibilidad. Para mejorarla, una línea de trabajo futura consistiría en separar de forma más estricta el núcleo lógico del sistema respecto a la capa dependiente de Blender. El núcleo lógico debería incluir funciones de validación de CSV, migración de esquemas, anonimización, deduplicación y cálculo de métricas sin importar directamente `bpy`. Por otro lado, la capa de integración con Blender quedaría reservada para los paneles, operadores, temporizadores, manejadores y acceso a la escena.

Esta separación permitiría ejecutar una parte más amplia de las pruebas con `unittest` en un entorno Python estándar, mientras que las pruebas de integración seguirían realizándose dentro de Blender. De este modo, el proyecto ganaría en mantenibilidad, verificabilidad y claridad a la hora de distinguir entre errores de lógica interna y errores derivados del entorno de ejecución.

D.11. Mantenimiento y ampliación

El mantenimiento de la *suite* requiere conservar la coherencia entre el logger, el formato CSV y el add-on de análisis. El CSV actúa como formato intermedio entre ambos complementos, por lo que cualquier cambio en su estructura afecta directamente a la captura, la validación, el análisis y la interpretación posterior de los datos. Si se añaden nuevos campos al CSV, deben actualizarse la cabecera, la construcción de registros en el logger, la validación de lectura en el módulo de análisis, las pruebas asociadas y la documentación correspondiente.

Del mismo modo, cualquier nueva métrica debe incorporarse de forma coherente en la lógica de cálculo, reflejarse en la interfaz de usuario y documentarse en el manual de usuario para facilitar su interpretación. No basta con añadir el cálculo numérico: también debe explicarse qué representa la métrica, en qué condiciones es fiable, qué limitaciones tiene y cómo debe interpretarse dentro del proceso de modelado 3D.

Desde el punto de vista del mantenimiento, una mejora prioritaria sería desacoplar con mayor claridad el código que contiene lógica general del código

que depende directamente de Blender. Actualmente, determinadas partes del sistema requieren la presencia de módulos como `bpy`, `bmesh` o `mathutils`, lo que dificulta la ejecución de pruebas automatizadas en un entorno Python convencional. Separar el núcleo lógico en módulos independientes permitiría probar de forma más sencilla la validación de datos, el tratamiento de CSV, la anonimización, la deduplicación y el cálculo de métricas, dejando la interacción con Blender en una capa específica de integración.

Una posible reorganización futura consistiría en dividir el sistema en dos niveles. El primero estaría formado por módulos de lógica pura, independientes del entorno gráfico, encargados de procesar datos, validar esquemas, calcular métricas y transformar registros. Estos módulos podrían ejecutarse y probarse con `unittest` desde un intérprete Python estándar. El segundo nivel correspondería a la integración con Blender, incluyendo paneles, operadores, temporizadores, manejadores internos, acceso a objetos de la escena y generación de visualizaciones dentro del entorno 3D.

Esta separación permitiría distinguir con mayor claridad entre errores de lógica interna y errores derivados del contexto de ejecución de Blender. Además, facilitaría la ampliación del sistema, ya que nuevas funcionalidades podrían desarrollarse y probarse primero en el núcleo lógico antes de integrarse en la interfaz del complemento. También mejoraría la reproducibilidad de las pruebas automatizadas, al evitar que estas dependan innecesariamente de una sesión activa de Blender o de la disponibilidad del módulo `bpy`.

La ampliación del logger puede realizarse añadiendo nuevos operadores detectables al conjunto de identificadores normalizados y revisando si dichos eventos requieren un registro forzado. Sin embargo, cualquier ampliación de la captura debe realizarse con cautela, ya que un aumento excesivo del número de eventos registrados puede incrementar el tamaño del CSV, introducir ruido en los datos o afectar al rendimiento durante la sesión de modelado. Por ello, cada nuevo evento debería justificarse en función de su utilidad analítica y probarse en sesiones reales de Blender.

En el caso de nuevas visualizaciones, se recomienda mantener separada la lógica de cálculo respecto a la creación de objetos gráficos en Blender. Las funciones encargadas de calcular métricas, normalizar valores o preparar datos deberían ser independientes de las funciones que generan paneles, gráficos o elementos visuales en la escena. Esta separación reduce el acoplamiento, facilita la depuración y permite reutilizar los resultados en otros formatos o herramientas externas.

También sería conveniente reforzar la gestión del formato CSV mediante un esquema más explícito. Aunque el proyecto incorpora una versión de

esquema y mecanismos de compatibilidad, futuras ampliaciones deberían documentar con precisión el tipo de cada campo, sus valores permitidos, su significado y su relación con las métricas calculadas. Esto reduciría el riesgo de incompatibilidades entre versiones del logger y del add-on de análisis.

Otra línea de mantenimiento relevante es mejorar la gestión de errores y advertencias. El sistema debería informar de forma clara cuando un CSV no tiene el formato esperado, cuando faltan columnas necesarias, cuando existen valores no válidos o cuando una métrica no puede calcularse con fiabilidad. Estos mensajes son importantes tanto para el usuario final como para la depuración técnica del sistema.

Desde el punto de vista de la privacidad, cualquier ampliación debe revisar qué nuevos datos se registran, si pueden asociarse a una persona o sesión concreta y si deben incluirse en las exportaciones anonimizadas. La incorporación de nuevas métricas o campos de captura debe respetar el principio de minimización de datos, registrando únicamente la información necesaria para los objetivos del análisis.

En resumen, el mantenimiento futuro del proyecto debería centrarse en cuatro líneas principales: conservar la coherencia entre logger, CSV y análisis; separar la lógica independiente de Blender de la capa de integración; reforzar las pruebas automatizadas sobre módulos desacoplados; y documentar con precisión cualquier cambio en métricas, campos registrados o visualizaciones. Estas mejoras permitirían que la *suite* evolucionara de forma más robusta, verificable y mantenible.

Apéndice *E*

Manual de usuario

E.1. Introducción

Este manual describe el uso práctico de la *suite* formada por dos add-ons para Blender. La organización del contenido sigue un enfoque guiado por tareas donde cada acción se acompaña de explicación y comprobación visual [14]. El primer add-on, *Data Logger 3D*, permite registrar una sesión de modelado y exportar un CSV con los datos capturados. El segundo add-on permite leer esos CSV, calcular métricas estadísticas A1–A16, obtener métricas geométricas y UV, y generar visualizaciones dentro de Blender.

El contenido se ordena siguiendo el flujo real de trabajo: requisitos del sistema, instalación, configuración, uso del logger, exportación, uso del add-on de análisis, interpretación de métricas, diagnóstico de incidencias, privacidad y recomendaciones para organizar datos.

E.2. Requisitos del sistema

Para utilizar la *suite* se requiere una instalación funcional de Blender con soporte para add-ons de Python. El sistema debe permitir instalar scripts o paquetes ZIP desde el panel de preferencias. También se recomienda disponer de permisos de escritura en la carpeta del proyecto para poder exportar los CSV generados.

Elemento	Requisito
Blender	Versión compatible con la API utilizada por los add-ons. Mínimo 4.0.0.
Python	Python integrado en Blender. No se requiere ejecutar Python externo para el uso básico.
Add-on de captura	Archivo <code>Data_Logger_3D.py</code> .
Add-on de análisis	Archivo ZIP del paquete de análisis.
Permisos de archivo	Escritura en la carpeta donde se guarda el <code>.blend</code> para exportar CSV.
Dependencias científicas	NumPy y Matplotlib si se utilizan las funciones analíticas y gráficas que las requieren.

Tabla E.1: Requisitos básicos para utilizar la *suite*.

E.3. Instalación y configuración

La instalación de ambos add-ons se realiza desde el sistema estándar de Blender. Se recomienda instalar primero el logger y después el add-on de análisis.

1. Abrir Blender.
2. Acceder a *Edit* → *Preferences* → *Add-ons*.
3. Pulsar *Install*.
4. Seleccionar `Data_Logger_3D.py` para instalar *Data Logger 3D*.
5. Activar la casilla del add-on instalado.
6. Repetir el proceso con el ZIP del add-on de análisis.
7. Guardar las preferencias si se desea conservar la activación en futuras sesiones.

La Figura E.1 muestra el punto de entrada para instalar los complementos. El usuario debe acceder a la sección de add-ons desde las preferencias de Blender y comprobar que se encuentra en el buscador o listado de complementos antes de pulsar el botón de instalación.

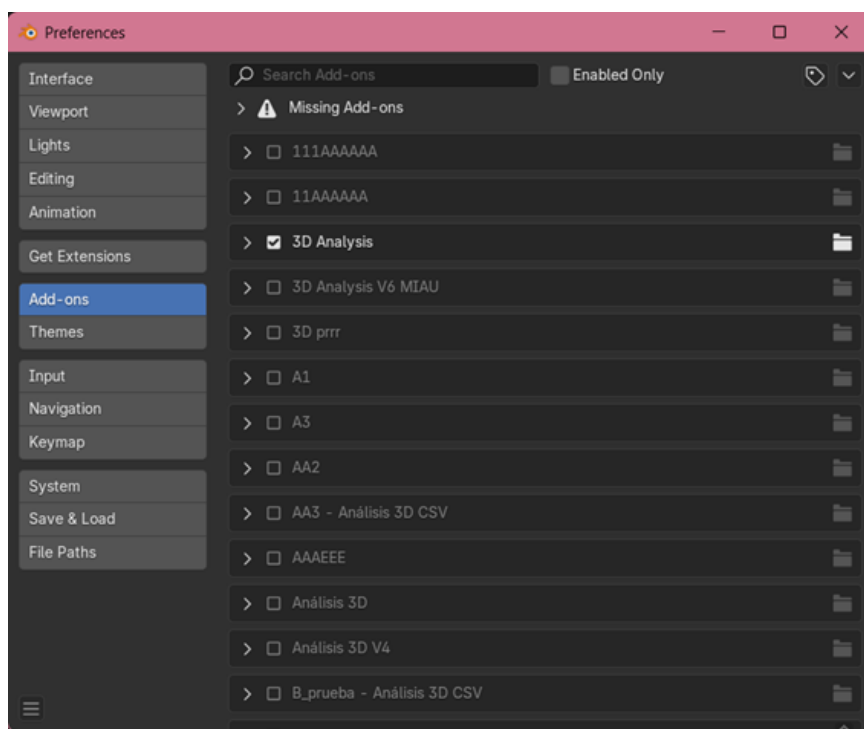


Figura E.1: Ventana de preferencias de Blender para la instalación de add-ons.

En la Figura E.2 se localiza el botón *Install*, que abre el explorador de archivos. Desde esa ventana se selecciona primero el archivo `Data_Logger_3D.py` y, posteriormente, el archivo ZIP del add-on de análisis. Tras seleccionar cada archivo, Blender añade el complemento al listado de add-ons disponibles.

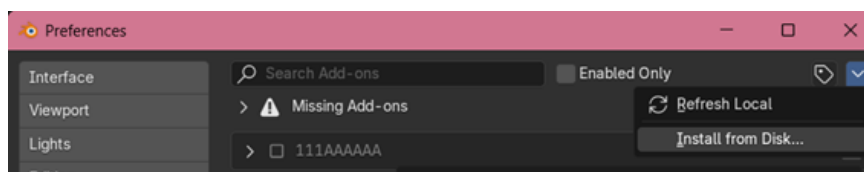


Figura E.2: Botón de instalación de add-ons desde las preferencias de Blender.

Una vez instalado el primer complemento, debe activarse la casilla correspondiente. La Figura E.3 permite comprobar que el add-on de captura se encuentra instalado y habilitado. Si la casilla no está marcada, el panel *Data Logger* no aparecerá en la barra lateral del visor 3D.

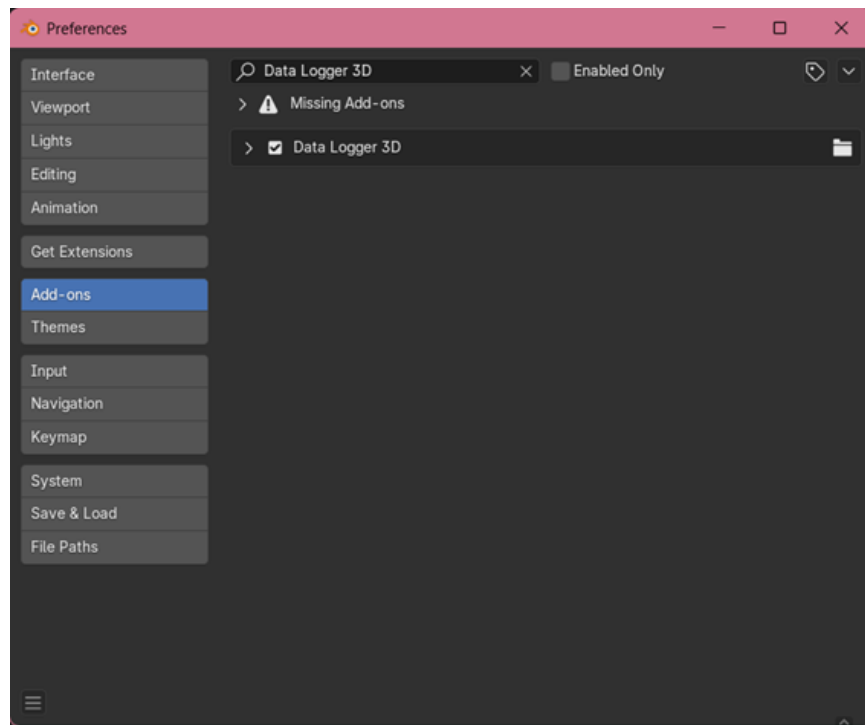


Figura E.3: Add-on Data Logger 3D instalado y activado en Blender.

El segundo complemento se activa del mismo modo. En la Figura E.4 se muestra el add-on de análisis y visualización ya disponible en Blender. Cuando ambos add-ons aparecen activados, la suite queda preparada para capturar datos, exportar CSV y realizar análisis posteriores.

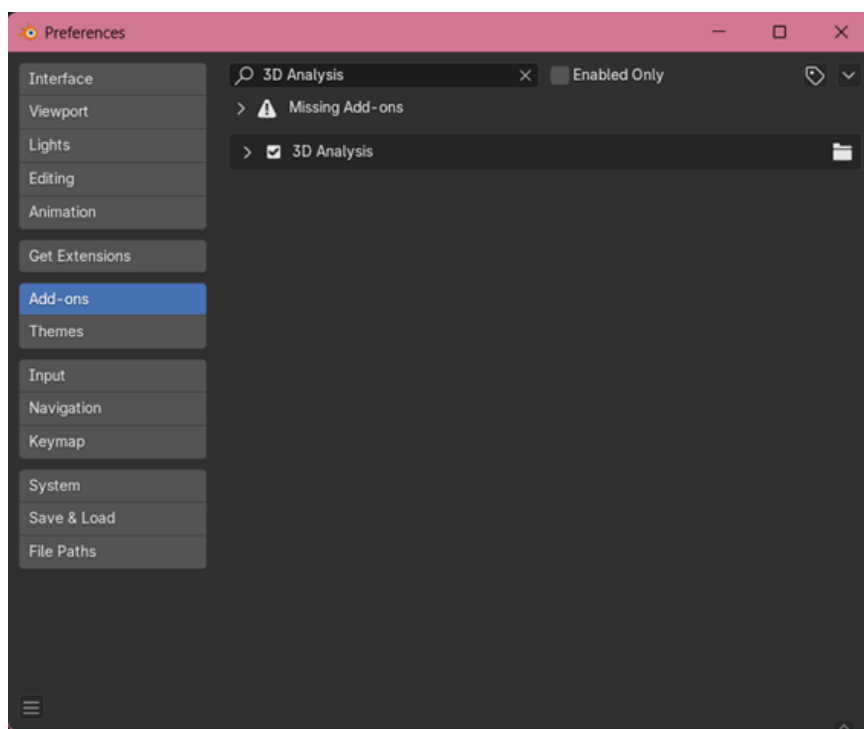


Figura E.4: Add-on de análisis y visualización instalado y activado en Blender.

Configuración de Auto Run

Si el archivo `.blend` necesita ejecutar scripts automáticamente al abrirse, Blender puede requerir que esté activada la opción Auto Run. Esta opción se encuentra en *Edit* → *Preferences* → *File Paths* → *Auto Run Python Scripts*. Si Auto Run está desactivado, el logger muestra un aviso y no debe iniciarse la captura hasta que el usuario revise esta configuración.

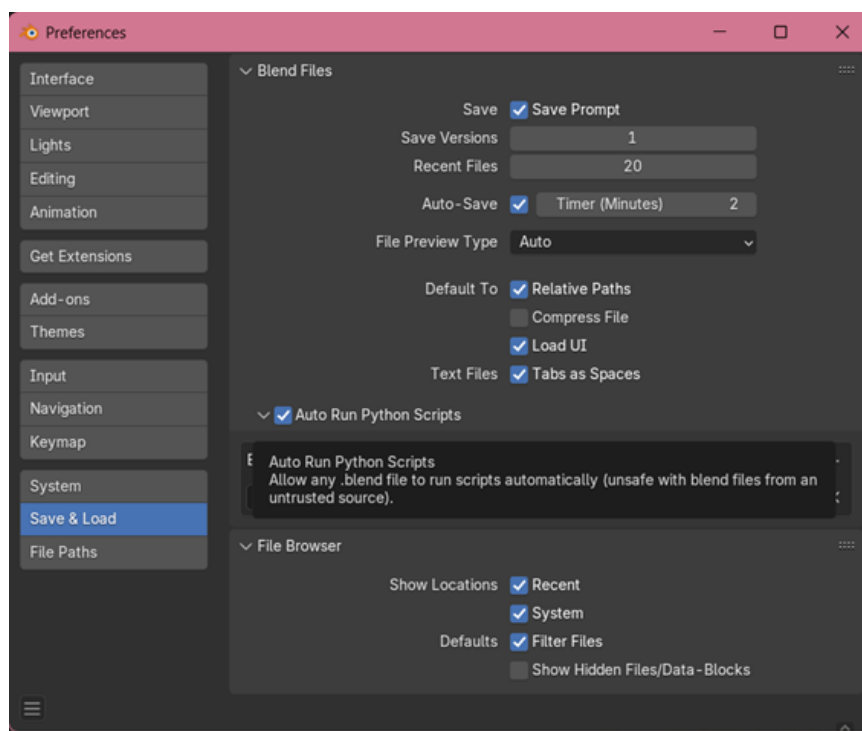


Figura E.5: Configuración de Auto Run Python Scripts en las preferencias de Blender.

La Figura E.5 sirve como comprobación previa al uso del logger. Si Blender bloquea la ejecución automática de scripts, el usuario debe activar esta opción o confirmar manualmente la ejecución cuando el programa lo solicite. Sin esta configuración, el consentimiento y el inicio automático de la captura pueden no mostrarse al abrir el archivo.

E.4. Uso del Data Logger 3D

Preparación de la sesión

El funcionamiento previsto del add-on *Data Logger 3D* se basa en la carga del archivo de Blender y en la aceptación explícita del consentimiento. Una vez instalado y activado el add-on, se recomienda guardar el archivo `.blend`, cerrar Blender y volver a abrir el proyecto. Al abrir de nuevo el archivo, el sistema muestra la ventana de consentimiento. Cuando el usuario acepta la recopilación de datos técnicos, el logger comienza la captura automáticamente.

Este flujo evita que el usuario tenga que iniciar manualmente la captura desde el panel en condiciones normales de uso. El panel del add-on permanece disponible para comprobar el estado del registro, detener la captura si fuera necesario y exportar posteriormente el archivo CSV generado.

1. Abrir o crear el proyecto de modelado en Blender.
2. Instalar y activar el add-on *Data Logger 3D*.
3. Guardar el archivo `.blend`.
4. Cerrar Blender.
5. Volver a abrir el archivo `.blend` guardado.
6. Revisar la configuración de Auto Run si Blender muestra el aviso correspondiente.
7. Aceptar la ventana de consentimiento de recopilación de datos técnicos.
8. Comprobar en la pestaña *Data Logger* que la captura se encuentra activa.

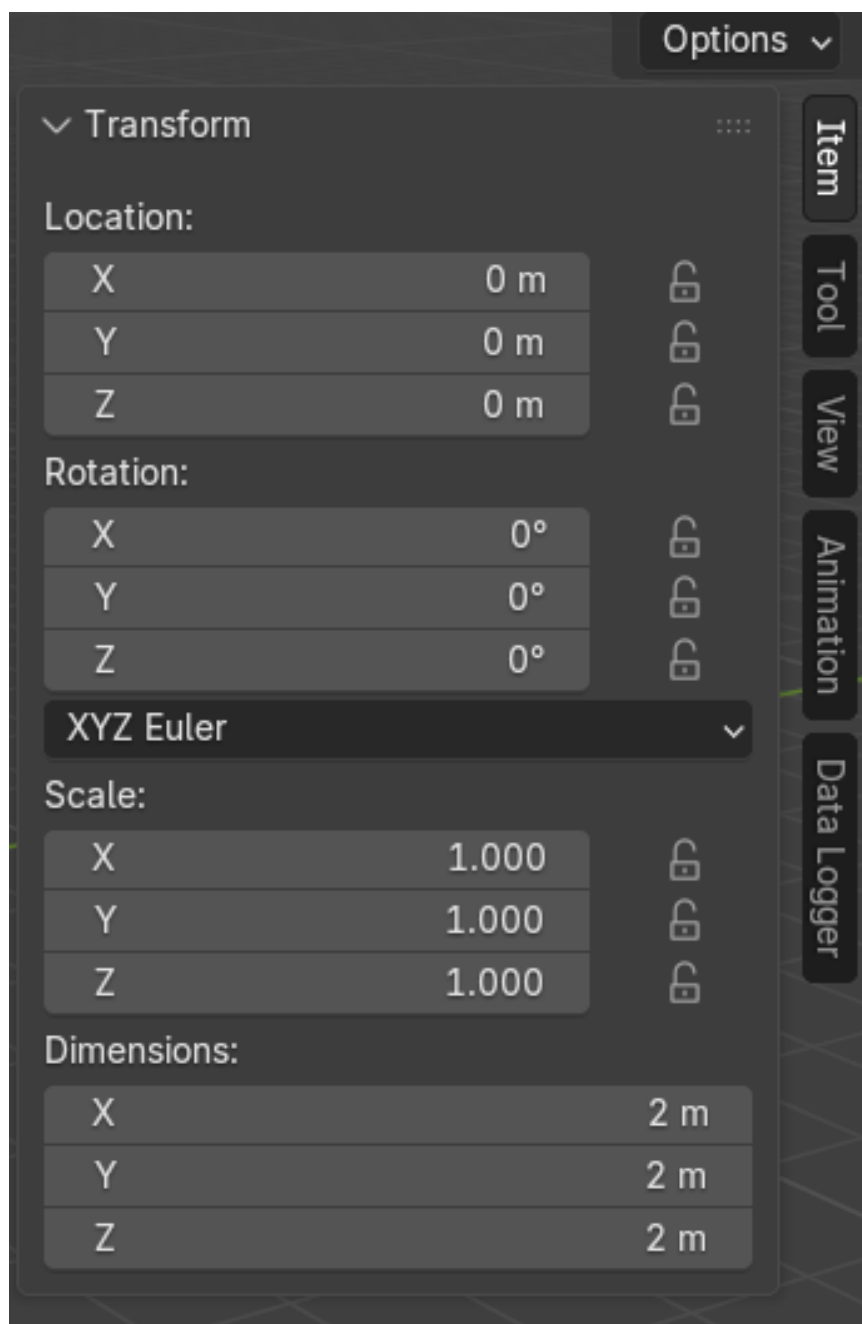


Figura E.6: Ubicación de la pestaña Data Logger en la barra lateral del visor 3D.

La Figura E.6 muestra dónde se encuentra el panel dentro del visor 3D. Para abrir la barra lateral se puede utilizar la tecla N; después debe

seleccionarse la pestaña *Data Logger*. Desde este panel se verifica si la captura está activa y se accede a las opciones de parada, depuración y exportación.

Consentimiento

El sistema solicita consentimiento antes de registrar datos técnicos de la sesión. Esta ventana aparece al volver a abrir el archivo `.blend` después de haber instalado y activado el add-on. La captura no comienza hasta que el usuario acepta expresamente dicha recopilación.

Una vez aceptado el consentimiento, el add-on inicia automáticamente el registro de la sesión. El consentimiento se almacena en el propio archivo mediante un bloque interno, por lo que se recomienda guardar de nuevo el `.blend` después de aceptarlo. De esta forma, la aceptación puede conservarse junto al proyecto para sesiones posteriores.

Los datos capturados son técnicos: transformaciones, posición de vista, deltas topológicos, cambios UV, modificadores, modos y operadores relevantes. No se registran pulsaciones generales de teclado, contenido textual externo, conversaciones, credenciales ni datos personales directos.

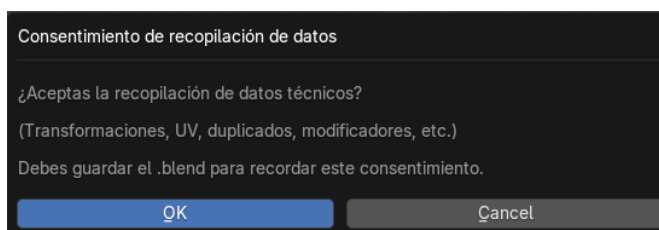


Figura E.7: Ventana de consentimiento mostrada antes de iniciar la captura de datos técnicos.

En la Figura E.7 se observa el cuadro de consentimiento. El usuario debe leer el aviso y aceptar únicamente si está de acuerdo con la recopilación de datos técnicos. Al aceptar, el add-on inicia la captura; si se cancela o no se acepta, no debe registrarse la sesión.

Inicio automático y control manual de la captura

El inicio normal de la captura se produce automáticamente tras aceptar la ventana de consentimiento. En ese momento, el logger queda activo y comienza a registrar en segundo plano los cambios significativos de la

sesión de modelado. Por tanto, en el flujo previsto no es necesario pulsar manualmente *Start Logger* para comenzar la grabación.

El panel principal del logger muestra el estado de la captura y permite controlar manualmente el proceso cuando sea necesario. Si el logger está detenido, el botón *Start Logger* permite iniciar el registro de forma manual. Si el logger está activo, el botón *Stop Logger* permite detenerlo. Al detenerse, el CSV se incrusta en el archivo `.blend`, lo que permite conservar los datos junto al proyecto antes de exportarlos.

Este control manual resulta útil para comprobar el funcionamiento del add-on, reiniciar una captura o detener el registro antes de finalizar la sesión.

La Figura E.8 representa el estado inicial del panel cuando el logger no está capturando. En este caso, el usuario puede iniciar manualmente la captura si no se ha iniciado de forma automática, o utilizar el panel para comprobar que el add-on está correctamente cargado.

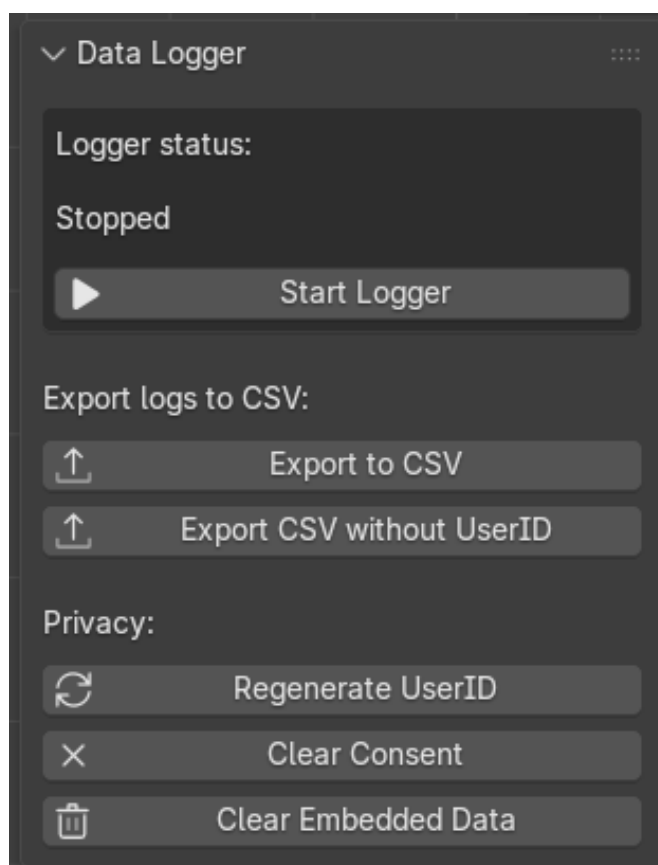


Figura E.8: Panel principal del Data Logger 3D al parar la captura.

Cuando el logger está activo, el panel cambia de estado, como se muestra en la Figura E.9. En este momento el usuario puede continuar modelando con normalidad. El registro se realiza en segundo plano y solo es necesario volver al panel si se desea detener la captura o exportar los datos.



Figura E.9: Panel principal del Data Logger 3D durante una sesión de captura activa.

Operaciones detectadas

Durante la captura se registran cambios espaciales, topológicos y contextuales. También se detectan operaciones concretas relevantes para el análisis del proceso de modelado: **Ctrl+V**, **Shift+D**, **Alt+D**, operaciones de fusión o disolución, cambios UV y cambios de modo. Estas operaciones aparecen en el CSV como banderas o deltas.

Panel de depuración

El panel de depuración está pensado para revisión técnica. Muestra información como el último operador detectado, el motivo del último registro, el estado del hash UV y el objeto activo. No es necesario utilizarlo durante una sesión ordinaria, pero resulta útil para comprobar que el add-on está capturando eventos.

La Figura E.10 muestra el panel de depuración. Este bloque no es necesario para un uso ordinario, pero ayuda a comprobar que el sistema detecta operadores, cambios UV y motivos de registro. Su uso está orientado a pruebas o revisión técnica.

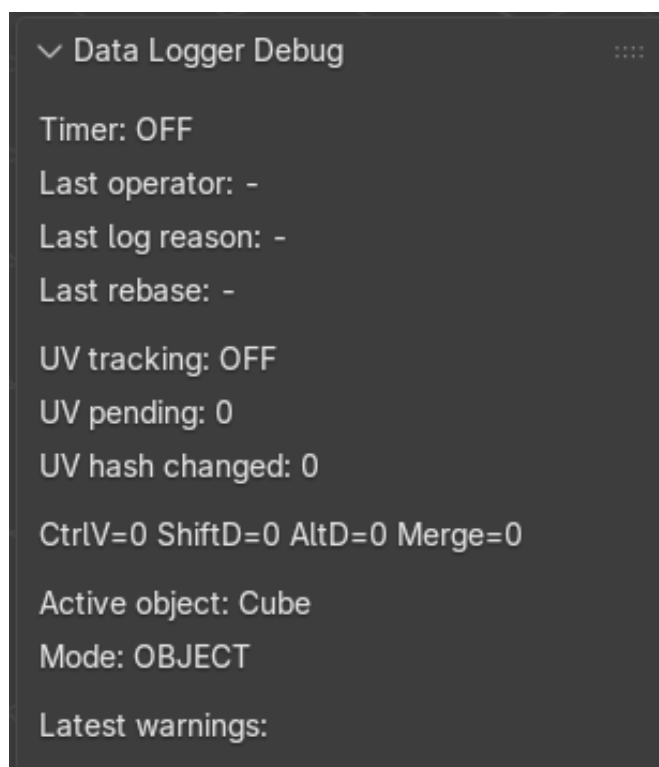


Figura E.10: Panel de depuración del Data Logger 3D con información del último evento registrado.

Exportación CSV

Para exportar los datos se debe pulsar *Export CSV* desde el panel del logger. El archivo `.blend` debe estar guardado previamente. El CSV se

genera en la misma carpeta que el proyecto y utiliza un nombre derivado del archivo `.blend`.

La Figura E.11 muestra la acción de exportación. Antes de pulsarla, el usuario debe asegurarse de que la captura se ha detenido y de que el archivo `.blend` ya está guardado. Si no existe un proyecto guardado, el add-on no puede determinar con seguridad dónde colocar el CSV.

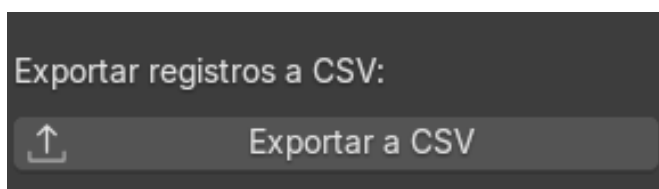


Figura E.11: Botón de exportación del CSV generado por el Data Logger 3D.

Además de la exportación mediante el botón del panel, los datos también pueden consultarse desde el propio archivo `.blend`. El logger guarda una copia interna del CSV en un bloque de texto de Blender, accesible desde el editor de texto mediante el *Text Editor*. Esta opción resulta útil si se desea comprobar rápidamente si la sesión ha generado datos o recuperar el contenido del registro sin realizar todavía una exportación externa.

La Figura E.12 muestra el acceso al editor de texto de Blender, desde el cual puede seleccionarse el bloque interno que contiene los datos registrados.

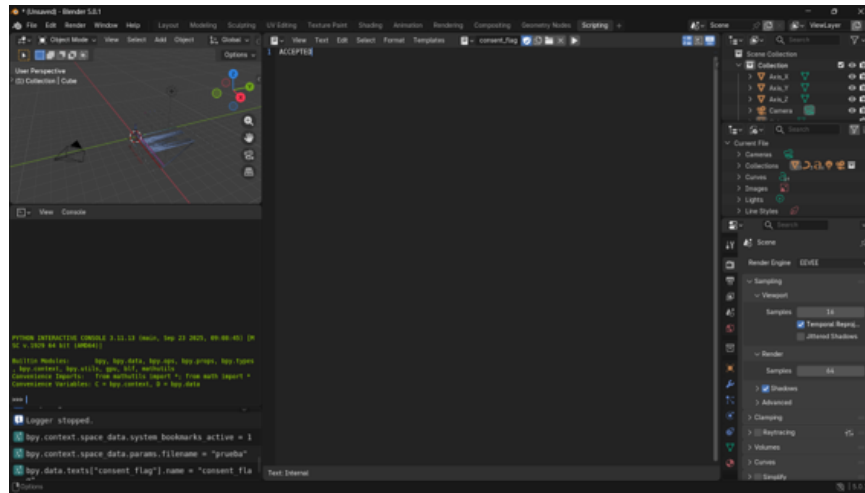


Figura E.12: Acceso al editor de texto de Blender para consultar los bloques internos del archivo.

Una vez seleccionado el bloque de texto correspondiente al registro, es posible visualizar el contenido CSV almacenado internamente. La Figura E.13 muestra un ejemplo de datos registrados dentro del *Text Block*. Aunque esta alternativa permite inspeccionar o copiar el contenido, para el flujo habitual de análisis se recomienda utilizar el archivo CSV exportado, ya que mantiene una ubicación externa clara y facilita su carga posterior en el add-on de análisis.

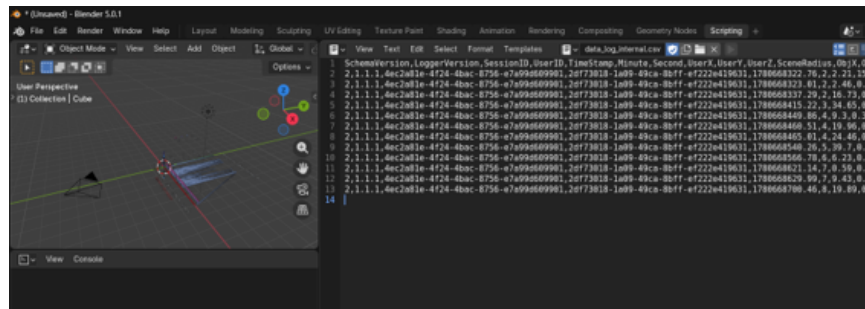


Figura E.13: Contenido CSV almacenado como bloque de texto interno dentro del archivo de Blender.

Después de exportar, debe comprobarse en el explorador de archivos que el CSV se ha creado junto al proyecto, como se observa en la Figura E.14. Este archivo será la entrada del add-on de análisis, por lo que conviene

conservarlo con un nombre reconocible y no modificar manualmente su cabecera.

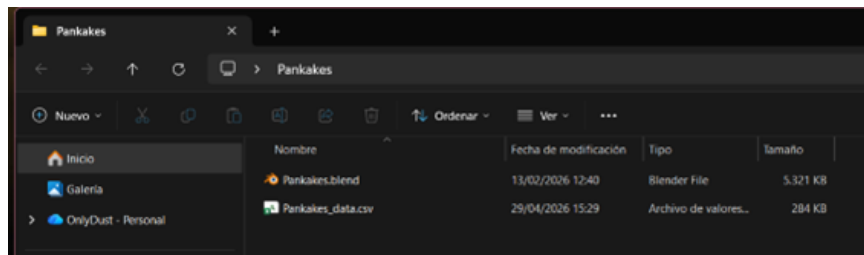


Figura E.14: Archivo CSV exportado en la misma carpeta que el proyecto de Blender.

E.5. Uso del add-on de análisis

El add-on de análisis se organiza en pestañas para que el usuario no tenga que trabajar con todas las opciones al mismo tiempo. La primera pestaña se dedica al análisis de CSV, la segunda a las métricas de malla y la tercera a la generación de gráficos y visualizaciones. Esta organización separa claramente el análisis de sesiones, el análisis del objeto seleccionado y la representación visual de resultados.

La Figura E.15 muestra la pestaña inicial de análisis CSV. Desde esta vista se seleccionan archivos o carpetas, se lanzan los cálculos estadísticos y se revisan los resultados asociados a las métricas A1–A16.

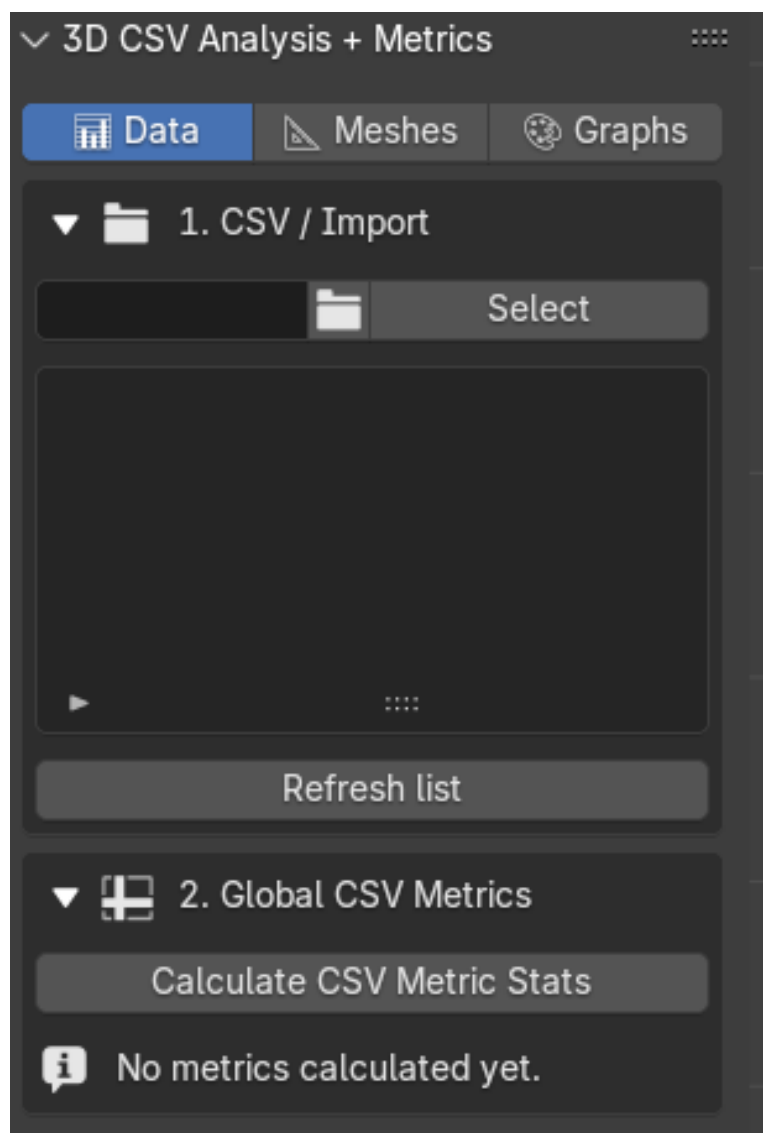


Figura E.15: Pestaña de análisis CSV del add-on de análisis y visualización.

La Figura E.16 corresponde a la pestaña de métricas de malla. En esta sección el usuario debe seleccionar el objeto activo y, cuando proceda, un objeto de referencia. Las métricas calculadas dependen del estado real de la geometría en Blender.

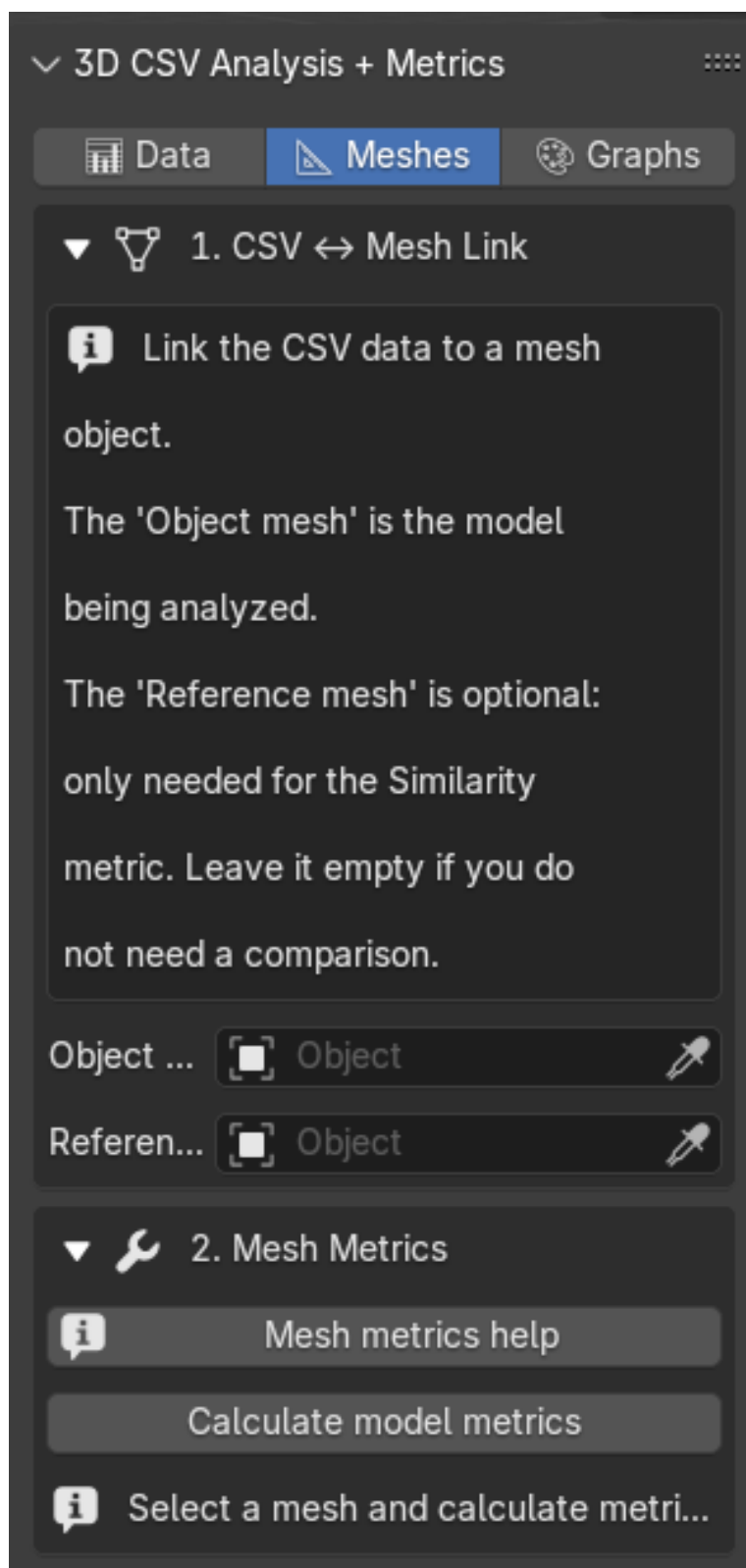


Figura E.16: Pestaña de métricas de malla del add-on de análisis y visualización.

La Figura [E.17](#) muestra la pestaña de gráficos. Esta parte permite generar representaciones visuales a partir de los resultados, por ejemplo gráficos temporales, comparaciones entre varios CSV, forest plots o gráficos radar.

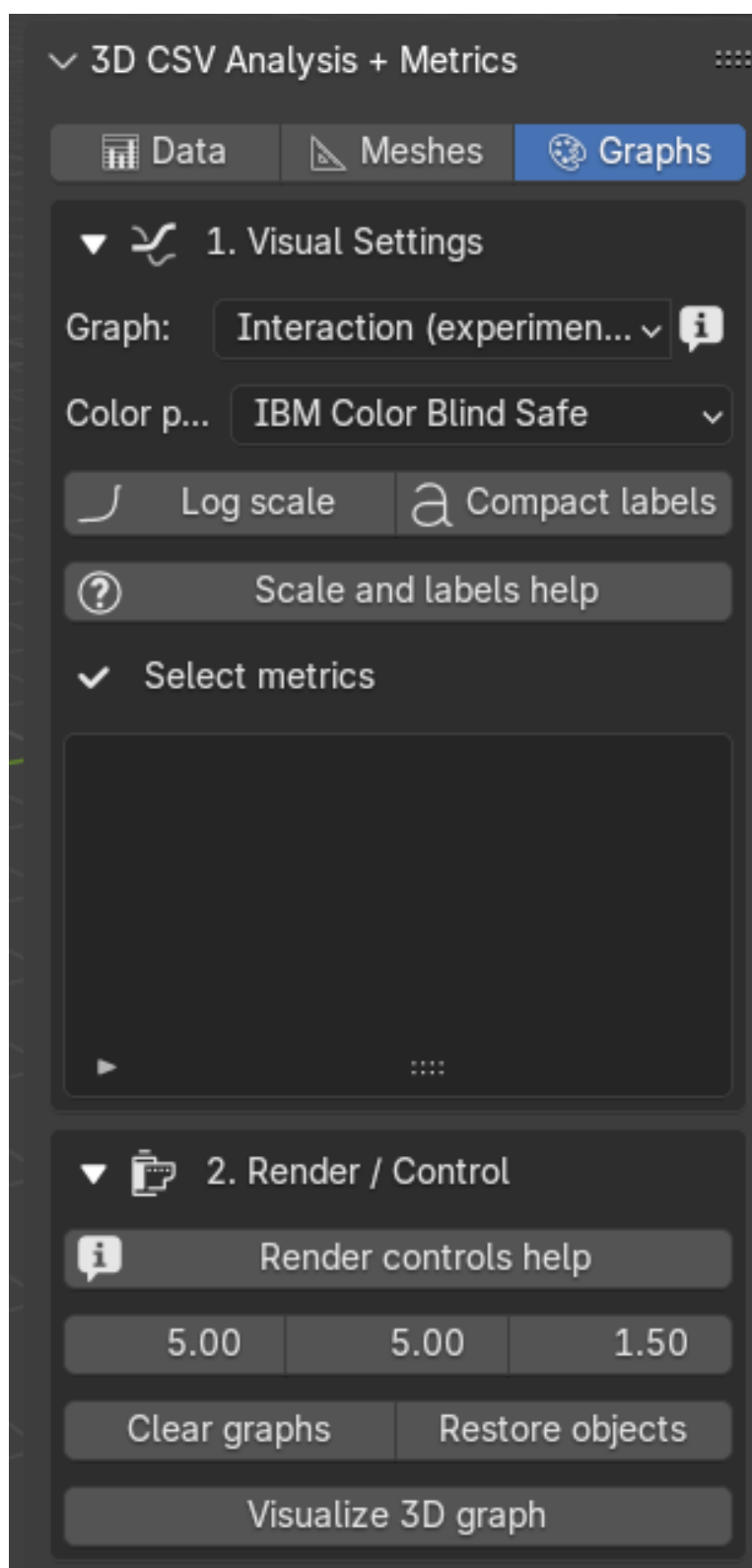


Figura E.17: Pestaña de gráficos y visualizaciones del add-on de análisis y visualización.

Selección de CSV o carpeta

El add-on de análisis permite seleccionar un CSV individual o una carpeta con varios CSV. La selección de carpeta resulta útil cuando se desea comparar sesiones o usuarios. Los archivos deben conservar la estructura de cabecera generada por el logger.

1. Abrir el panel del add-on de análisis en Blender.
2. Seleccionar el CSV exportado o la carpeta que contiene varios CSV.
3. Comprobar que la ruta aparece correctamente en el panel.
4. Ejecutar la acción de cálculo correspondiente.

En la Figura E.18 se muestra la selección de un archivo CSV individual. Este modo es adecuado para revisar una sesión concreta y comprobar sus métricas antes de realizar comparaciones con otros registros.

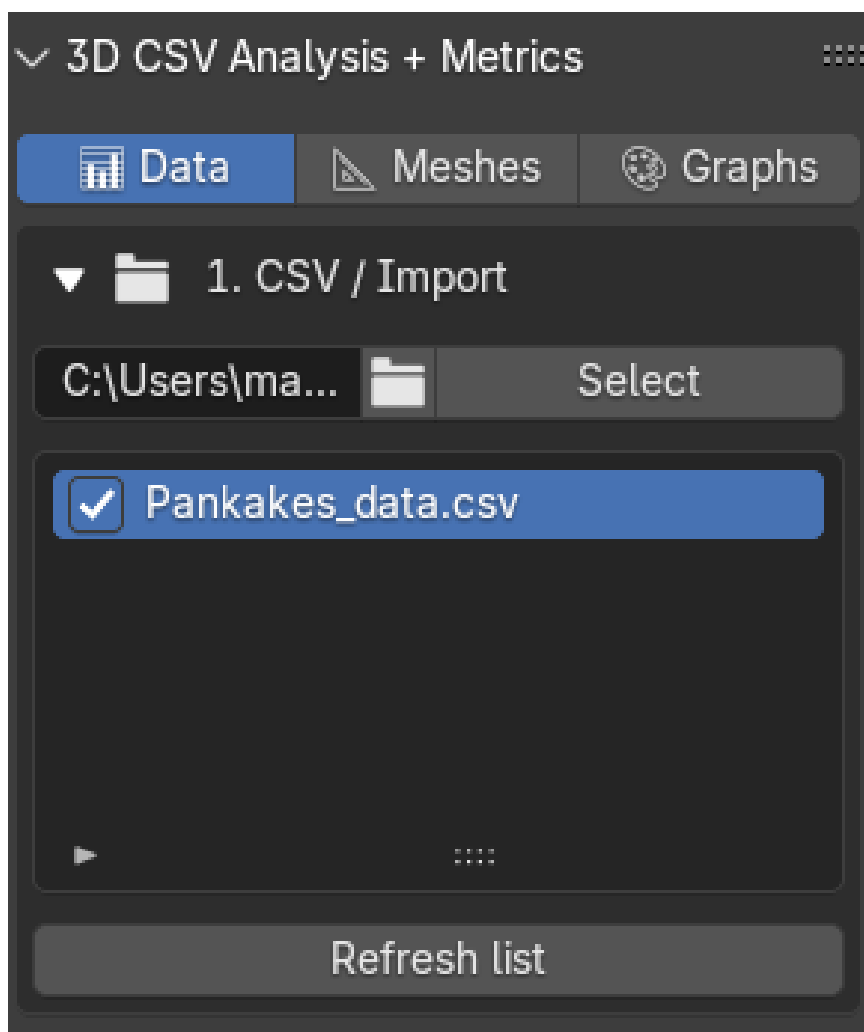


Figura E.18: Selección de un archivo CSV generado por el Data Logger 3D.

La Figura E.19 muestra la selección de una carpeta. Este modo debe utilizarse cuando se desea comparar varias sesiones o varios usuarios. Todos los CSV incluidos deben proceder del mismo formato de logger para que las columnas sean compatibles.

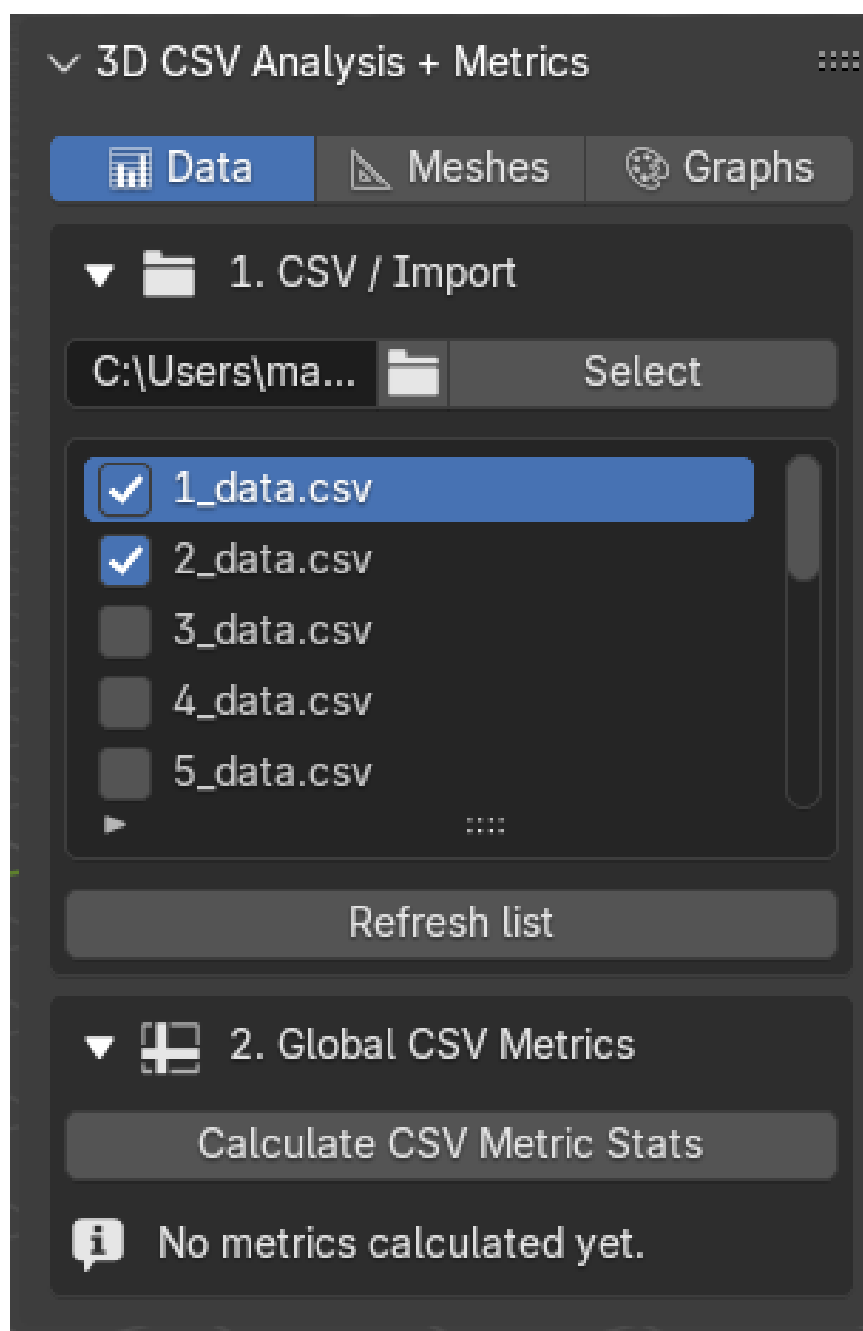


Figura E.19: Selección de una carpeta con varios CSV para análisis comparativo.

Cálculo de métricas estadísticas

El cálculo de métricas estadísticas procesa los registros CSV y genera indicadores A1–A16. Estas métricas resumen duración, pausas, velocidad, proximidad, manipulación de vista, actividad, evolución de vértices, errores, trabajo UV y cambios de objeto. La interfaz muestra resultados agrupados para evitar repetición.

Después de seleccionar la ruta, el usuario debe ejecutar el cálculo de métricas. La Figura E.20 muestra el botón correspondiente. Al pulsarlo, el add-on lee el CSV, valida los valores y calcula los bloques de métricas que se muestran en el panel.

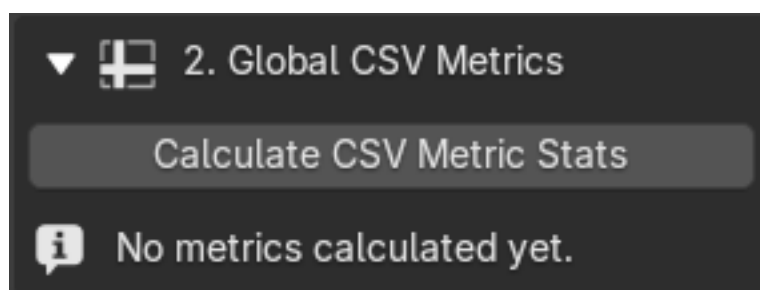


Figura E.20: Acción de cálculo de métricas estadísticas a partir del CSV seleccionado.

Bloque visible	Interpretación práctica
A1	Duración total de la sesión.
A2–A3	Pausas y distribución temporal de pausas.
A4–A5	Velocidad de movimiento y proximidad al objeto.
A6–A9	Manipulación de vista, velocidad en actividad/inactividad y distancia recorrida.
A10–A11	Estrategias de aceleración o cambios de ritmo.
A12–A13	Evolución de vértices y errores topológicos.
A14–A16	Trabajo en <i>Object Mode</i> , UV, evolución de malla y cambios de objeto.

Tabla E.2: Lectura práctica de los bloques A1–A16.

La Tabla E.2 resume cómo debe leerse cada bloque de métricas. Esta tabla no sustituye a los resultados generados por el add-on, sino que actúa como guía para interpretar los valores mostrados en pantalla.

La Figura E.21 muestra el flujo de cálculo de métricas estadísticas: primero se ejecuta la acción de cálculo y, posteriormente, el panel presenta los resultados agrupados en bloques interpretables.

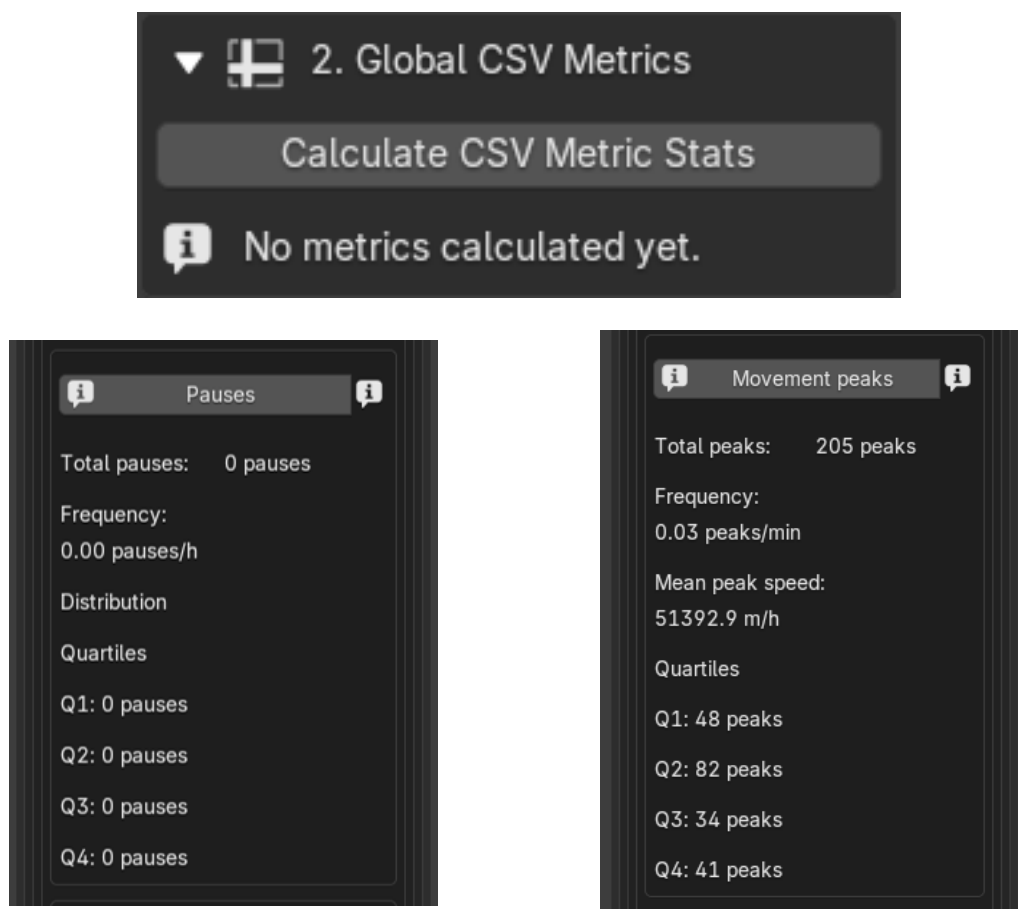


Figura E.21: Proceso de cálculo y visualización de métricas estadísticas del CSV en el add-on de análisis.

Cálculo de métricas geométricas y UV

Para calcular métricas geométricas o UV se debe seleccionar el objeto de Blender correspondiente. Algunas métricas pueden requerir además un objeto de referencia, por ejemplo en comparaciones de similitud o en el cálculo de la distancia de Hausdorff. Si la malla no tiene UV, el sistema no podrá calcular métricas de área, islas, estiramiento o densidad de texeles.

En el caso de la métrica de similitud, es importante que el objeto analizado y el objeto de referencia se encuentren lo más alineados posible antes de realizar el cálculo. Para obtener una comparación más precisa, ambos modelos deberían presentar una orientación, escala y posición similares. Este ajuste debe realizarse manualmente en Blender, ya que la alineación automática entre modelos puede resultar compleja y no siempre fiable, especialmente

cuando existen diferencias de escala, rotación, topología o nivel de detalle entre los objetos comparados.

Por tanto, antes de ejecutar la comparación de similitud, se recomienda revisar visualmente ambos objetos y, si es necesario, ajustar manualmente su rotación, tamaño y colocación en la escena. De esta forma, la métrica refleja con mayor fidelidad las diferencias geométricas reales entre los modelos, en lugar de verse afectada por desplazamientos, escalados o rotaciones previas.

Las métricas geométricas ayudan a valorar la calidad de la malla: caras no cuadrangulares, duplicados de vértices, normales potencialmente invertidas, ángulos, similitud y distancia respecto a una referencia. Las métricas UV ayudan a revisar el despliegue y la consistencia de coordenadas UV.

La Figura E.22 muestra la selección del objeto analizado y del objeto de referencia. Antes de ejecutar las métricas conviene revisar visualmente que ambos modelos están correctamente seleccionados y que, para similitud o Hausdorff, se encuentran alineados de forma manual.

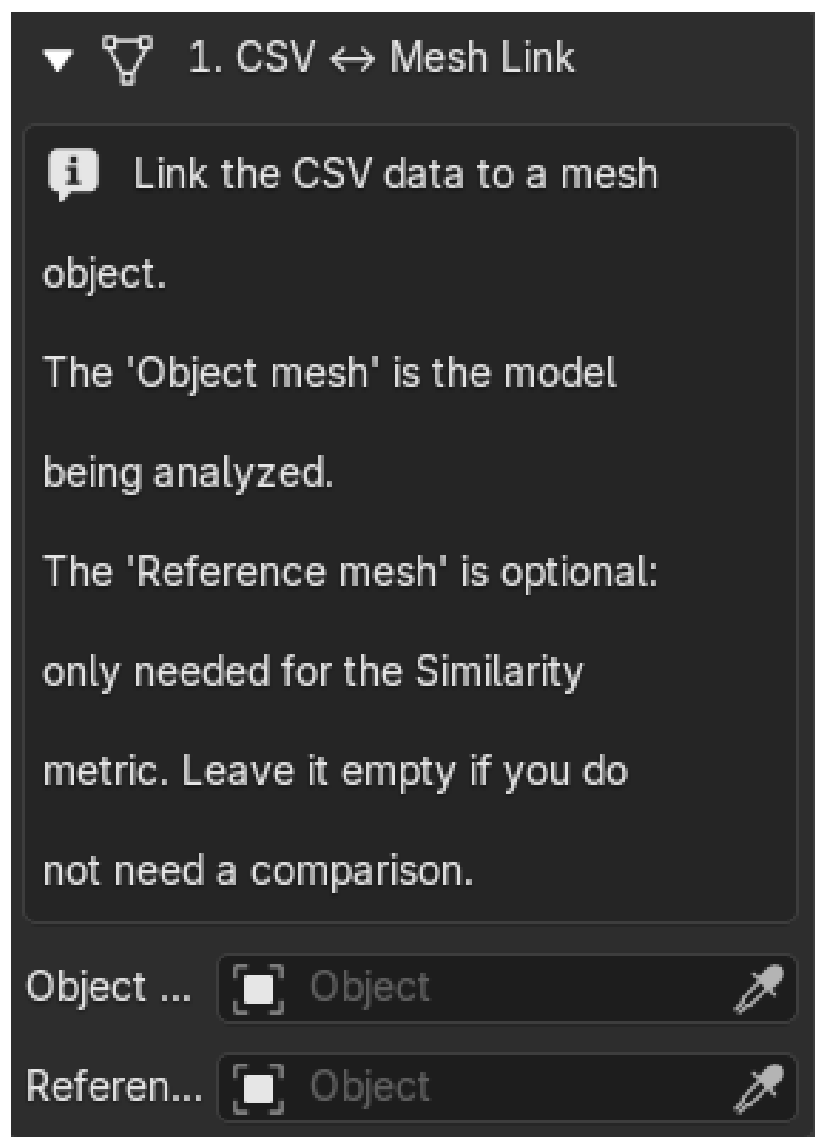


Figura E.22: Selección del objeto activo y el de referencia para el cálculo de métricas geométricas y UV.

La Figura E.23 muestra varios bloques de resultados generados por el cálculo de métricas geométricas y UV. En ellos se incluyen valores relacionados con el mapeado UV, las normales, las transformaciones, la topología y la similitud frente a un objeto de referencia. Si algún valor no aparece, debe comprobarse que el objeto tiene malla válida, que existe una capa UV activa y que se ha seleccionado referencia cuando la métrica lo necesita.

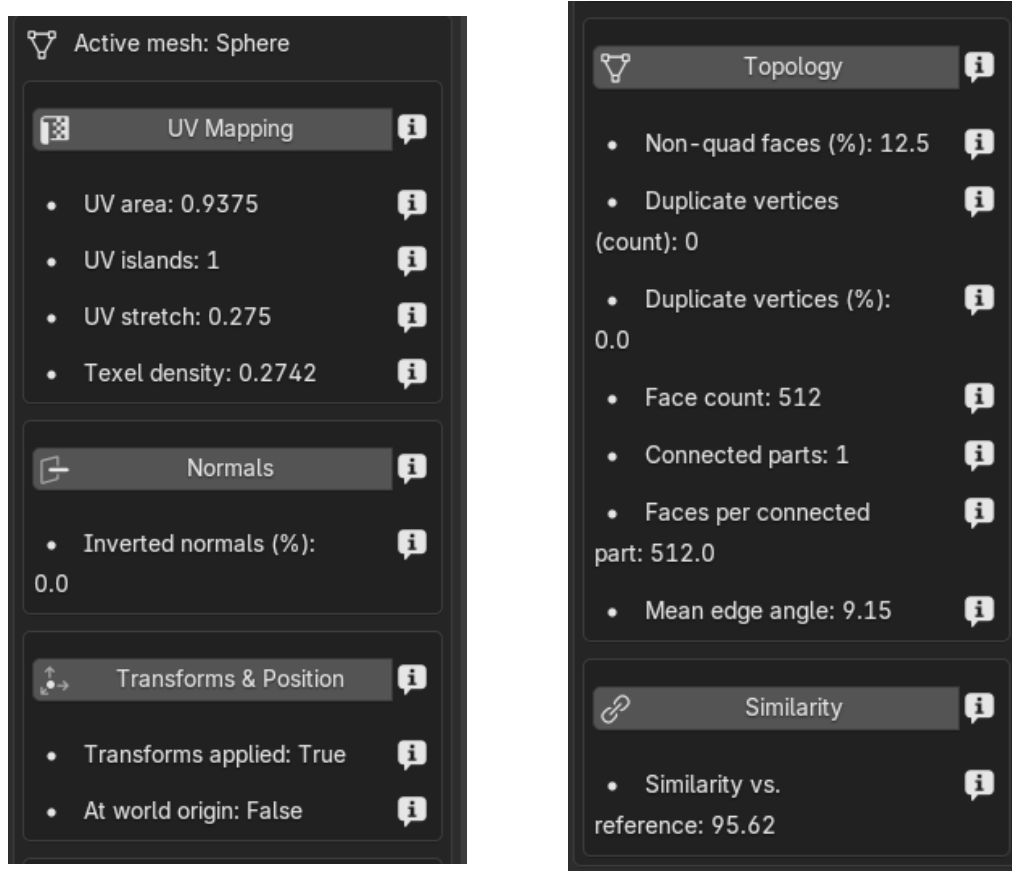


Figura E.23: Resultados de métricas UV, geométricas y de similitud calculadas sobre el objeto seleccionado.

Visualizaciones

El add-on puede generar visualizaciones dentro de Blender, como gráficos temporales, comparaciones entre CSV, superficies, radar o representaciones 3D. Antes de crear una nueva visualización se recomienda eliminar gráficos anteriores para evitar acumulación de objetos en la escena.

La Figura E.24 muestra una visualización temporal. Este tipo de gráfico ayuda a observar cómo evoluciona una métrica a lo largo de la sesión y permite detectar fases de mayor o menor actividad.

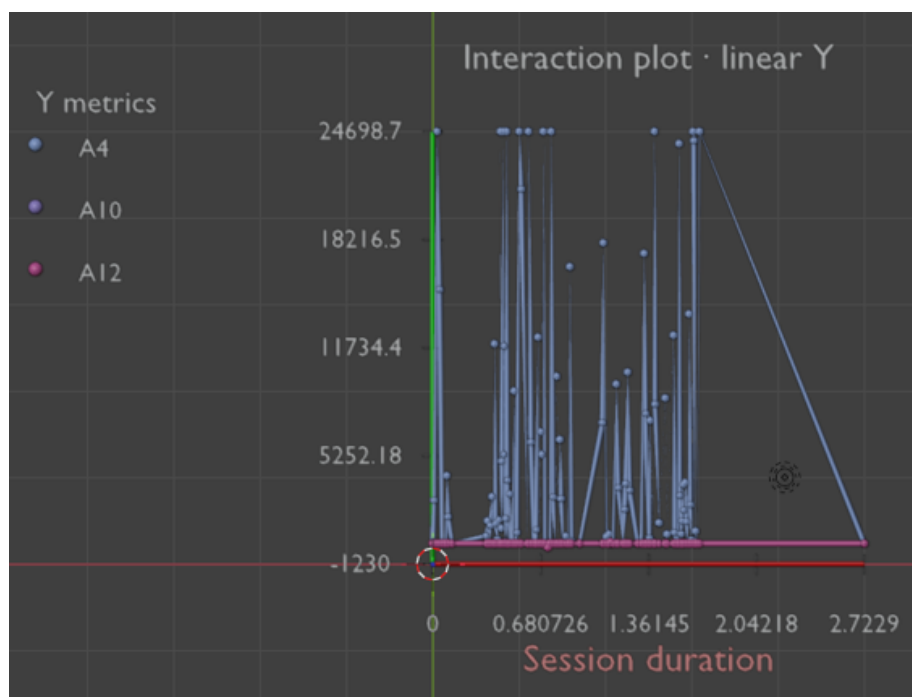


Figura E.24: Visualización temporal generada a partir de los datos de una sesión.

La Figura E.25 representa una comparación entre varios CSV. Debe utilizarse cuando se han exportado varias sesiones con el mismo formato y se desea comparar valores agregados entre registros distintos.

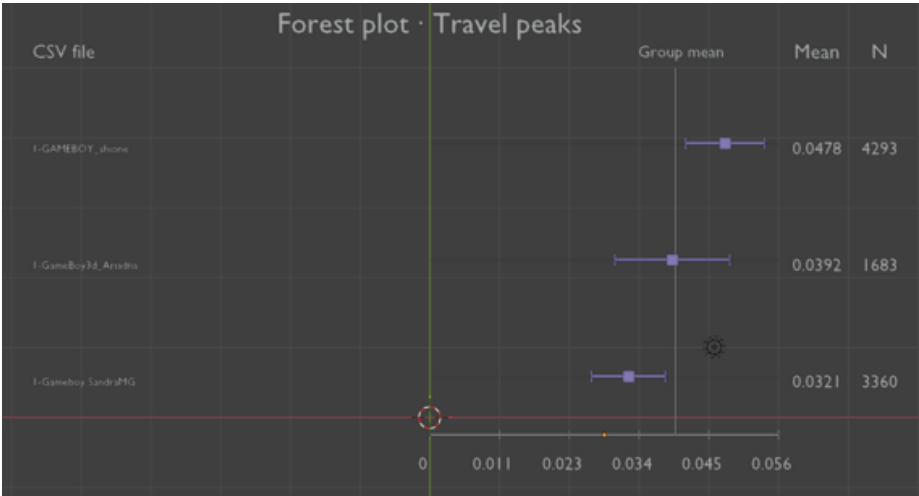


Figura E.25: Comparación visual de varias sesiones a partir de múltiples archivos CSV.

La Figura E.26 muestra un gráfico radar. Esta visualización permite comparar varias métricas normalizadas de una sesión en una única representación, por lo que resulta útil como resumen visual del comportamiento registrado.

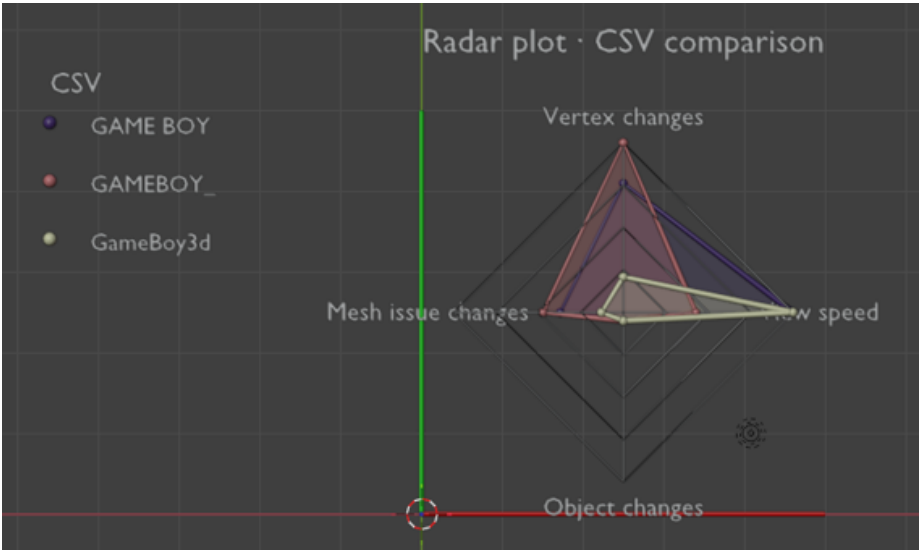


Figura E.26: Gráfico radar generado para representar métricas normalizadas de una sesión.

E.6. Interpretación de métricas

La interpretación debe realizarse con cautela. Una métrica alta no implica necesariamente un error: puede reflejar una sesión larga, una fase intensa de edición o una estrategia de trabajo concreta. Por ello, se recomienda analizar los bloques en conjunto y comparar sesiones con condiciones similares.

Las métricas temporales ayudan a identificar duración y pausas. Las métricas espaciales describen navegación y proximidad. Las métricas de estrategia reflejan cambios de modo, edición, UV y evolución de la malla. Las métricas geométricas y UV deben interpretarse junto con el estado visual del objeto, ya que dependen de la geometría seleccionada y de la existencia de referencia cuando proceda.

E.7. Diagnóstico y resolución de incidencias

Incidencia	Causa probable	Solución recomendada
El logger no se inicia	Auto Run desactivado o add-on no activado.	Revisar preferencias, activar Auto Run si procede y comprobar que el add-on está habilitado.
No aparece el panel	Add-on no instalado o pestaña lateral cerrada.	Activar el add-on y abrir la barra lateral con la tecla N.
No se exporta CSV	El <code>.blend</code> no está guardado o no existe registro incrustado.	Guardar el proyecto, iniciar/detener captura y volver a exportar.
El CSV está vacío	No se detectaron cambios o la captura no llegó a iniciarse.	Repetir sesión comprobando el estado del logger.
El análisis rechaza datos	Cabecera incorrecta, valores no numéricos, NaN o infinitos.	Revisar el CSV y comprobar que procede del logger.
No se calculan métricas UV	El objeto no tiene capa UV activa.	Crear o seleccionar una capa UV válida.
La comparación geométrica falla	Falta objeto de referencia o la malla no es válida.	Seleccionar referencia y objeto objetivo correctamente.
La escena contiene demasiados gráficos	Visualizaciones anteriores no eliminadas.	Usar la opción de limpieza o borrar objetos generados.

Tabla E.3: Problemas frecuentes y soluciones recomendadas.

E.8. Consideraciones éticas y privacidad

El uso del logger debe estar precedido por consentimiento informado. El sistema registra datos técnicos de interacción y modelado, no contenido

personal directo. Aun así, se recomienda tratar los CSV como datos de investigación: almacenarlos en ubicaciones controladas, evitar compartirlos sin autorización y seudonimizar cualquier información que pueda asociarse a personas concretas.

Cuando los datos se utilicen en contexto académico, debe indicarse la finalidad del análisis, el tipo de datos capturados, la forma de almacenamiento y las condiciones de eliminación o reutilización. Si se incorporan participantes externos, deben añadirse los documentos de información y consentimiento correspondientes.

E.9. Recomendaciones para análisis de datos

Para obtener resultados comparables se recomienda utilizar nombres de archivo claros, conservar una carpeta por sesión o participante, no mezclar CSV generados con versiones incompatibles del logger y anotar manualmente cualquier incidencia relevante de la sesión. En comparaciones entre usuarios, las sesiones deberían tener condiciones similares de duración, tarea y versión de Blender.

También se recomienda revisar visualmente los modelos antes de interpretar métricas geométricas y UV. La comparación frente a un objeto de referencia solo es significativa si ambos modelos responden a una misma tarea o a una referencia común.

E.10. Resumen operativo

El flujo básico de uso es el siguiente: instalar ambos add-ons, activar el logger, aceptar el consentimiento, iniciar captura, realizar la sesión de modelado, detener captura, exportar CSV, abrir el add-on de análisis, seleccionar CSV o carpeta, calcular métricas y revisar resultados. Este flujo separa captura y análisis, reduce interferencias durante el modelado y facilita la reutilización posterior de los registros.

Apéndice *F*

Anexo de sostenibilización curricular

F.1. Introducción

El anexo refleja la conexión entre el Trabajo de Fin de Grado y la sostenibilidad académica. El proyecto implica una *suite* compuesta por dos complementos para Blender: uno orientado a documentar la ejecución durante sesiones de modelado 3D y otro destinado a analizar dichos registros, calcular indicadores y presentar resultados dentro del entorno laboral.

La sostenibilidad comprendida en este sentido va más allá del impacto ecológico directo. En realidad, implica prácticas responsables de uso de recursos digitales, el escogimiento de herramientas accesibles, la protección de los datos de los usuarios y la aplicación de dicho sistema en ambientes educativos. A partir de esta óptica, el proyecto tiene conexiones con diversas competencias que la CRUE promueve, sobre todo aquellas relacionadas con el desarrollo de soluciones tecnológicas apropiadas, fiables y amigables con el usuario humano.

F.2. Relación con los Objetivos de Desarrollo Sostenible

El proyecto no aborda los Objetivos de Desarrollo Sostenible de forma directa como finalidad principal, ya que su objetivo central es técnico. Sin embargo, algunas decisiones tomadas durante el desarrollo sí guardan relación con varios ODS. La Tabla [F.1](#) resume esta vinculación.

ODS	Relación con el proyecto
ODS 4: Educación de calidad	La herramienta puede ayudar a analizar procesos de aprendizaje en modelado 3D, no solo el resultado final. Esto permite observar ritmos de trabajo, pausas, cambios de modo y evolución de la escena.
ODS 9: Industria, innovación e infraestructura	Se desarrolla una solución tecnológica modular, basada en add-ons, que puede ampliarse o adaptarse a otros usos relacionados con creación digital, docencia o investigación.
ODS 10: Reducción de las desigualdades	El uso de Blender, Python y bibliotecas abiertas reduce la dependencia de licencias comerciales y facilita que el sistema pueda utilizarse en entornos con recursos limitados.
ODS 12: Producción y consumo responsables	El registro por cambios evita almacenar información repetida de forma innecesaria y reduce el volumen de datos generado durante la captura.
ODS 16: Paz, justicia e instituciones sólidas	El sistema incorpora consentimiento previo y seudonimización del identificador de usuario, lo que refuerza una gestión más transparente de los datos recogidos.

Tabla F.1: Relación del proyecto con distintos Objetivos de Desarrollo Sostenible.

F.3. Sostenibilidad tecnológica

Desde el punto de vista tecnológico, una de las decisiones más importantes ha sido evitar un registro continuo e indiscriminado de información. El add-on *Data Logger 3D* almacena datos cuando detecta cambios relevantes en la escena, en el objeto activo, en el modo de trabajo o en determinadas operaciones del usuario. Este enfoque basado en deltas reduce la cantidad de filas generadas y hace que los archivos resultantes sean más manejables.

También resulta relevante la separación entre captura y análisis. El primer add-on se centra en registrar la actividad de forma no intrusiva, mientras que el segundo procesa los archivos CSV cuando el usuario decide analizarlos. Esta división ayuda a no sobrecargar Blender durante el modelado y facilita que cada parte pueda evolucionar de forma independiente.

El uso de CSV como formato intermedio responde a una decisión sencilla pero práctica. Aunque no es el formato más avanzado, permite revisar los

datos con herramientas comunes, compartirlos fácilmente y reutilizarlos fuera de Blender si fuera necesario. Para el alcance del proyecto, esta solución resulta suficiente y evita introducir dependencias más complejas.

Por último, el proyecto se apoya en tecnologías abiertas como Blender, Python, NumPy y Matplotlib. Esta elección favorece la continuidad del trabajo, ya que se trata de herramientas gratuitas, documentadas y utilizadas por comunidades amplias.

F.4. Sostenibilidad educativa

El proyecto tiene una aplicación clara en contextos de enseñanza del modelado 3D. En este tipo de tareas suele evaluarse principalmente el resultado final: la calidad del modelo, la limpieza de la geometría o el cumplimiento de una entrega. Sin embargo, el proceso seguido por el estudiante queda normalmente menos documentado.

La herramienta desarrollada permite acercarse a ese proceso mediante registros temporales, cambios de modo, operaciones realizadas, evolución topológica y métricas calculadas a partir de los datos. Esta información puede servir como apoyo para detectar hábitos de trabajo, momentos de inactividad, uso de determinadas operaciones o formas de abordar una escena.

No se pretende sustituir la evaluación docente ni automatizar la interpretación del rendimiento. Las métricas deben entenderse como información complementaria, útil para apoyar la observación y la reflexión. En este sentido, el sistema puede ayudar a que el profesorado disponga de más evidencias sobre cómo se ha desarrollado una práctica, especialmente en actividades donde el proceso creativo tiene tanta importancia como el resultado.

La interfaz del segundo add-on también se ha diseñado teniendo en cuenta esta idea. La organización en bloques, el uso de etiquetas breves y la incorporación de *tooltips* buscan facilitar la lectura de resultados sin exigir conocimientos avanzados de análisis de datos. De este modo, las métricas se integran en el propio entorno de Blender y resultan más accesibles para perfiles vinculados al diseño, la animación o la creación 3D.

F.5. Sostenibilidad social y ética

El uso de software libre aporta una dimensión social relevante. Al no depender de licencias de pago, la solución puede utilizarse en centros educa-

tivos, talleres o proyectos personales con menos barreras económicas. Esto facilita que otros estudiantes o docentes puedan probar, adaptar o ampliar el trabajo.

Además, el sistema se ha planteado de forma modular. Esta característica permite reutilizar partes concretas del proyecto, por ejemplo el registro de actividad, el análisis de CSV o las visualizaciones, sin necesidad de rehacer toda la herramienta. Esta reutilización es importante desde una perspectiva sostenible, ya que evita duplicar esfuerzos y facilita futuras mejoras.

En cuanto a la gestión de datos, el proyecto incorpora consentimiento explícito antes de iniciar la captura y utiliza un identificador seudonimizado. Esta medida no elimina todas las cuestiones éticas asociadas al análisis de comportamiento, pero sí introduce una base mínima de transparencia y control por parte de la persona usuaria. Para usos futuros en grupos amplios o estudios formales, sería necesario revisar con más detalle la normativa aplicable y los protocolos de información y almacenamiento.

F.6. Sostenibilidad económica

El proyecto también presenta ventajas económicas. La solución no requiere servidores, plataformas externas ni licencias comerciales. El funcionamiento se apoya en Blender y en archivos locales, lo que reduce el coste de instalación y mantenimiento.

Esta característica es especialmente útil en entornos educativos, donde no siempre es viable incorporar herramientas propietarias o infraestructuras complejas. Además, al tratarse de una *suite* de add-ons, las mejoras futuras pueden incorporarse de manera progresiva: nuevas métricas, nuevos gráficos, cambios en la interfaz o adaptación a otras versiones de Blender.

La reutilización del sistema en prácticas docentes, proyectos de investigación o futuros trabajos académicos podría aumentar su valor a medio plazo. No obstante, esta posibilidad dependerá de que se mantenga una documentación clara y de que el código siga una organización comprensible para otras personas desarrolladoras.

F.7. Limitaciones desde la perspectiva de sostenibilidad

Aunque el proyecto incorpora decisiones coherentes con la sostenibilidad, también presenta limitaciones. La primera tiene que ver con la interpretación de los datos. Las métricas pueden describir actividad, pausas, cambios o características geométricas, pero no explican por sí solas la calidad del aprendizaje ni las razones detrás de cada acción del usuario.

Otra limitación está relacionada con el formato CSV. Para sesiones individuales o conjuntos moderados de datos funciona correctamente, pero podría quedarse corto si se quisiera analizar un número elevado de usuarios o sesiones muy largas. En ese caso sería recomendable valorar formatos más compactos o una base de datos ligera.

También debe tenerse en cuenta que el sistema depende del contexto de Blender y de sus mecanismos internos de eventos, modos y operadores. Cambios en futuras versiones del software podrían requerir ajustes en los add-ons para mantener la compatibilidad.

F.8. Reflexión personal

La realización del proyecto ha permitido comprobar que la sostenibilidad en informática no solo está relacionada con reducir consumo energético o utilizar menos recursos físicos. También tiene que ver con diseñar herramientas que puedan mantenerse, entenderse, reutilizarse y adaptarse a nuevas necesidades.

En este trabajo, la sostenibilidad aparece en decisiones concretas: usar software libre, separar la captura del análisis, registrar solo cambios relevantes, evitar dependencias innecesarias y tratar los datos del usuario con cierta precaución. Son decisiones técnicas, pero tienen consecuencias prácticas sobre la accesibilidad, la privacidad y la continuidad del sistema.

También ha sido importante observar el valor educativo del análisis de procesos. En modelado 3D, muchas decisiones ocurren durante la construcción del modelo y no quedan reflejadas en la entrega final. Registrar parte de esa actividad puede ayudar a entender mejor cómo se trabaja y cómo se aprende, siempre que los datos se interpreten con cuidado y dentro de su contexto.

F.9. Conclusión

En conjunto, el proyecto se relaciona con la sostenibilidad a través de una herramienta abierta, modular y de bajo coste, orientada a registrar y analizar procesos de modelado 3D. Su aportación principal no consiste en resolver de forma directa un problema ambiental o social amplio, sino en aplicar criterios sostenibles dentro del diseño de una solución tecnológica concreta.

El uso de herramientas libres, la captura diferencial de datos, la separación entre módulos, la atención a la privacidad y la posible aplicación educativa del sistema son los aspectos más relevantes desde esta perspectiva. Estas decisiones muestran que incluso en un proyecto técnico y acotado es posible incorporar criterios de sostenibilidad de manera práctica y razonable.

Bibliografía

- [1] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.
- [2] Blender Foundation. Blender python api documentation. <https://docs.blender.org/api/current/>, 2026. Consultado en 2026.
- [3] Mario Botsch, Leif Kobbelt, Mark Pauly, Pierre Alliez, and Bruno Lévy. *Polygon Mesh Processing*. AK Peters, 2010.
- [4] Doug A. Bowman, Ernst Kruijff, Joseph J. LaViola Jr., and Ivan Poupyrev. *3D User Interfaces: Theory and Practice*. Addison-Wesley, 2004.
- [5] Paolo Cignoni, Claudio Rocchini, and Roberto Scopigno. Metro: Measuring error on simplified surfaces. In *Computer Graphics Forum*, volume 17, pages 167–174, 1998.
- [6] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. From data mining to knowledge discovery in databases. *AI Magazine*, 17(3):37–54, 1996.
- [7] Free Software Foundation. GNU General Public License, version 3. <https://www.gnu.org/licenses/gpl-3.0.html>, 2007. Consultado en 2026.
- [8] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

- [9] GitHub. Github documentation. <https://docs.github.com/>, 2026. Consultado en 2026.
- [10] LaTeX Project. Latex documentation. <https://www.latex-project.org/help/documentation/>, 2026. Consultado en 2026.
- [11] Matplotlib Development Team. Matplotlib documentation. <https://matplotlib.org/stable/contents.html>, 2026. Consultado en 2026.
- [12] Wes McKinney. *Python for Data Analysis*. O'Reilly Media, 3 edition, 2022.
- [13] Tamara Munzner. *Visualization Analysis and Design*. CRC Press, 2014.
- [14] Jakob Nielsen. *Usability Engineering*. Morgan Kaufmann, 1994.
- [15] NumPy Developers. Numpy documentation. <https://numpy.org/doc/>, 2026. Consultado en 2026.
- [16] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.
- [17] Roger S. Pressman. *Software Engineering: A Practitioner's Approach*. McGraw-Hill, 2005.
- [18] Python Software Foundation. Python documentation. <https://docs.python.org/3/>, 2026. Consultado en 2026.
- [19] Ken Schwaber and Jeff Sutherland. The scrum guide. <https://scrumguides.org/>, 2020. Consultado en 2026.
- [20] Wil M. P. van der Aalst. *Process Mining: Data Science in Action*. Springer, 2016.
- [21] Guido van Rossum, Barry Warsaw, and Nick Coghlan. PEP 8 – style guide for python code. <https://peps.python.org/pep-0008/>, 2001. Consultado en 2026.